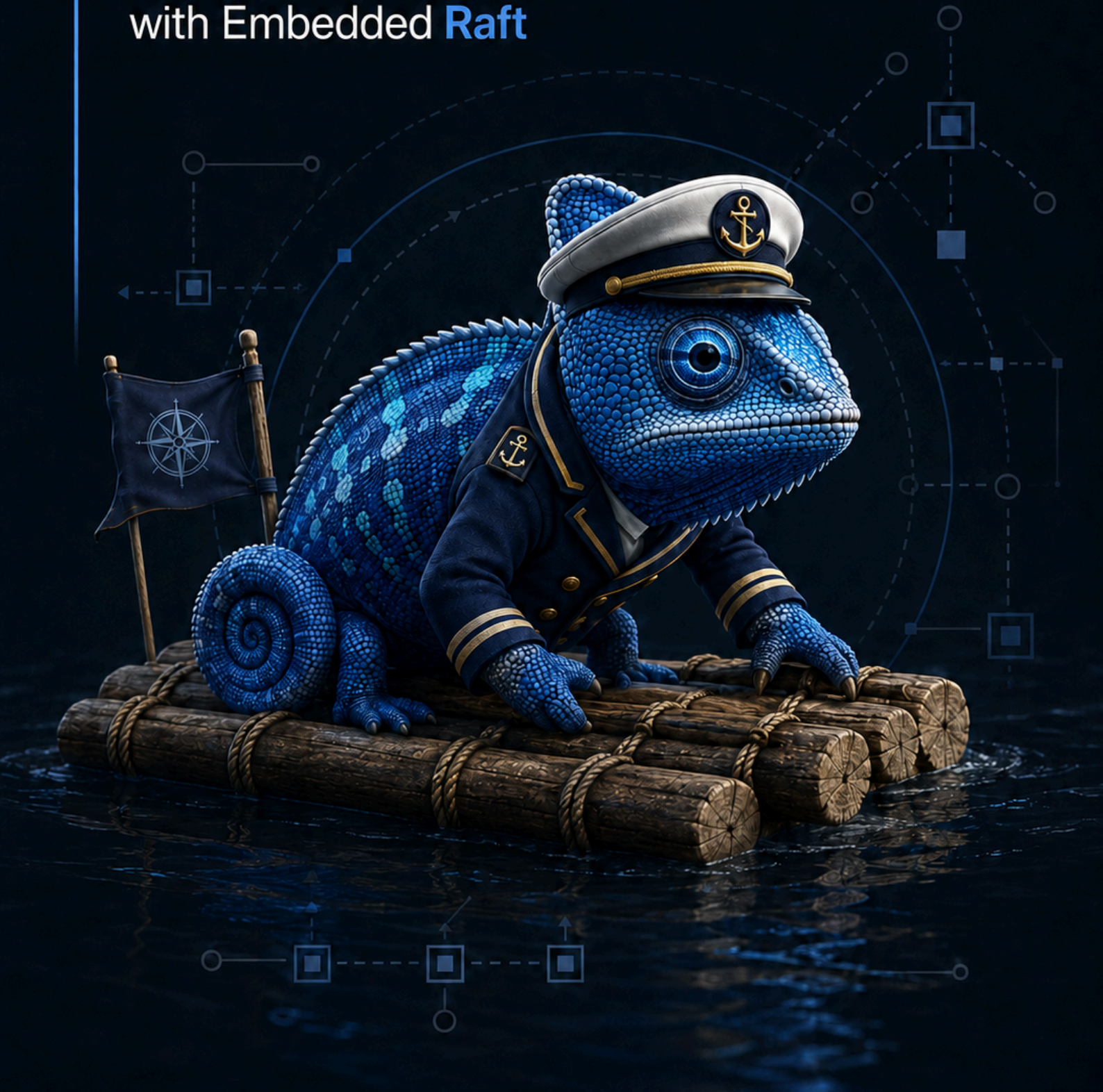
Kommander

Building Distributed Systems in .NET
with Embedded Raft



Kommander: Building Distributed Systems in .NET with Embedded Raft

A practical guide to replicated services, partitioned consensus, and production-grade coordination

Andres Gutierrez

2026

Contents

1 Preface

This book teaches you how to build distributed systems in .NET using Kommander, an open-source embedded Raft consensus library for C#.

Kommander is a library, not a database. It gives your application the machinery for multiple nodes to agree on ordered changes and recover after failures. You keep control of your API, your domain model, your state machine, and your persistence.

The book starts with a concrete problem: three instances of a service must apply the same changes in the same order without losing data when one node crashes. It builds from that problem through working code, distributed systems concepts, failure scenarios, and source-code walkthroughs.

By the end, you will be able to build, operate, and test a production-grade replicated service.

1.1 Who This Book Is For

You are a professional .NET developer who builds backend services. You are comfortable with C#, ASP.NET Core, asynchronous programming, and databases. You may know what replication means without knowing the details of Raft, quorum intersection, or log matching.

1.2 How to Read This Book

The book is organized in six parts. Parts I through III take you from zero to a partitioned replicated service. Part IV covers production operations. Part V explains the implementation internals. Part VI covers testing and verification.

You do not need to read the Raft paper first. The book introduces every concept before it depends on it.

2 The Problem with One Copy

You have a service. It stores state: customer records, account balances, session tokens, configuration entries. The service runs on one machine, backed by one process and one in-memory data structure. It works. Clients send requests, the service updates state, and everything is consistent because there is exactly one copy of the truth.

Then the machine crashes.

Every client that depended on that state is now staring at a connection timeout. If the state lived only in memory, it is gone. If it lived on disk, it is at least unavailable until the machine comes back, assuming it comes back at all. This is the oldest problem in distributed systems: a single point of failure.

The obvious fix is to run more than one copy. But the moment you do, a harder problem appears.

2.1 A Simple Service

Let's make this concrete. Here is a minimal ASP.NET Core key/value service:

```
using System.Collections.Concurrent;

var store = new ConcurrentDictionary<string, string>();

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
app.MapPut("/kv/{key}", (string key, HttpRequest request) =>
{
    using var reader = new StreamReader(request.Body);
    string value = reader.ReadToEndAsync().Result;
    store[key] = value;
    return Results.Ok(new { key, value });
});

app.MapGet("/kv/{key}", (string key) =>
{
    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});

app.MapGet("/kv", () =>
{
    return Results.Ok(store.ToDictionary(x => x.Key, x => x.Value));
});

app.Run();
```

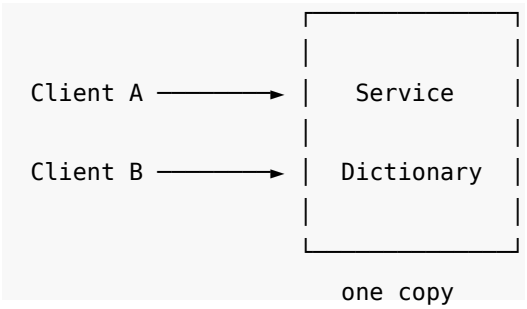
This service does exactly what it should. A client writes a value:

```
PUT /kv/customer:42  →  body: "active"
```

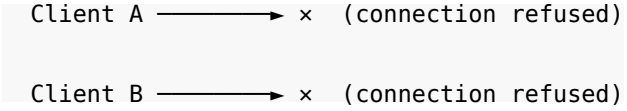
Another client reads it:

```
GET /kv/customer:42  →  { "key": "customer:42", "value": "active" }
```

The dictionary is the entire state of the system. It lives in one process, on one machine. As long as that process is running, every client sees the same state.



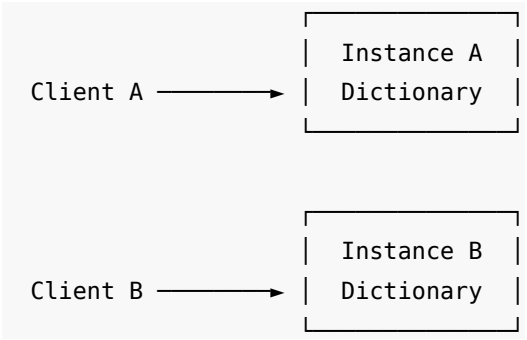
Now kill the process.



Every key is gone. Every client is blocked. The service has a single point of failure in two dimensions: it is a single point of failure for **availability** (no one can reach the data) and for **durability** (the data itself may be lost).

2.2 The Naive Fix: Run Two Copies

The instinct is to run a second instance. Maybe a load balancer distributes requests between them:



This is worse than the single-copy system. Not because it fails more often, but because it fails *silently*.

Client A writes to Instance A:

```
PUT /kv/customer:42  →  "active"     (on Instance A)
```

Client B writes to Instance B:

```
PUT /kv/customer:42  →  "suspended"  (on Instance B)
```

Now Client C reads from Instance A and sees `"active"`. Client D reads from Instance B and sees `"suspended"`. Neither response is wrong. Both instances faithfully served whatever was written to them. But the system as a whole is no longer consistent. Two clients asked the same question and received different answers.

```
Instance A                    Instance B

customer:42 = "active"        customer:42 = "suspended"

      |                             |
      ▼                             ▼
Client C reads "active"      Client D reads "suspended"
```

This is **divergence**. The two copies of state are no longer equivalent, and no mechanism exists to reconcile them. Running two independent copies of a stateful service does not produce redundancy. It produces two independent systems that happen to have the same API.

2.3 Why "Just Copy the Data" Does Not Work

You might think: what if Instance A forwarded every write to Instance B? Then both would have the same data.

This breaks down as soon as two writes arrive at the same time.

Consider two concurrent requests:

```
Client A  →  Instance A:  PUT /kv/balance  "100"
Client B  →  Instance B:  PUT /kv/balance  "200"
```

Each instance applies its own write locally. Then each forwards the write to the other instance. Now both instances have received both writes. But in which order?

```
Instance A:  balance = "100"  →  receives "200"  →  balance = "200"
Instance B:  balance = "200"  →  receives "100"  →  balance = "100"
```

The final state depends on the order in which the forwarded writes arrive. And that order depends on the network, which offers no guarantees. Instance A ends up with `"200"` and Instance B ends up with `"100"`. The system has diverged again, despite both instances having processed the same set of writes.

This is the **ordering problem**. Two instances cannot independently decide the order of concurrent operations and arrive at the same result. The problem is not communication. Both instances received both writes. The problem is that no one decided which write goes first.

2.4 The Real Requirement

What we actually need is a system where:

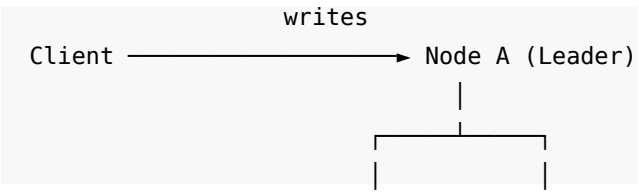
1. Multiple nodes hold copies of the same state.
2. When a client writes to any node, every node eventually reflects that write.
3. All nodes apply writes in the **same order**.
4. If a node crashes, the remaining nodes continue to serve correct state.
5. When the crashed node recovers, it catches up to the current state.

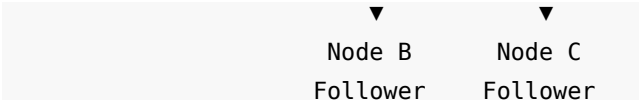
Requirements 1 and 2 describe replication. Requirement 3 is the hard one. Requirement 4 is availability. Requirement 5 is recovery.

Requirement 3, applying writes in the same order, is what distinguishes a replicated system from a collection of independent copies. Without a shared ordering, replication produces divergence. With a shared ordering, every node is a faithful replica of a single logical state machine.

2.5 One Leader, One Order

One approach to establishing a shared order: designate one node as the **leader**. All writes go through the leader. The leader assigns a sequence number to each write and sends the writes, in order, to the other nodes (the **followers**).





This is the core idea behind a family of protocols called **consensus protocols**. The leader decides the order. The followers accept it. If the leader crashes, the remaining nodes elect a new leader and continue.

But even this simple model raises questions:

- What if a follower misses a write because of a network problem?
- What if the leader crashes after sending a write to some followers but not all?
- What if two nodes both believe they are the leader?
- What if a node restarts? How does it learn what it missed?
- How do the remaining nodes know the leader has actually crashed, rather than being temporarily slow?

These are not hypothetical edge cases. They are scenarios that will occur in any system that runs long enough on real hardware and real networks. A protocol that does not handle them will corrupt data silently.

2.6 Consensus

Consensus is the formal name for the problem of getting a group of nodes to agree on a single value (or, more usefully, on an ordered sequence of values). A consensus protocol provides guarantees:

- **Agreement**: all nodes that decide on a value decide on the same value.
- **Validity**: the decided value was actually proposed by some node.
- **Termination**: every non-faulty node eventually decides.

Several consensus protocols exist. Paxos is the oldest and most studied. Raft is newer, designed explicitly to be understandable. Both solve the same fundamental problem, but with different implementation approaches.

This book uses **Raft**. Not because it is better than Paxos, but because it was designed for implementors. The algorithm has a clear separation between leader election, log replication, and safety. Each piece can be understood independently.

2.7 What Raft Provides

At its core, Raft maintains a **replicated log**: an ordered sequence of entries that every node in the cluster agrees on. A leader accepts new entries, assigns them positions in the log, and replicates them to followers. An entry is **committed** once a majority of nodes have confirmed that they hold it. Once committed, the entry will not be lost even if some nodes crash.

Applications use this replicated log as a foundation. Each log entry represents a command: “set key X to value Y”, “delete key Z”, “increment counter W”. Every node processes the committed entries in the same order, and therefore every node arrives at the same state.

```
                    Raft


Client
   |
   ▼
Application API
   |
   ▼
Command
   |
   ▼
Raft Leader
   |
   ▼
Replicated Log
   |
   ▼
Quorum
   |
   ▼
Commit
   |
   ▼
Application Callback
   |
   ▼
Application State Machine
```

This separation is important. Raft replicates the *commands*. The application decides what those commands *mean*. Raft does not know anything about key/value stores, account balances, or session tokens. It guarantees that if a command is committed at position 42 in the log, then every node in the cluster will see the same command at position 42. No node will see a different command at that position.

The application takes it from there.

2.8 Embedded Consensus

Most developers interact with consensus indirectly. They write to etcd, or they use ZooKeeper, or they put state in a database that happens to use Raft internally. In all of these cases, the consensus protocol runs in a separate process or cluster:

```
Your Application
     |
     ▼
External Coordination Service (etcd / ZooKeeper / Consul)
     |
```

```
     ▼
Consensus
```

This works, but it means your application depends on an external system for its most critical state. The external system must be deployed, monitored, upgraded, and secured independently. Network latency between your application and the coordination service is on the critical path of every write.

There is another approach: embed the consensus protocol directly in your application:

```
Your Application Process
     │
     ├── Domain logic
     ├── State machine
     ├── Consensus library
     │       └── Raft
     │
     ▼
Application-owned distributed service
```

In this model, your service *is* the replicated system. There is no external dependency. Consensus runs in-process. Your application controls the API, the state machine, the persistence, and the routing. The consensus library provides the machinery for multiple nodes to agree on ordered commands.

This is the model this book teaches. The library is called **Kommander**, and the next chapter introduces it.

Both approaches can coexist. **Kahuna** is an open-source distributed services platform built on top of Kommander. It provides key/value storage, distributed locking, and coordination features similar to etcd, ZooKeeper, and Consul. Kahuna uses Kommander's embedded Raft internally, but it exposes those capabilities as a standalone service that other applications can connect to. If your use case calls for an external coordination service in the .NET ecosystem, Kahuna fills that role. If you want consensus embedded directly in your own application, Kommander is the library underneath.

2.9 What This Means for the Key/Value Service

Let's revisit the simple service from the beginning of this chapter. By the end of Part I of this book, the in-memory dictionary will be replaced with a replicated state machine:

```
Before (Chapter 1):

Client
   │
   ▼
ASP.NET API
   │
   ▼
```

```
ConcurrentDictionary



After (Chapter 4):

Client
   |
   ▼
ASP.NET API
   |
   ▼
Kommander
   |
   ▼
Replicated command
   |
   ▼
Application state machine (dictionary)
```

The dictionary still exists, but it is no longer the source of truth. Kommander's replicated log is the source of truth. The dictionary is a projection of the committed log, built by applying each committed command, in order, to an initially empty dictionary.

When a node crashes, the remaining nodes continue to serve reads and accept writes. When the crashed node restarts, it replays the committed log entries and rebuilds the dictionary to the current state.

That is the system we will build. The next chapter introduces the tool we will use to build it.

2.10 Summary

- A single-instance stateful service is a single point of failure for both availability and durability.
- Running multiple independent copies does not produce consistency. It produces divergence.
- Replication without ordering guarantees is insufficient. Concurrent writes applied in different orders produce different state.
- Consensus protocols solve the ordering problem by ensuring all nodes agree on the same ordered sequence of commands.
- Raft is a consensus protocol designed for implementors. It separates leader election, log replication, and safety.
- Embedded consensus places the protocol inside your application process, removing the dependency on an external coordination service.
- Kommander is a .NET library that provides embedded Raft consensus. The next chapter introduces it.

3 Introducing Kommander

Chapter 1 showed why a replicated service needs consensus. Every node must apply the same writes in the same order. A leader must coordinate that order. An election must choose a new leader when the old one fails.

This chapter introduces the tool that provides these guarantees: **Kommander**, an open-source embedded Raft consensus library for C# and .NET.

3.1 What Kommander Is

Kommander is a library, not a database. It is not a standalone server. It is not a coordination service that you deploy and connect to. It is a NuGet package that you add to your own application:

```
dotnet add package Kommander
```

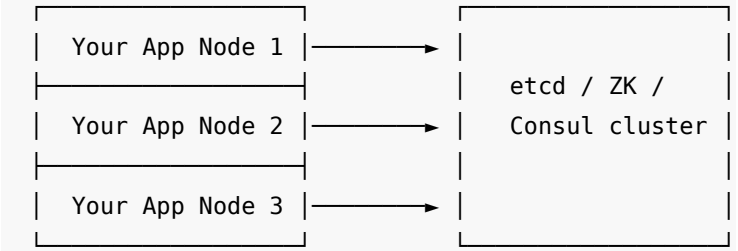
After you add it, your application process contains a Raft node. That node participates in leader election, replicates commands to other nodes, and delivers committed commands to your code through a callback. Your application controls the API, the state machine, and the domain logic. Kommander provides the replication machinery.

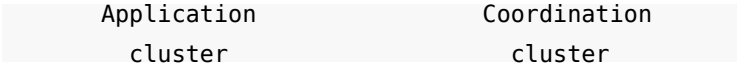
```
Your Application Process
     │
     ├── Your domain logic
     ├── Your state machine
     ├── Kommander (Raft library)
     │      ├── Leader election
     │      ├── Log replication
     │      ├── Quorum tracking
     │      └── Commit coordination
     │
     └── Your API (ASP.NET Core, gRPC, etc.)
```

This model is called **embedded consensus**. The consensus protocol runs inside your process. There is no external service to deploy, monitor, or secure.

3.2 The Alternative: External Coordination

Most developers interact with consensus through an external coordination service. etcd, ZooKeeper, and Consul are common examples. In that model, your application connects to a separate cluster:





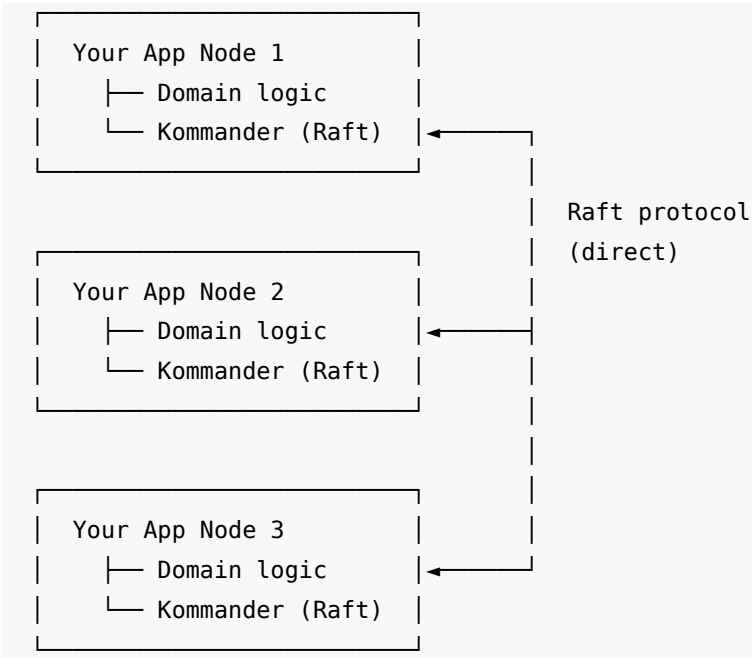
This approach works. Many production systems rely on it. In the .NET ecosystem, **Kahuna** is an open-source distributed services platform that fills this role. Kahuna provides key/value storage, distributed locking, and coordination as a standalone service, similar to etcd, ZooKeeper, and Consul. Under the hood, Kahuna uses Kommander's embedded Raft for its consensus layer.

But the external model has costs:

1. **Operational dependency.** The coordination cluster must be deployed, upgraded, and monitored independently from your application. If it goes down, your application cannot coordinate.
2. **Network latency.** Every write passes through the network to the coordination service and back. That round trip sits on the critical path.
3. **Schema mismatch.** The coordination service has its own data model (key/value, znodes, service entries). You must translate between your domain and the service's model.
4. **Capacity limits.** The coordination service has its own throughput limits. Your application must share them with all other clients of that service.

3.3 The Embedded Approach

With embedded consensus, each application node runs a Raft node inside its own process:



The Raft nodes communicate directly with each other. There is no external service. Each application process is both a Raft participant and a domain service.

This approach has its own trade-offs:

1. **Simplicity.** One deployment, not two. No separate cluster to manage.
2. **Latency.** Consensus happens in-process and over direct node-to-node links. No extra hop.
3. **Responsibility.** Your application now owns the consensus layer. You configure it, tune it, and reason about it.

Kommander is designed for the third point. It exposes the consensus primitives so that you can build on them, while it handles the protocol correctness internally.

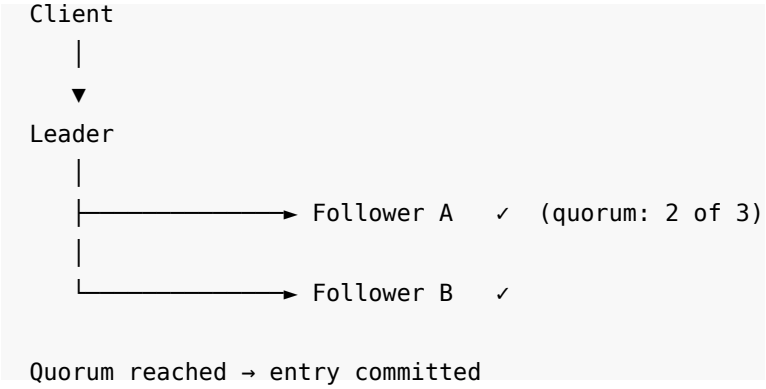
3.4 Raft in 60 Seconds

Raft is the consensus protocol that Kommander implements. Here is the essential model.

A **cluster** is a set of nodes. Each node is in one of three states for a given partition: **leader**, **follower**, or **candidate**.

- The **leader** accepts commands from clients. It assigns each command a position in an ordered log. It sends the log entries to the followers.
- The **followers** receive log entries from the leader and write them to their own logs.
- A **candidate** is a node that is running an election to become leader.

The key guarantee: a log entry is **committed** when a majority of nodes (a **quorum**) have confirmed that they hold it. Once committed, the entry will not be lost, even if some nodes crash.



When the leader crashes, the followers notice that heartbeats have stopped. One of them becomes a candidate, requests votes, and wins an election. The new leader picks up where the old one left off.

This is a simplified view. The full Raft protocol handles many edge cases: split votes, log divergence, network partitions, and more. Later chapters cover these details. For now, the important point is: Raft gives you an ordered, replicated, durable log. Your application builds its state on top of that log.

3.5 What Kommander Provides vs. What You Provide

The boundary between Kommander and your application is clear:

Kommander provides	Your application provides
Leader election	API endpoints (HTTP, gRPC, etc.)
Log replication and quorum	Domain logic and validation
Commit coordination	State machine (the thing that processes commands)
Heartbeats and failure detection	Command serialization (encoding commands as bytes)
Log persistence (WAL adapters)	Request routing (sending writes to the leader)
Cluster membership changes	Read strategy (local reads vs. confirmed reads)

Kommander does not know what your commands mean. It treats every command as a byte array with a string type label. It guarantees that if a command is committed at log position 42, every node in the cluster will see the same command at position 42. Your application decides what that command does.

```
Kommander's view:

Log position 42:  type = "SetValue"
                  data = [0x7B, 0x22, 0x6B, ...]   (opaque bytes)

Your application's view:

Log position 42:  {"key": "customer:42", "value": "active"}
                  → dictionary["customer:42"] = "active"
```

3.6 The Central Mental Model

Every write in a Kommander-based application follows the same flow:

```
1. Client sends a request
        |
        ▼
2. Your API receives the request
        |
        ▼
3. Your code creates a command (a byte array with a type)
        |
        ▼
4. Your code calls ReplicateLogs on the Raft leader
        |
        ▼
5. Kommander replicates the command to followers
        |
```

```
        ▼
6. A quorum of nodes confirms the entry
        |
        ▼
7. Kommander commits the entry
        |
        ▼
8. Kommander fires OnReplicationReceived on each node
        |
        ▼
9. Your callback applies the command to your state machine
```

Steps 4 through 8 are Kommander's responsibility. Steps 1 through 3 and step 9 are yours.

This flow is the same whether you have one node or fifty. It is the same whether you use an in-memory WAL or RocksDB. It is the same whether nodes communicate over gRPC, REST, or in-memory channels.

3.7 Partitions

A single Raft group processes commands sequentially through one leader. For many applications, one group is enough. But when throughput demands grow, a single leader becomes a bottleneck.

Kommander supports **partitioned Raft**. Instead of one Raft group for all commands, you can run multiple independent groups called **partitions**. Each partition elects its own leader, maintains its own log, and operates independently.

```
Node A              Node B              Node C

Partition 1:        Partition 1:        Partition 1:
  Leader              Follower            Follower

Partition 2:        Partition 2:        Partition 2:
  Follower            Leader              Follower

Partition 3:        Partition 3:        Partition 3:
  Follower            Follower            Leader
```

A node can be leader for some partitions and follower for others. This distributes the write load across the cluster.

3.7.1 Partition 0: The System Partition

Kommander reserves **partition 0** as the **system partition**. This partition stores the cluster's own configuration: the partition map, membership roster, and other internal state. Kommander manages it automatically. Your application data uses partitions numbered 1 and above.

The `InitialPartitions` setting in `RaftConfiguration` controls how many application partitions the cluster creates at startup. The default is 1.

Partitions are a scalability feature. Part III of this book covers them in detail. For now, the examples use a single application partition (partition 1).

3.8 The Pluggable Architecture

Kommander separates the consensus core from the infrastructure it depends on. Four interfaces define that boundary:

1. **Storage (IWAL).** The write-ahead log. Kommander ships adapters for RocksDB, SQLite, and in-memory storage. RocksDB is the production choice. In-memory is useful for tests.
2. **Communication (ICommunication).** The network transport. Kommander ships gRPC and REST adapters, plus an in-memory adapter for tests. gRPC is the production choice.
3. **Discovery (IDiscovery).** How a node finds its peers. Static discovery uses a fixed list of endpoints. Dynamic discovery lets you update the list at runtime. Multicast discovery finds peers on the local network.
4. **Time (HybridLogicalClock).** A hybrid clock that combines wall-clock time with a logical counter. Kommander uses it for proposal tickets and causality tracking.

You choose the implementations that match your environment:

Concern	Development/Test	Production
Storage	`InMemoryWAL`	`RocksDbWAL` or `SqliteWAL`
Communication	`InMemoryCommunication`	`GrpcCommunication`
Discovery	`StaticDiscovery`	`StaticDiscovery` or `DynamicDiscovery`
Clock	`HybridLogicalClock`	`HybridLogicalClock`

This pluggability is not just a test convenience. It lets you evolve your infrastructure choices without changing your application logic. The consensus behavior stays the same regardless of which adapters you plug in.

3.9 The Entry Point: RaftManager

`RaftManager` is the class that brings everything together. It implements the `IRaft` interface, which defines all the operations you need: joining a cluster, checking leadership, replicating commands, and leaving the cluster.

Here is how you construct a `RaftManager`:

```
using Kommander;
using Kommander.Communication.Grpc;
using Kommander.Discovery;
```

```
using Kommander.Time;
using Kommander.WAL;
using Microsoft.Extensions.Logging;

ILoggerFactory loggerFactory = LoggerFactory.Create(
    builder => builder.AddConsole()
);

ILogger<IRaft> logger = loggerFactory.CreateLogger<IRaft>();

RaftConfiguration configuration = new()
{
    NodeName = "node-1",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 1
};

IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery([
        new RaftNode("localhost:8002"),
        new RaftNode("localhost:8003")
    ]),
    new RocksDbWAL(
        path: "./data",
        revision: "node-1",
        logger
    ),
    new GrpcCommunication(),
    new HybridLogicalClock(),
    logger
);
```

The constructor takes six arguments:

1. **RaftConfiguration** sets the node identity (name, host, port) and cluster settings (initial partitions, timeouts, intervals). Chapter 3 covers the essential properties. Appendix C is the full reference.
2. **IDiscovery** tells the node where to find its peers.
3. **IWAL** is the write-ahead log that persists Raft entries.
4. **ICommunication** is the network transport for Raft messages.
5. **HybridLogicalClock** provides timestamps for proposals.
6. **ILogger** receives diagnostic log output.

The `RaftManager` does not start any cluster activity at construction time. It only begins participating in the cluster when you call `JoinCluster`.

3.10 The Lifecycle

A Kommander node goes through a simple lifecycle:

```
Construct RaftManager
        |
        ▼
Register callbacks (OnReplicationReceived, OnLogRestored, etc.)
        |
        ▼
Call JoinCluster()
        |
        ▼
Node participates in elections and replication
        |
        ▼
Call LeaveCluster() to shut down gracefully
```

After `JoinCluster`, the node discovers its peers, participates in leader elections, and begins accepting or forwarding commands. After `LeaveCluster`, the node stops participating and optionally disposes its resources.

The two critical callbacks are:

- **OnReplicationReceived**: fires when a committed command is ready for your application to apply. This is where you update your state machine.
- **OnLogRestored**: fires during startup for each committed entry that the WAL already contains. This is how your application rebuilds its state after a restart.

3.11 IRaft: The Interface

`IRaft` is the interface that `RaftManager` implements. It defines the full surface area that your application uses. Here are the methods and events that matter most in Part I:

Member	Purpose
`JoinCluster()`	Start the node and join the cluster
`LeaveCluster(dispose)`	Stop the node and leave the cluster
`AmILeader(partitionId, ct)`	Check if this node leads the given partition
`ReplicateLogs(partitionId, type, data)`	Propose a command for replication
`OnReplicationReceived`	Callback: a committed command is ready to apply
`OnLogRestored`	Callback: a persisted command is being replayed on startup
`OnLeaderChanged`	Callback: a new leader was elected for a partition

The remaining members of `IRaft` handle advanced features: batch replication, manual commit/rollback, partition management, read confirmation, and cluster membership. Later chapters introduce them as needed.

3.12 What This Means for the Key/Value Service

In Chapter 1, the key/value service used a `ConcurrentDictionary` as its only state. The dictionary was the source of truth. When the process crashed, the state was gone.

With Kommander, the dictionary still exists. But it is no longer the source of truth. The Raft log is the source of truth. The dictionary is a local projection built by applying each committed command in order.

```
Before (Chapter 1):                 After (with Kommander):

Client                              Client
   |                                    |
   ▼                                    ▼
ASP.NET API                         ASP.NET API
   |                                    |
   ▼                                    ▼
ConcurrentDictionary                ReplicateLogs()
                                        |
                                        ▼
                                    Raft (leader election, quorum, commit)
                                        |
                                        ▼
                                    OnReplicationReceived callback
                                        |
```

```
                    ▼
          Dictionary (local projection)
```

When a node crashes, the remaining nodes continue to serve reads and accept writes. When the crashed node restarts, Kommander replays the WAL entries through the `OnLogRestored` callback. The application rebuilds the dictionary from those entries.

The next chapter builds this system step by step.

3.13 Summary

- Kommander is a .NET library that embeds Raft consensus in your application process.
- The embedded approach removes the dependency on an external coordination service.
- Raft guarantees an ordered, replicated, durable log. Your application builds state on that log.
- Kommander handles leader election, log replication, quorum, and commit. Your application handles the API, domain logic, and state machine.
- `RaftManager` is the entry point. It takes a configuration, discovery, WAL, communication, clock, and logger.
- Four pluggable interfaces (IWAL, ICommunication, IDiscovery, HybridLogicalClock) let you choose the right implementation for your environment.
- Partition 0 is the system partition. Application data uses partitions numbered 1 and above.
- The next chapter creates a single-node instance and replicates the first command.

4 Your First Replicated Command

This chapter builds a working replicated service. By the end, you will have an ASP.NET Core API that proposes commands through Kommander and applies them in a callback. The service runs on a single node. Even with one node, every step of the Raft lifecycle executes: propose, append to the WAL, reach quorum (trivially, a quorum of one), commit, and deliver the committed entry to your application.

4.1 Setting Up the Project

Create a new ASP.NET Core web project and add the Kommander package:

```
dotnet new web -n ReplicatedStore
cd ReplicatedStore
dotnet add package Kommander
```

Kommander requires a logger, so the default `WebApplication.CreateBuilder` provides everything you need.

4.2 Configuring the Node

Every Kommander node needs a `RaftConfiguration`. The configuration tells the node its own identity (name, host, port) and how many application partitions to create at startup.

```
RaftConfiguration configuration = new()
{
    NodeName = "node-1",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 1
};
```

Five properties matter for this chapter:

Property	Purpose
`NodeName`	A human-readable name for log output. If you do not set `NodeId`, Kommander derives a numeric id from this name.
`Host`	The hostname or IP address that peer nodes use to reach this node.
`Port`	The TCP port for Raft communication.
`InitialPartitions`	The number of application partitions the cluster creates on first startup. Default is 1.
`NodeId`	A numeric identity, unique in the cluster. Optional when `NodeName` is set.

`RaftConfiguration` has many more properties for timeouts, WAL tuning, failure detection, and security. Appendix C covers them all. The defaults are safe for getting started.

4.3 Choosing the Adapters

Chapter 2 described Kommander's four pluggable interfaces. For a single-node development setup, the simplest combination is:

Concern	Adapter	Why
Storage	`InMemoryWAL`	No disk setup. State lives only in memory.
Communication	`InMemoryCommunication`	No network setup. Only works for a single process.
Discovery	`StaticDiscovery`	A fixed list of peer endpoints. Empty for a single node.
Clock	`HybridLogicalClock`	The same clock used in production.

```
using Kommander;
using Kommander.Communication.Memory;
using Kommander.Discovery;
using Kommander.Time;
using Kommander.WAL;

ILoggerFactory loggerFactory = LoggerFactory.Create(
    builder => builder.AddConsole()
);

ILogger<IRaft> logger = loggerFactory.CreateLogger<IRaft>();

IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery([]),
    new InMemoryWAL(logger),
    new InMemoryCommunication(),
    new HybridLogicalClock(),
    logger
);
```

Note the empty list in `StaticDiscovery`. A single-node cluster has no peers. Kommander still runs the full Raft lifecycle. The node holds an election, votes for itself, and wins with a quorum of one.

4.4 Registering the Callback

Before you call `JoinCluster`, register the callback that receives committed commands. The `OnReplicationReceived` event fires each time Kommander commits an entry on a partition.

The callback receives two arguments:

1. The **partition id** (an `int`). This tells you which partition the entry belongs to.
2. A **RaftLog** object. This contains the entry's metadata and payload.

The `RaftLog` fields you will use in this chapter:

Field	Type	Meaning
`Id`	`long`	The log position (monotonically increasing within a partition).
`LogType`	`string`	The application-defined type label you passed to `ReplicateLogs`.
`LogData`	`byte[]`	The application-defined payload you passed to `ReplicateLogs`.
`Type`	`RaftLogType`	The log entry state: `Proposed`, `Committed`, `CommittedCheckpoint`, etc.
`Term`	`long`	The Raft term in which this entry was proposed.

The callback must return `Task<bool>`. Return `true` to indicate that the application processed the entry. Return `false` to signal a problem (Kommander logs a warning but does not retry).

4.5 Building the ReplicatedStore

Here is the complete single-node replicated key/value store. Read through it first. The sections that follow explain each part.

```
using System.Collections.Concurrent;
using System.Text;
using System.Text.Json;
using Kommander;
using Kommander.Communication.Memory;
using Kommander.Discovery;
using Kommander.Time;
using Kommander.WAL;

var builder = WebApplication.CreateBuilder(args);

// --- State machine: a plain dictionary ---
var store = new ConcurrentDictionary<string, string>();

// --- Kommander setup ---
ILogger<IRaft> raftLogger = builder.Services
    .BuildServiceProvider()
    .GetRequiredService<ILoggerFactory>()
```

```
    .CreateLogger<IRaft>();

RaftConfiguration configuration = new()
{
    NodeName = "node-1",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 1
};

IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery([]),
    new InMemoryWAL(raftLogger),
    new InMemoryCommunication(),
    new HybridLogicalClock(),
    raftLogger
);

// --- Callback: apply committed commands to the dictionary ---
raft.OnReplicationReceived += (partitionId, log) =>
{
    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
};

// --- Join the cluster ---
await raft.JoinCluster();

// --- Wait for this node to become leader ---
int partitionId = 1;
using var leaderTimeout = new CancellationTokenSource(TimeSpan.FromSeconds(10));
await raft.WaitForLeader(partitionId, leaderTimeout.Token);

// --- API endpoints ---
```

```
var app = builder.Build();

app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    using var reader = new StreamReader(request.Body);
    string value = await reader.ReadToEndAsync();

    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(key, value)
    );

    var result = await raft.ReplicateLogs(partitionId, "SetValue", payload);

    return result.Success
        ? Results.Ok(new { key, value, logIndex = result.LogIndex })
        : Results.StatusCode(503, new { error = result.Status.ToString() });
});

app.MapGet("/store/{key}", (string key) =>
{
    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});

app.MapGet("/store", () =>
{
    return Results.Ok(store.ToDictionary(x => x.Key, x => x.Value));
});

app.Lifetime.ApplicationStopping.Register(() =>
{
    raft.LeaveCluster(dispose: true).Wait();
});

app.Run();

// --- Command record ---
record SetValueCommand(string Key, string Value);
```

4.6 Walking Through the Code

4.6.1 The State Machine

```
var store = new ConcurrentDictionary<string, string>();
```

The dictionary is the application's state machine. It is the same data structure from Chapter 1. The difference is that writes no longer go directly to the dictionary. Writes go through Kommander first.

4.6.2 The Write Path

When a client sends `PUT /store/customer:42` with body `"active"`, the handler does the following:

1. Serializes the key and value into a JSON byte array.
2. Calls `ReplicateLogs` with partition id 1, type `"SetValue"`, and the serialized payload.
3. Kommander appends the entry to the WAL, replicates it (trivially, to itself), and commits it.
4. Returns the result to the client.

```
var result = await raft.ReplicateLogs(partitionId, "SetValue", payload);
```

`ReplicateLogs` is an async method. It returns a `RaftReplicationResult` with four properties:

Property	Type	Meaning
`Success`	`bool`	`true` if the command was committed.
`Status`	`RaftOperationStatus`	The specific outcome: `Success`, `NodeIsNotLeader`, `ProposalTimeout`, etc.
`LogIndex`	`long`	The log position assigned to this entry.
`TicketId`	`HLCTimestamp`	The hybrid logical clock timestamp that identifies this proposal.

The `autoCommit` parameter defaults to `true`. This means Kommander commits the entry automatically after a quorum confirms it. Chapter 10 covers manual commit and rollback.

4.6.3 The Commit Callback

After `ReplicateLogs` commits the entry, Kommander fires `OnReplicationReceived` on every node that holds the partition. On a single-node cluster, that means this node only.

```
raft.OnReplicationReceived += (partitionId, log) =>
{
    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );
```

```
        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
};
```

The callback checks `log.LogType` to determine the command type. This is the same string you passed to `ReplicateLogs`. The callback deserializes `log.LogData`, extracts the key and value, and applies them to the dictionary.

This is the core pattern for every Kommander application: the write path proposes commands as opaque byte arrays, and the callback interprets them.

4.6.4 The Read Path

```
app.MapGet("/store/{key}", (string key) =>
{
    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});
```

Reads go directly to the dictionary. There is no Raft involvement in the read path. On a single-node cluster, this is safe because the only node is always the leader and always has the latest state.

On a multi-node cluster, reading from a follower can return stale data. Chapter 6 covers strongly consistent reads.

4.6.5 Joining and Leaving

```
await raft.JoinCluster();
```

`JoinCluster` starts the node. It contacts the discovery service, discovers peers (none, in this case), and begins the Raft lifecycle. The node runs an election, votes for itself, and becomes leader.

```
await raft.WaitForLeader(partitionId, leaderTimeout.Token);
```

`WaitForLeader` blocks until a leader is elected for the given partition. On a single node, this happens almost instantly. The method returns the leader's endpoint as a string. In a multi-node cluster, you use this to know which node to send writes to.

```
app.Lifetime.ApplicationStopping.Register(() =>
{
    raft.LeaveCluster(dispose: true).Wait();
});
```

`LeaveCluster` stops the node. The `dispose: true` parameter also disposes the `RaftManager` and its resources. The shutdown hook ensures the node leaves cleanly when the process stops.

4.7 The Lifecycle of One Command

Here is the full sequence when a client writes `PUT /store/balance "100"`:

```
Client
   │
   │  PUT /store/balance  body: "100"
   ▼
ASP.NET Core handler
   │
   │  Serialize: {"Key":"balance","Value":"100"}
   ▼
raft.ReplicateLogs(1, "SetValue", bytes)
   │
   ▼
Kommander (Raft leader, single node)
   │
   ├── 1. Assign log position (e.g. Id = 5)
   ├── 2. Append to WAL as Proposed
   ├── 3. Quorum check: 1 of 1 = majority ✓
   ├── 4. Mark entry as Committed in WAL
   └── 5. Fire OnReplicationReceived
              │
              │  log.LogType = "SetValue"
              │  log.LogData = {"Key":"balance","Value":"100"}
              ▼
         Callback deserializes and applies:
         store["balance"] = "100"
   │
   ▼
Return RaftReplicationResult
   │  Success = true
   │  LogIndex = 5
   ▼
HTTP 200 OK  {"key":"balance","value":"100","logIndex":5}
```

The entire sequence is synchronous from the client's perspective. `ReplicateLogs` does not return until the entry is committed. When the client receives a successful response, the command is in the log and applied to the state machine.

4.8 Testing It

Start the application:

```
dotnet run
```

Write a value:

```
curl -X PUT http://localhost:5000/store/customer:42 \
  -H "Content-Type: text/plain" \
  -d "active"
```

Read it back:

```
curl http://localhost:5000/store/customer:42
```

List all entries:

```
curl http://localhost:5000/store
```

4.9 What Happens When the Process Stops

Stop the process with `Ctrl+C`. Start it again with `dotnet run`.

The dictionary is empty. Every key is gone.

This is expected. `InMemoryWAL` stores the log in memory. When the process exits, the log is lost. The dictionary is a projection of the log. If the log is gone, the dictionary starts empty.

This is the same problem from Chapter 1. But now the fix is different. In Chapter 1, there was no log. The dictionary was the only state. In this architecture, the dictionary is derived from the log. If the log survives the restart, the dictionary can be rebuilt.

4.10 Adding Durability: RocksDbWAL

Replace the `InMemoryWAL` with a `RocksDbWAL`:

```
using Kommander.WAL;

IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery([]),
    new RocksDbWAL(
        path: "./raft-data",
        revision: "node-1",
        raftLogger
    ),
    new InMemoryCommunication(),
    new HybridLogicalClock(),
    raftLogger
);
```

`RocksDbWAL` takes three required arguments:

1. **path**: the directory where RocksDB stores its files.
2. **revision**: a subdirectory name that identifies this node's data. Use a value unique to this node, such as the node name.
3. **logger**: the same logger you pass to `RaftManager`.

An optional fourth argument, `syncWrites`, defaults to `true`. This means every WAL write is flushed to disk before Kommander considers it persisted. Set it to `false` only in development or testing where durability is not important.

4.11 Rebuilding State on Restart: OnLogRestored

When the node starts with a durable WAL, Kommander reads all committed entries from disk and fires the `OnLogRestored` callback for each one. This is how your application rebuilds its state machine after a restart.

Register `OnLogRestored` before calling `JoinCluster`:

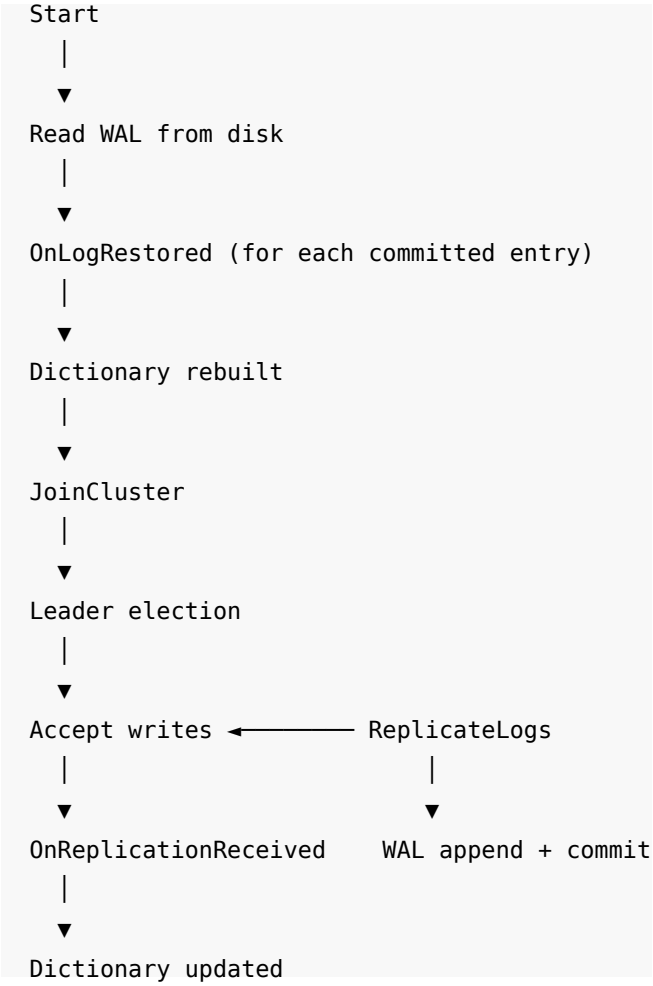
```
raft.OnLogRestored += (partitionId, log) =>
{
    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
};
```

The code is identical to the `OnReplicationReceived` callback. The same logic that applies new commands also rebuilds state from persisted commands. In many applications, you can extract a shared method:

```
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;
```

Now the lifecycle is complete:

1. Node starts. Kommander reads the WAL.
2. `OnLogRestored` fires for each committed entry. The dictionary is rebuilt.
3. Node joins the cluster and becomes leader.
4. New writes arrive. `ReplicateLogs` appends them to the WAL and commits them.
5. `OnReplicationReceived` fires. The dictionary is updated.
6. Node stops. `LeaveCluster` cleans up.
7. Node restarts. Go to step 1.



4.12 Partition 0 vs. Partition 1

You may have noticed that the code uses `partitionId = 1`, not `partitionId = 0`.

Partition 0 is the **system partition**. Kommander reserves it for internal configuration: the partition map, cluster membership, and other coordination state. Your application data belongs on partition 1 and above.

When `InitialPartitions = 1`, Kommander creates one application partition: partition 1. If you set `InitialPartitions = 8`, Kommander creates partitions 1 through 8. Partition 0 always exists regardless of this setting.

The `OnReplicationReceived` callback fires for all partitions, including partition 0. Kommander's internal entries on partition 0 use the reserved type `_RaftSystem`. Your callback should either check for your own types or ignore entries with `LogType` starting with `_Raft`.

4.13 Summary

- `RaftManager` is constructed with a configuration, discovery, WAL, communication, clock, and logger.
- `JoinCluster` starts the node. `LeaveCluster` stops it.
- `ReplicateLogs` proposes a command. The command is a byte array with a string type label.
- `OnReplicationReceived` fires when a command is committed and ready to apply. The application decides what the command means.
- Even a single node executes the full Raft lifecycle: propose, WAL append, quorum, commit, callback.
- `InMemoryWAL` is simple but loses state on restart. `RocksDbWAL` persists state to disk.
- `OnLogRestored` fires on startup for each persisted entry. Use it to rebuild the application state machine.
- Application data uses partition 1 and above. Partition 0 is reserved for Kommander's internal state.
- The next chapter adds two more nodes to create a real cluster.

5 Running a Three-Node Cluster

Chapter 3 built a replicated store on a single node. A single node does not tolerate failure. If that node crashes, every client is blocked. This chapter adds two more nodes. A three-node cluster tolerates one failure: any two nodes can form a quorum and continue operating while the third is down.

5.1 Why Three Nodes

Raft requires a **majority** (a quorum) of nodes to agree before an entry is committed. The quorum formula is `floor(N/2) + 1`, where N is the number of nodes in the cluster.

Nodes	Quorum	Failures tolerated
1	1	0
2	2	0
3	2	1
5	3	2
7	4	3

Two nodes are worse than one for fault tolerance. Both must agree for a commit, so a single failure still blocks the cluster. Three is the minimum for useful redundancy.

The general rule: a cluster of `2F + 1` nodes tolerates `F` failures.

5.2 What Changes from Chapter 3

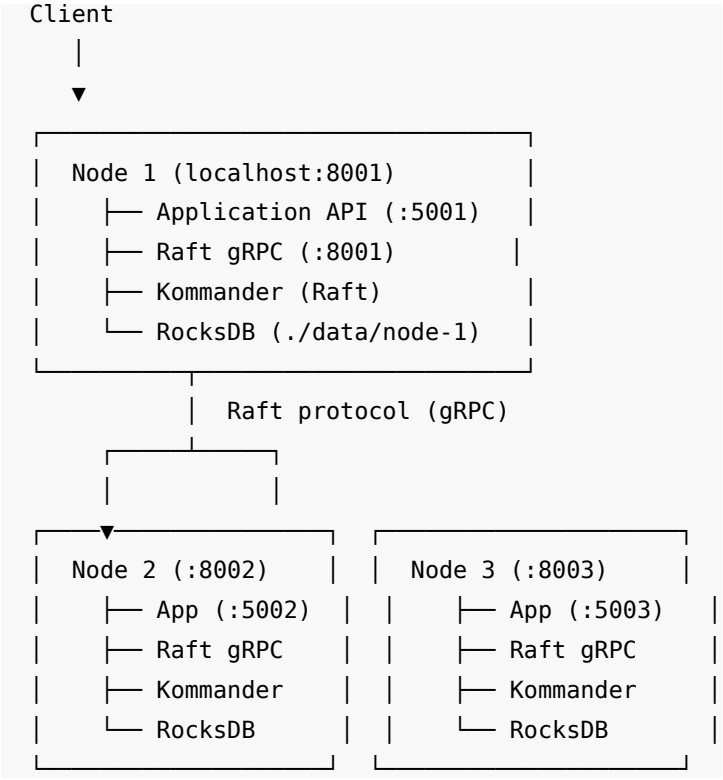
The single-node setup used `InMemoryCommunication` (no network) and `InMemoryWAL` (no disk). A three-node cluster must communicate over the network and persist state to survive restarts.

Concern	Chapter 3 (single node)	Chapter 4 (three nodes)
Communication	`InMemoryCommunication`	`GrpcCommunication`
Storage	`InMemoryWAL`	`RocksDbWAL`
Discovery	`StaticDiscovery([])`	`StaticDiscovery` with peer endpoints
Hosting	Minimal API only	ASP.NET Core with gRPC endpoints

5.3 The Three-Node Architecture

Each node runs an ASP.NET Core process that hosts two sets of endpoints:

1. **Application endpoints**: the `PUT /store/{key}` and `GET /store/{key}` routes that clients use.
2. **Raft endpoints**: gRPC services that Kommander uses for leader election, log replication, and heartbeats.



5.4 The Complete Node Code

The following program runs one node. You launch three instances of this program with different arguments for the node name, Raft port, and HTTP port. The code is longer than Chapter 3 because it includes gRPC hosting and leader-aware routing.

```
using System.Collections.Concurrent;
using System.Text;
using System.Text.Json;
using Kommander;
using Kommander.Communication.Grpc;
using Kommander.Data;
using Kommander.Discovery;
using Kommander.Time;
using Kommander.WAL;

// --- Parse command-line arguments ---
string nodeName = args.Length > 0 ? args[0] : "node-1";
int raftPort = args.Length > 1 ? int.Parse(args[1]) : 8001;
int httpPort = args.Length > 2 ? int.Parse(args[2]) : 5001;

// --- All three Raft endpoints ---
int[] allRaftPorts = [8001, 8002, 8003];

// --- ASP.NET Core builder ---
var builder = WebApplication.CreateBuilder(args);
builder.WebHost.UseUrls($"http://localhost:{httpPort}");

// Add gRPC services for Kommander
builder.Services.AddKommanderGrpc();

// --- State machine ---
var store = new ConcurrentDictionary<string, string>();

// --- Logger ---
ILogger<IRaft> raftLogger = builder.Services
    .BuildServiceProvider()
    .GetRequiredService<ILoggerFactory>()
    .CreateLogger<IRaft>();

// --- Kommander configuration ---
RaftConfiguration configuration = new()
{
    NodeName = nodeName,
    Host = "localhost",
    Port = raftPort,
```

```
    InitialPartitions = 1,
    GrpcScheme = "http://",
    HttpScheme = "http://"
};

// --- Peer discovery: list every other node ---
List<RaftNode> peers = allRaftPorts
    .Where(p => p != raftPort)
    .Select(p => new RaftNode($"localhost:{p}"))
    .ToList();

// --- Create the Raft node ---
IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery(peers),
    new RocksDbWAL(
        path: "./raft-data",
        revision: nodeName,
        raftLogger
    ),
    new GrpcCommunication(),
    new HybridLogicalClock(),
    raftLogger
);

// --- Apply committed commands ---
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;
```

```
// --- Log leader changes ---
raft.OnLeaderChanged += (partitionId, leaderEndpoint) =>
{
    Console.WriteLine(
        $"Partition {partitionId}: leader is now {leaderEndpoint}"
    );
    return Task.FromResult(true);
};

// --- Join the cluster ---
await raft.JoinCluster();

// --- Build the application ---
var app = builder.Build();

// Map Kommander gRPC endpoints
app.MapGrpcRaftRoutes();

// --- Application endpoints ---
int partitionId = 1;

app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    // Check if this node is the leader
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)
    {
        string? leaderEndpoint = raft.GetPartitionLeaderHint(partitionId);
        if (leaderEndpoint is not null)
        {
            // Find the HTTP port for the leader
            int leaderRaftPort = int.Parse(leaderEndpoint.Split(':')[1]);
            int leaderHttpPort = leaderRaftPort - 3000;
            return Results.Redirect(
                $"http://localhost:{leaderHttpPort}/store/{key}",
                permanent: false,
                preserveMethod: true
            );
        }

        return Results.Json(
            new { error = "No leader available" },
            statusCode: 503
        );
```

```
    }

    using var reader = new StreamReader(request.Body);
    string value = await reader.ReadToEndAsync();

    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(key, value)
    );

    var result = await raft.ReplicateLogs(partitionId, "SetValue", payload);

    return result.Success
        ? Results.Ok(new { key, value, logIndex = result.LogIndex })
        : Results.Json(
            new { error = result.Status.ToString() },
            statusCode: 503
        );
});

app.MapGet("/store/{key}", (string key) =>
{
    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});

app.MapGet("/store", () =>
{
    return Results.Ok(store.ToDictionary(x => x.Key, x => x.Value));
});

// --- Leader info endpoint ---
app.MapGet("/leader", async () =>
{
    bool isLeader = await raft.AmILeader(partitionId, default);
    string? leaderHint = raft.GetPartitionLeaderHint(partitionId);
    return Results.Ok(new
    {
        node = nodeName,
        isLeader,
        leaderHint
    });
});
```

```
app.Lifetime.ApplicationStopping.Register(() =>
{
    raft.LeaveCluster(dispose: true).Wait();
});

app.Run();

record SetValueCommand(string Key, string Value);
```

5.5 Walking Through the New Parts

5.5.1 gRPC Hosting

Two lines wire Kommander's gRPC services into the ASP.NET Core pipeline:

```
builder.Services.AddKommanderGrpc();
```

This registers the gRPC service with the dependency injection container. It also configures compression support for snapshot transfers.

```
app.MapGrpcRaftRoutes();
```

This maps the Raft gRPC service to the application's endpoint routing. After this call, peer nodes can reach this node's Raft endpoints over gRPC.

Both methods come from the `Kommander.Communication.Grpc` namespace.

5.5.2 Transport Scheme

```
GrpcScheme = "http://",
HttpScheme = "http://"
```

The default scheme is `https://`. For local development without TLS certificates, set both to `http://`. Chapter 20 covers transport security for production deployments.

When you use the `http://` scheme with gRPC, you also need to enable unencrypted HTTP/2. Add this environment variable before starting the process:

```
export ASPNETCORE_URLS="http://localhost:8001"
```

Or pass it on the command line. The builder's `UseUrls` call in the code above handles the HTTP endpoint. The Raft gRPC port is separate and configured through `RaftConfiguration.Port`.

5.5.3 Static Discovery with Peers

```
List<RaftNode> peers = allRaftPorts
    .Where(p => p != raftPort)
    .Select(p => new RaftNode($"localhost:{p}"))
    .ToList();
```

Each node lists every other node's Raft endpoint. Node 1 lists nodes 2 and 3. Node 2 lists nodes 1 and 3. Node 3 lists nodes 1 and 2. `StaticDiscovery` takes this list and provides it to Kommander when it needs to contact peers.

A `RaftNode` takes a single argument: the endpoint string in `host:port` format.

5.5.4 Leader-Aware Routing

```
bool isLeader = await raft.AmILeader(partitionId, default);
if (!isLeader)
{
    string? leaderEndpoint = raft.GetPartitionLeaderHint(partitionId);
    // ... redirect to the leader
}
```

Only the leader can accept writes through `ReplicateLogs`. If a write arrives at a follower, the handler checks `AmILeader` and redirects the client to the leader. `GetPartitionLeaderHint` returns the current leader's Raft endpoint as a best-effort hint.

This is a simple routing strategy. Chapter 5 covers more advanced approaches.

5.5.5 OnLeaderChanged

```
raft.OnLeaderChanged += (partitionId, leaderEndpoint) =>
{
    Console.WriteLine(
        $"Partition {partitionId}: leader is now {leaderEndpoint}"
    );
    return Task.FromResult(true);
};
```

This callback fires every time a new leader is elected for a partition. The two arguments are the partition id and the new leader's endpoint. You will see this fire during startup (initial election) and whenever a leader fails and a new one takes over.

5.6 Starting the Cluster

Open three terminals. In each one, start a node with different arguments:

Terminal 1 (Node 1):

```
dotnet run -- node-1 8001 5001
```

Terminal 2 (Node 2):

```
dotnet run -- node-2 8002 5002
```

Terminal 3 (Node 3):

```
dotnet run -- node-3 8003 5003
```

Within a few seconds, the three nodes discover each other, run an election, and one becomes leader. You will see `OnLeaderChanged` messages in each terminal. All three nodes report the same leader.

5.7 Leader Election

When the three nodes start, they go through the following sequence:

```
Time ──────────────────────────────────────────►

Node 1:  starts  |  discovers peers  |  waits  |  votes  |  becomes leader
Node 2:  starts  |  discovers peers  |  waits  |  votes  |  becomes follower
Node 3:  starts  |  discovers peers  |  waits  |  votes  |  becomes follower
```

Each node waits for a random duration between `StartElectionTimeout` (default: 2000 ms) and `EndElectionTimeout` (default: 4000 ms). The randomization prevents all three nodes from starting elections at the same time.

The first node whose timer expires becomes a **candidate**. It increments its **term** (a monotonically increasing number that identifies each election round) and sends vote requests to the other two nodes. If it receives votes from a majority (two of three, including itself), it becomes the leader.

After winning, the leader sends heartbeats every `HeartbeatInterval` (default: 500 ms) to the followers. Each heartbeat resets the follower's election timer. As long as the heartbeats arrive, the followers do not start new elections.

5.8 Writing to the Cluster

Send a write to any node. If you hit the leader, the write succeeds directly. If you hit a follower, the HTTP response redirects you to the leader.

```
# Write to the leader (assume node-1 is leader)
curl -X PUT http://localhost:5001/store/greeting \
  -H "Content-Type: text/plain" \
  -d "hello from the cluster"
```

The leader proposes the command, replicates it to the two followers, and waits for a quorum. Two of three nodes must confirm the entry. After the quorum confirms, Kommander commits the entry and fires `OnReplicationReceived` on all three nodes.

```
Client
   │
   │  PUT /store/greeting
   ▼
Node 1 (Leader)
   │
   ├── 1. Append to local WAL
   │
   ├── 2. Send AppendEntries to Node 2 ──► Node 2 appends to WAL ──► ACK
   │
   ├── 3. Send AppendEntries to Node 3 ──► Node 3 appends to WAL ──► ACK
   │
   ├── 4. Quorum reached (2 of 3)
   │
   ├── 5. Commit
```

```
    │
    └── 6. OnReplicationReceived fires on all 3 nodes
```

5.9 Reading from Any Node

Read the value from any node:

```
curl http://localhost:5001/store/greeting
curl http://localhost:5002/store/greeting
curl http://localhost:5003/store/greeting
```

All three return the same value. Every node applied the committed command to its local dictionary.

Note: these reads are not strongly consistent. A follower may briefly lag behind the leader. For most applications, this is acceptable. Chapter 6 explains how to get strongly consistent reads when you need them.

5.10 Surviving a Follower Failure

Stop Node 3 with `Ctrl+C`. The cluster continues operating:

```
curl -X PUT http://localhost:5001/store/status \
  -H "Content-Type: text/plain" \
  -d "two-node quorum"
```

This write succeeds. Node 1 (leader) and Node 2 (follower) form a quorum of two. That is enough for a three-node cluster.

Restart Node 3:

```
dotnet run -- node-3 8003 5003
```

Node 3 rejoins the cluster as a follower. The leader detects that Node 3 is behind and sends the missing log entries. This is called **catch-up** or **backfill**. After catch-up completes, Node 3 has the same state as the other two nodes.

```
curl http://localhost:5003/store/status
# Returns: {"key":"status","value":"two-node quorum"}
```

The entry that was written while Node 3 was down is now available on Node 3.

5.11 Surviving a Leader Failure

Stop the leader (Node 1):

```
  Node 1 (Leader):   stopped

  Node 2 (Follower): heartbeats stop arriving...
  Node 3 (Follower): heartbeats stop arriving...
```

After the election timeout (2 to 4 seconds by default), one of the remaining nodes notices that heartbeats have stopped. It becomes a candidate, starts a new election, and wins:

```
  Time ──────────────────────────────────────────────►

  Node 1:  × (stopped)
  Node 2:  no heartbeat | timeout | candidate | requests votes | leader ✓
  Node 3:  no heartbeat | timeout |             | votes for Node 2
```

You will see `OnLeaderChanged` messages in the terminals for Node 2 and Node 3. The new leader accepts writes:

```
# Write to the new leader
curl -X PUT http://localhost:5002/store/recovery \
  -H "Content-Type: text/plain" \
  -d "leader failover complete"
```

Restart Node 1:

```
dotnet run -- node-1 8001 5001
```

Node 1 rejoins as a follower. It catches up on the entries it missed and resumes normal operation. It does not become leader again unless the current leader fails.

5.12 The Quorum Boundary

A three-node cluster tolerates one failure. It does not tolerate two. Stop two nodes and try to write:

```
# Stop Node 2 and Node 3. Only Node 1 (if it was leader) remains.
curl -X PUT http://localhost:5001/store/test \
  -H "Content-Type: text/plain" \
  -d "should fail"
```

The write hangs or times out. One node out of three is not a quorum. Kommander cannot commit the entry because a majority cannot confirm it.

This is correct behavior. The cluster is protecting you from split-brain scenarios. If it allowed one node to commit alone, and the other two nodes were alive but partitioned away, the cluster would have two independent histories.

Start the other nodes again. The cluster recovers, the pending write (if it was still in flight) may succeed or the client can retry.

5.13 Terms

Each election increments the **term**, a monotonically increasing counter. The term serves two purposes:

1. **Ordering elections.** If a node receives a message with a higher term, it knows a newer election has occurred and steps down.
2. **Detecting stale leaders.** A leader in term 3 cannot overwrite entries from term 4.

You can observe terms in the log output. The first election might be term 1. After a leader failure and re-election, the new leader operates in term 2 (or higher, if there were split votes).

Terms are internal to Raft. Your application code does not interact with them directly. But they appear in log output and in the `RaftLog.Term` field, which is useful for debugging.

5.14 Data Directory Layout

After running the three-node cluster, the `raft-data` directory contains one subdirectory per node:

```
raft-data/
  node-1/    ← RocksDB files for Node 1
  node-2/    ← RocksDB files for Node 2
  node-3/    ← RocksDB files for Node 3
```

Each node has its own RocksDB instance. The `revision` parameter in the `RocksDbWAL` constructor determines the subdirectory name. In a production deployment, each node runs on a separate machine with its own data directory.

To reset the cluster, stop all nodes and delete the `raft-data` directory:

```
rm -rf ./raft-data
```

5.15 Summary

- A three-node cluster tolerates one node failure. Any two nodes form a quorum.
- `GrpcCommunication` provides the network transport. `MapGrpcRaftRoutes()` wires the Raft gRPC service into ASP.NET Core.
- `StaticDiscovery` lists the other nodes' Raft endpoints.
- `RocksDbWAL` persists the WAL to disk. State survives restarts.
- Only the leader accepts writes through `ReplicateLogs`. Followers must redirect or forward.
- Leader election happens automatically when the current leader is unreachable. The election timeout is randomized to avoid collisions.
- `OnReplicationReceived` fires on all three nodes for every committed entry. Each node maintains its own copy of the state machine.
- `OnLeaderChanged` fires whenever a new leader is elected for a partition.
- A stopped node catches up automatically when it rejoins the cluster. The leader sends the missing entries.
- The cluster does not accept writes when fewer than a quorum of nodes are available. This prevents split-brain divergence.
- The next chapter covers leadership and request routing in more detail.

6 Leadership and Request Routing

Chapter 4 built a three-node cluster and added a simple leader redirect to the write handler. This chapter goes deeper. It explains what leadership means in Raft, why local leadership checks can be wrong, and how to build routing that handles leader changes correctly.

6.1 Only the Leader Accepts Writes

In Raft, only the leader for a partition can propose new log entries. If a follower receives a write, it cannot replicate it. The follower must either reject the request or forward it to the leader.

When you call `ReplicateLogs` on a follower in the default full-replication mode, Kommander returns a result with `Status = NodeIsNotLeader`. The entry is not proposed, not replicated, and not committed. Your application must decide what to do: redirect the client, forward the request internally, or return an error.

```
Client
   │
   │  PUT /store/key
   ▼
Follower Node
   │
   │  raft.ReplicateLogs(...)
   │  → result.Status = NodeIsNotLeader
   ▼
Application decides:
   Option A: redirect client to leader (HTTP 307)
   Option B: proxy the request to the leader
   Option C: return 503 with a hint
```

6.2 Three Ways to Check Leadership

Kommander provides three methods to check whether the local node leads a partition. Each has a different cost and a different guarantee.

6.2.1 AmILeaderQuick

```
bool isLeader = await raft.AmILeaderQuick(partitionId);
```

`AmILeaderQuick` reads a published in-memory field. It does not contact the partition executor. It does not contact other nodes. The cost is a single volatile read.

The guarantee: `AmILeaderQuick` returns the node's **local belief** about leadership. If the node believes it is the leader, it returns `true`. If the node knows it is not the leader, it returns `false`.

The risk: a minority-partitioned leader still believes it leads. `AmILeaderQuick` returns `true` on that node even though it cannot form a quorum. The node lost contact with the majority, but it has not yet received a higher-term message that would cause it to step down.

Use `AmILeaderQuick` for fast pre-checks where a false positive is acceptable. For example, you might use it to decide which node should run a periodic background task. If two nodes both run the task briefly during a leader transition, the system tolerates that.

6.2.2 AmILeader

```
bool isLeader = await raft.AmILeader(partitionId, cancellationToken);
```

`AmILeader` also reads published local state, but it checks the partition's role snapshot in addition to the leader endpoint. It is still a local check with no network round-trip. The cost is slightly higher than `AmILeaderQuick`, but still cheap.

The guarantee is the same: local belief. A minority-partitioned leader returns `true` from `AmILeader` just as it does from `AmILeaderQuick`. Neither method contacts other nodes to confirm.

Use `AmILeader` when you want the most current local state. It is the standard check for the write path, as shown in Chapter 4.

6.2.3 ConfirmLeadershipAsync

```
bool confirmed = await raft.ConfirmLeadershipAsync(
    partitionId,
    cancellationToken
);
```

`ConfirmLeadershipAsync` contacts a quorum of nodes to confirm that this node is still the leader. It implements the **read-index protocol** from the Raft dissertation (section 6.4). This is the only leadership check that detects a minority-partitioned leader.

The guarantee: if `ConfirmLeadershipAsync` returns `true`, a majority of nodes have confirmed that this node leads the partition in the current term, and all committed entries up to that point have been applied locally. A local read after a `true` result is **linearizable**.

The cost: one quorum round-trip. But Kommander optimizes this in two ways:

1. **Coalescing.** Multiple concurrent callers share a single in-flight ack round. If 100 clients call `ConfirmLeadershipAsync` at the same time, Kommander sends one round of heartbeats and returns the result to all 100 callers.
2. **Freshness caching.** A confirmation that completed within the last heartbeat interval is reused. Steady-state cost is approximately one quorum round-trip per heartbeat interval, regardless of read volume.

If the confirmation times out (the node cannot reach a quorum within `LeadershipConfirmationTimeout`, default 2 seconds), the method returns `false`. This is the correct behavior for a minority-partitioned leader: it cannot reach a quorum, so the confirmation fails, and your application can redirect the client.

6.2.4 Choosing the Right Check

Method	Cost	Guarantee	Use when
`AmILeaderQuick`	One volatile read	Local belief	Fast pre-filter; false positive is tolerable
`AmILeader`	Local state read	Local belief (more current)	Standard write-path gate
`ConfirmLeadershipAsync`	Quorum round-trip	Quorum-confirmed	Strongly consistent reads (Chapter 6)

For writes, `AmILeader` is sufficient. `ReplicateLogs` has its own safety net: if the node loses leadership between the `AmILeader` check and the replication attempt, the replication fails and returns a non-success status. The client retries.

For reads, the choice depends on your consistency requirements. Chapter 6 covers this in detail.

6.3 Finding the Leader

When a node is not the leader, it needs to tell the client where the leader is. Kommander provides two methods for this.

6.3.1 WaitForLeader

```
string leaderEndpoint = await raft.WaitForLeader(
    partitionId,
    cancellationToken
);
```

`WaitForLeader` blocks until a leader is known for the given partition. It returns the leader's endpoint as a string in `host:port` format. If no leader is elected before the cancellation token fires, the method throws.

Use `WaitForLeader` during startup or when you need to wait for the cluster to be ready. It is not suitable for per-request routing because it blocks when there is no leader.

6.3.2 WaitForLeaderStableAsync

```
string leaderEndpoint = await raft.WaitForLeaderStableAsync(
    partitionId,
    minStableFor: TimeSpan.FromSeconds(2),
    timeout: TimeSpan.FromSeconds(10),
    cancellationToken
);
```

`WaitForLeaderStableAsync` waits until the same leader has been stable for a minimum duration. This is useful after a leader change: the first elected leader may immediately lose a re-election if another node has a more up-to-date log. Waiting for stability avoids routing to a leader that is about to lose the position.

The `timeout` parameter sets an upper bound on the wait. If leadership keeps churning and never stabilizes, the method throws a `TimeoutException` rather than blocking forever.

6.3.3 GetPartitionLeaderHint

```
string? leaderHint = raft.GetPartitionLeaderHint(partitionId);
```

`GetPartitionLeaderHint` returns the current best-effort leader hint for a partition. It does not block and does not wait. If no leader is known, it returns `null`.

For a locally hosted partition, the hint is the node's own leader belief. For a partition hosted on other nodes, the hint comes from gossiped load reports.

This is the best choice for per-request routing: fast, non-blocking, and good enough for a first attempt. If the hint is wrong, the target node rejects the write, and the client can retry.

6.4 Routing Strategies

6.4.1 Strategy 1: Client-Side Redirect

The simplest approach. The follower returns an HTTP redirect (307 Temporary Redirect with `preserveMethod: true`) to the leader's endpoint. The client follows the redirect automatically.

```
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        if (leader is not null)
        {
            string leaderUrl = BuildLeaderUrl(leader, $"/store/{key}");
           return Results.Redirect(leaderUrl, permanent: false, preserveMethod:
true);
        }

        return Results.Json(
            new { error = "No leader available, retry later" },
            statusCode: 503
        );
    }

    // ... handle the write on the leader
});
```

Advantages:

- Simple to implement.

- The client learns the leader's address and can send future requests directly.
- No extra network hop on the server side.

Disadvantages:

- The client must support redirects with method preservation.
- Exposes internal node addresses to the client.

6.4.2 Strategy 2: Server-Side Proxy

The follower proxies the request to the leader using `HttpClient`. The client sees a single endpoint and does not know which node is the leader.

```
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        if (leader is not null)
        {
            string leaderUrl = BuildLeaderUrl(leader, $"/store/{key}");

            using var reader = new StreamReader(request.Body);
            string body = await reader.ReadToEndAsync();

            using var client = new HttpClient();
            var response = await client.PutAsync(
                leaderUrl,
                new StringContent(body)
            );

            string content = await response.Content.ReadAsStringAsync();
            return Results.Text(
                content,
                contentType: "application/json",
                statusCode: (int)response.StatusCode
            );
        }

        return Results.Json(
            new { error = "No leader available, retry later" },
            statusCode: 503
        );
    }
```

```
    // ... handle the write on the leader
});
```

Advantages:

- The client does not need to know about leader routing.
- Internal node addresses stay internal.
- Works behind a load balancer with no special configuration.

Disadvantages:

- Extra network hop: client to follower to leader.
- The follower must parse and re-serialize the request body.
- You must manage `HttpClient` lifetime properly (use `IHttpClientFactory` in production).

6.4.3 Strategy 3: Leader Endpoint in Response Headers

Return the leader's endpoint in a response header. The client uses it for subsequent requests. This avoids redirects while still giving the client enough information to route directly.

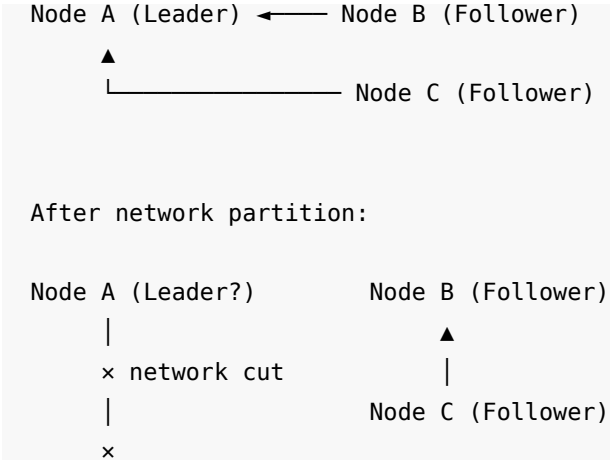
```
app.MapPut("/store/{key}", async (string key, HttpContext context) =>
{
    bool isLeader = await raft.AmILeader(partitionId, default);

    string? leader = raft.GetPartitionLeaderHint(partitionId);
    if (leader is not null)
        context.Response.Headers["X-Leader-Endpoint"] = leader;

    if (!isLeader)
    {
        context.Response.StatusCode = 503;
        await context.Response.WriteAsJsonAsync(
            new { error = "Not the leader", leader }
        );
        return;
    }

    // ... handle the write on the leader
});
```

The client reads the `X-Leader-Endpoint` header and uses it for the next request. This is a common pattern in distributed databases.

6.5 The Stale Leader Problem

The scenario that makes leadership checks subtle:

```
Before partition:
```



Node A was the leader. A network partition isolates Node A from Nodes B and C. On Node A:

- `AmILeader` returns `true`. Node A has not received a higher-term message.
- `AmILeaderQuick` returns `true`. Same reason.
- `ReplicateLogs` accepts the proposal but cannot commit it. There is no quorum (A alone is 1 of 3).
- The client waits for `ReplicateLogs` to return. It eventually times out.

On Nodes B and C:

- Heartbeats from A stop arriving.
- After the election timeout, one of them becomes a candidate and wins.
- The new leader accepts writes. Quorum is 2 of 3 (B and C). Writes commit.

When the partition heals:

- Node A receives a message with a higher term.
- Node A steps down to follower.
- Node A discards any uncommitted entries and catches up from the new leader.

The key insight: writes on Node A during the partition never committed. No data was lost or corrupted. But the client waited for a response that never came. Good routing and timeouts are the application's defense.

6.5.1 How ConfirmLeadershipAsync Helps

If Node A calls `ConfirmLeadershipAsync` during the partition, the quorum round fails. The method returns `false` within `LeadershipConfirmationTimeout` (default: 2 seconds). The application knows the leadership is not confirmed and can return a 503 immediately.

This is why `ConfirmLeadershipAsync` matters for reads. A write to a stale leader fails to commit (it times out). But a read from a stale leader returns stale data silently. The read looks correct but returns values that the new leader has already overwritten. `ConfirmLeadershipAsync` prevents this.

6.6 The OnLeaderChanged Event

```
raft.OnLeaderChanged += (partitionId, leaderEndpoint) =>
{
    Console.WriteLine(
        $"Partition {partitionId}: leader is now {leaderEndpoint}"
    );
    return Task.FromResult(true);
};
```

`OnLeaderChanged` fires whenever a new leader is elected for a partition. The callback receives the partition id and the new leader's endpoint.

Use this event for:

- **Logging.** Record leader changes for operational visibility.
- **Cache invalidation.** If your routing logic caches the leader endpoint, invalidate the cache here.
- **Application logic.** Some applications run background tasks only on the leader. Use `OnLeaderChanged` to start or stop those tasks.

The event fires on every node, not just the new leader. Every node in the cluster learns about the leadership change.

6.7 Timing Parameters

Three configuration properties control the timing of leader election:

Property	Default	Purpose
`HeartbeatInterval`	500 ms	How often the leader sends heartbeats to followers
`StartElectionTimeout`	2000 ms	Lower bound of the randomized election timeout
`EndElectionTimeout`	4000 ms	Upper bound of the randomized election timeout

The relationship between these values matters. The Raft paper recommends:

```
HeartbeatInterval << ElectionTimeout << MTBF
```

Where MTBF is the mean time between failures. The heartbeat must be much shorter than the election timeout so that a temporary network hiccup does not trigger a spurious election. The election timeout must be much shorter than the failure rate so that the cluster recovers before the next failure.

The defaults (500 ms heartbeat, 2000 to 4000 ms election timeout) work for most deployments. If your network has higher latency, increase both values proportionally.

6.8 Putting It Together: The ReplicatedStore Write Handler

Here is the recommended write handler that combines the patterns from this chapter:

```
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    // Fast local check
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)
    {
        // Try to help the client find the leader
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "NotLeader", leader },
            statusCode: 503
        );
    }

    // This node believes it is the leader. Proceed with the write.
    using var reader = new StreamReader(request.Body);
    string value = await reader.ReadToEndAsync();

    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(key, value)
    );

    using var timeout = new CancellationTokenSource(TimeSpan.FromSeconds(5));
    var result = await raft.ReplicateLogs(
        partitionId, "SetValue", payload,
        cancellationToken: timeout.Token
    );

    if (result.Success)
        return Results.Ok(new { key, value, logIndex = result.LogIndex });

    // Replication failed. The most common cause is a leadership change.
    if (result.Status == RaftOperationStatus.NodeIsNotLeader)
    {
        string? newLeader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "LeaderChanged", leader = newLeader },
            statusCode: 503
        );
    }

    return Results.Json(
        new { error = result.Status.ToString() },
        statusCode: 503
```

```
    );
});
```

This handler:

1. Checks `AmILeader` first. If not the leader, returns 503 with the leader hint.
2. Attempts the write with a timeout.
3. If `ReplicateLogs` returns `NodeIsNotLeader` (leadership changed between the check and the replication), returns 503 with the new leader hint.
4. For any other failure, returns 503 with the status.

The client reads the `leader` field from the error response and retries against that endpoint.

6.9 Summary

- Only the partition leader can propose writes. Followers must redirect or forward.
- `AmILeaderQuick` is a fast volatile read. It reports local belief. Use it for pre-checks where a false positive is acceptable.
- `AmILeader` reads published local state. It also reports local belief, with slightly more current data. Use it for standard write-path gates.
- `ConfirmLeadershipAsync` contacts a quorum and confirms leadership. Use it for strongly consistent reads (Chapter 6).
- A minority-partitioned leader believes it leads but cannot commit writes. Writes time out. Reads return stale data.
- `GetPartitionLeaderHint` returns a non-blocking best-effort leader hint. Use it for routing.
- `WaitForLeader` blocks until a leader is known. Use it during startup.
- Three routing strategies exist: client-side redirect, server-side proxy, and leader endpoint in headers. Choose based on whether clients need to see internal addresses.
- The `OnLeaderChanged` event fires on every node when a new leader is elected.
- The next chapter uses `ConfirmLeadershipAsync` to implement strongly consistent reads.

7 Strongly Consistent Reads

In Chapter 5, the read handler read directly from the local dictionary. This works most of the time. But under certain conditions, it returns stale data. This chapter explains why and shows how to fix it.

7.1 The Problem with Local Reads

Consider a three-node cluster where Node A is the leader:

```
Node A (Leader)
   │
   ├── store["balance"] = "500"     (latest committed value)
   │
Node B (Follower)
```

```
    |
    └── store["balance"] = "500"     (replicated)

Node C (Follower)
    |
    └── store["balance"] = "500"     (replicated)
```

A client reads `GET /store/balance` from Node A and receives `"500"`. This is correct.

Now imagine a network partition isolates Node A from Nodes B and C:

```
Node A (Leader?)              Node B ◄──► Node C
     |                             |
     × network cut                 ▼
     |                        New election
     ×                        Node B becomes leader (term 2)
```

Node B wins a new election. A client writes `balance = "300"` to Node B. The write commits on Nodes B and C (quorum of two).

```
Node A (stale leader, term 1)     Node B (leader, term 2)
   |                                 |
   store["balance"] = "500"          store["balance"] = "300"  (committed)
```

Now a different client reads `GET /store/balance` from Node A. The read handler checks `AmILeader`. Node A still believes it is the leader (it has not received a higher-term message). The handler reads from the local dictionary and returns `"500"`.

This response is wrong. The committed value is `"300"`. Node A served stale data as if it were the current truth.

7.2 Why Writes Do Not Have This Problem

Writes are safe because `ReplicateLogs` requires a quorum to commit. On the partitioned Node A:

1. The client calls `ReplicateLogs` on Node A.
2. Node A appends the entry to its local WAL.
3. Node A tries to send the entry to Nodes B and C.
4. The network is cut. No follower acknowledges.
5. Quorum is not reached. The write never commits.
6. `ReplicateLogs` times out and returns a failure.

The write path has a built-in safety check: the quorum requirement. The read path, as implemented so far, has no such check. It reads directly from local memory.

7.3 The Read-Index Protocol

The Raft dissertation (section 6.4) describes a protocol for linearizable reads. The idea is simple: before reading local state, confirm that this node is still the leader by contacting a quorum.

The protocol has two steps:

1. **Quorum confirmation.** The leader sends a heartbeat to all followers and waits for a majority to acknowledge. If a majority responds, the leader is still the leader. If not, the leader may have been deposed.
2. **Applied frontier check.** The leader records the commit index at the time of confirmation. It waits until its local state machine has applied all entries up to that index. After that point, a local read reflects all committed writes.

```
Client requests a consistent read
     |
     ▼
Leader: record current commit index (e.g. 42)
     |
     ▼
Leader: send heartbeat to followers
     |
     ├──► Follower B: ACK (same term) ✓
     |
     ├──► Follower C: ACK (same term) ✓
     |
     ▼
Quorum confirmed (2 of 3)
     |
     ▼
Wait until local applied index >= 42
     |
     ▼
Read from local state machine
     |
     ▼
Return result to client
```

If the leader is partitioned, step 1 fails. The heartbeats do not reach a quorum. The confirmation times out, and the application knows it must not serve the read.

7.4 ConfirmLeadershipAsync

Kommander implements the read-index protocol through `ConfirmLeadershipAsync`:

```
bool confirmed = await raft.ConfirmLeadershipAsync(
    partitionId,
    cancellationToken
);
```

If `confirmed` is `true`:

- A majority of nodes have acknowledged a heartbeat in the current term.

- All committed entries up to the confirmation point have been applied to the local state machine.
- A read from local state after this call is linearizable.

If `confirmed` is `false`:

- The node could not reach a quorum, or it is not the leader.
- The application must not read from local state. Redirect the client to another node or return an error.

7.4.1 Timeout

The confirmation waits up to `LeadershipConfirmationTimeout` (default: 2 seconds) for the quorum ack round. If the timeout expires, the method returns `false`. This bounds the worst case: a partitioned leader fails the confirmation within 2 seconds rather than blocking forever.

You can configure this timeout:

```
RaftConfiguration configuration = new()
{
    // ...
    LeadershipConfirmationTimeout = TimeSpan.FromSeconds(3)
};
```

Keep this value above one heartbeat round-trip under load.

7.5 Adding Consistent Reads to ReplicatedStore

Add a query parameter to the read endpoint. When the client requests `?consistent=true`, the handler confirms leadership before reading.

```
app.MapGet("/store/{key}", async (string key, HttpRequest request) =>
{
    bool consistent = request.Query.ContainsKey("consistent");

    if (consistent)
    {
        using var timeout = new CancellationTokenSource(
            TimeSpan.FromSeconds(3)
        );

        bool confirmed = await raft.ConfirmLeadershipAsync(
            partitionId,
            timeout.Token
        );

        if (!confirmed)
        {
            string? leader = raft.GetPartitionLeaderHint(partitionId);
```

```
            return Results.Json(
                new { error = "LeadershipNotConfirmed", leader },
                statusCode: 503
            );
        }
    }

    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});
```

The client chooses the consistency level per request:

```
# Fast read (eventual consistency, local state)
curl http://localhost:5001/store/balance

# Consistent read (linearizable, quorum-confirmed)
curl http://localhost:5001/store/balance?consistent=true
```

This pattern gives the application flexibility. Most reads can use the fast path. Only reads that require the latest committed value use the confirmed path.

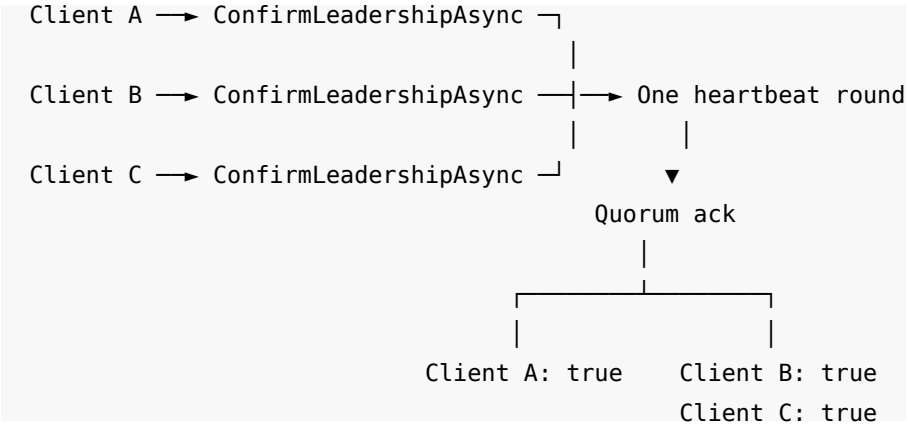
7.6 The Cost of Consistency

A consistent read adds one quorum round-trip compared to a local read. On a three-node cluster in the same data center, this typically adds 0.5 to 2 milliseconds of latency.

Kommander reduces this cost with two optimizations:

7.6.1 Coalescing

If multiple clients request consistent reads at the same time, Kommander sends a single heartbeat round and shares the result with all callers. If 100 clients call `ConfirmLeadershipAsync` within the same heartbeat interval, only one quorum round-trip occurs. All 100 callers receive the same `true` or `false` result.



7.6.2 Freshness Caching

If a quorum confirmation completed within the last heartbeat interval (default: 500 ms), Kommander reuses it. The reasoning: within one heartbeat interval, no other node can assemble an election quorum and take over leadership. A confirmation from 200 ms ago is still valid.

This means that under steady read traffic, the cost is approximately one quorum round-trip per heartbeat interval, regardless of how many reads arrive.

```
T = 0 ms     Confirmation round completes (quorum confirmed)
T = 100 ms   Read arrives → reuses cached confirmation (no round-trip)
T = 200 ms   Read arrives → reuses cached confirmation (no round-trip)
T = 400 ms   Read arrives → reuses cached confirmation (no round-trip)
T = 500 ms   Heartbeat interval expires → next confirmation starts a new round
```

7.7 Consistent Reads on Followers: ConfirmLocalApplicationAsync

`ConfirmLeadershipAsync` works only on the leader. If the client sends a consistent read to a follower, the follower cannot confirm its own leadership (it is not the leader).

For cases where you want to read from a follower and still guarantee that the read reflects all committed writes, Kommander provides `ConfirmLocalApplicationAsync`:

```
bool confirmed = await raft.ConfirmLocalApplicationAsync(
    partitionId,
    cancellationToken
);
```

This method works on any node, whether leader or follower. It contacts the leader, obtains the current commit index through a quorum ack round, and waits until the local node has applied all entries up to that index. If it returns `true`, the local state machine includes every committed write.

On the leader, `ConfirmLocalApplicationAsync` is equivalent to `ConfirmLeadershipAsync`. On a follower, it adds a network round-trip to the leader (which then runs its own quorum ack round) plus the time for the follower to apply any entries it has not yet processed.

7.7.1 When to Use Each Method

Method	Node	Guarantee	Use when
`ConfirmLeadershipAsync`	Leader only	Linearizable	You route reads to the leader
`ConfirmLocalApplicationAsync`	Any node	Linearizable	You want to read from a follower without forwarding

`ConfirmLocalApplicationAsync` is useful when you want to spread read load across the cluster. Each node can serve consistent reads from its own local state after confirming that it has caught up to the leader's commit index.

7.7.2 Follower Read Example

```
app.MapGet("/store/{key}", async (string key, HttpRequest request) =>
{
    bool consistent = request.Query.ContainsKey("consistent");

    if (consistent)
    {
        using var timeout = new CancellationTokenSource(
            TimeSpan.FromSeconds(3)
        );

        // Works on both leader and follower
        bool confirmed = await raft.ConfirmLocalApplicationAsync(
            partitionId,
            timeout.Token
        );

        if (!confirmed)
        {
            return Results.Json(
                new { error = "ConfirmationFailed" },
                statusCode: 503
            );
        }
    }

    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});
```

With this handler, a load balancer can distribute consistent reads across all three nodes. Each node confirms its own state before serving the read.

7.8 When Not to Use Consistent Reads

Consistent reads are not always necessary. Consider the consistency requirements of each read:

Scenario	Consistency needed	Recommendation
Dashboard showing latest metrics	Eventual	Local read (no confirmation)
User reads their own recent write	Linearizable	`ConfirmLeadershipAsync` or read-after-write at the application level
Bank balance check before a transfer	Linearizable	`ConfirmLeadershipAsync`
Search index lookup	Eventual	Local read from any node
Configuration value read by background job	Depends on use case	Local read is usually acceptable

The performance difference is real. A local read is a dictionary lookup: sub-microsecond. A confirmed read adds a quorum round-trip: typically 0.5 to 2 ms in the same data center, longer across regions. For applications that serve thousands of reads per second, this difference matters.

The design pattern: default to local reads, add confirmation where correctness requires it.

7.9 The Complete Stale-Read Scenario

Here is the full timeline of the stale-read problem and how `ConfirmLeadershipAsync` solves it.

7.9.1 Without Confirmation

```
T=0   Node A is leader. balance = "500".
T=1   Network partition isolates A from B, C.
T=2   Node B wins election (term 2).
T=3   Client writes balance = "300" to B. Commits on B, C.
T=4   Client reads from A:
        AmILeader → true (A has not seen term 2)
        Read from local state → "500" (STALE)
T=5   Partition heals. A discovers term 2. Steps down.
      A catches up. balance = "300" on all nodes.
```

Between T=1 and T=5, reads from Node A return stale data. The window can last for the entire duration of the network partition.

7.9.2 With Confirmation

```
T=0   Node A is leader. balance = "500".
T=1   Network partition isolates A from B, C.
T=2   Node B wins election (term 2).
T=3   Client writes balance = "300" to B. Commits on B, C.
T=4   Client reads from A:
```

```
        ConfirmLeadershipAsync → sends heartbeat to B, C
        B, C unreachable (partition)
        Timeout (2 seconds) → returns false
        Application returns 503 with leader hint
T=4.1 Client retries on B:
        ConfirmLeadershipAsync → quorum ack from C ✓
        Read from local state → "300" (CORRECT)
```

The confirmation fails on the partitioned node within 2 seconds. The client receives a clear error and can retry on the correct node.

7.10 Internals: How It Works

The read-index confirmation runs inside the `ReadIndexCoordinator`, one instance per partition. The coordinator manages three concerns:

1. **Round lifecycle.** When a `ConfirmLeadershipAsync` call arrives, the coordinator either joins an existing in-flight round or starts a new one. It triggers a heartbeat to all followers and records the current commit index. When a majority of followers acknowledge (same term), the round succeeds.
2. **Applied frontier wait.** After the quorum confirms, the coordinator checks whether the local state machine has applied all entries up to the recorded commit index. If entries are still being applied, the caller waits. Once the applied index reaches the recorded commit index, the read is safe.
3. **Expiry.** If the quorum ack round does not complete within `LeadershipConfirmationTimeout`, all waiters receive `false`. This prevents unbounded blocking on a partitioned leader.

Late arrivals (callers that arrive while a round is in flight but after the commit index has advanced past the round's recorded index) join a pending queue. They form the next round, which starts as soon as the current one finishes. The timeout budget spans the full wait for each caller, not just its own round.

7.11 Summary

- A local read from the leader is not automatically linearizable. A deposed leader can serve stale data.
- `ConfirmLeadershipAsync` implements the read-index protocol. It proves current leadership by contacting a quorum before the read.
- If confirmation succeeds, a local read after the call is linearizable. If it fails, the application must redirect or retry.
- `ConfirmLocalApplicationAsync` extends the guarantee to follower nodes. It confirms that the follower has applied all committed entries.
- Concurrent callers coalesce into a single quorum round-trip. A recent confirmation is reused within the heartbeat interval.
- The cost is approximately one quorum round-trip per heartbeat interval under steady traffic.

- `LeadershipConfirmationTimeout` (default: 2 seconds) bounds the worst case for a partitioned leader.
- Not every read needs consistency. Default to local reads and add confirmation where correctness requires it.
- The next chapter examines the full lifecycle of a replicated command, from proposal to commit.

8 Replication Deep Dive

The previous chapters used `ReplicateLogs`, checked the `Success` property, and moved on. This chapter opens the hood. It traces a command from the moment your application calls `ReplicateLogs` to the moment `OnReplicationReceived` fires on every node. It explains what happens at each step, what can go wrong, and what the result tells you.

8.1 The Full Lifecycle

When you call `ReplicateLogs`, Kommander executes the following sequence:

```
1. Admission
   |  Is this node the leader?
   |  Is the proposal queue full?
   ▼
2. Propose
   |  Assign a log position (Id) and a ticket (HLC timestamp)
   |  Write the entry to the local WAL as Proposed
   |  Fsync the WAL
   ▼
3. Replicate
   |  Send AppendEntries RPC to each follower
   |  Each follower appends to its own WAL and fsyncs
   |  Each follower sends an acknowledgment back
   ▼
4. Quorum
   |  Count acknowledgments (including the leader itself)
   |  A majority of voters must acknowledge
   ▼
5. Commit
   |  Write the Committed record to the leader's WAL
   |  Send commit notification to followers
   ▼
6. Apply
   |  Fire OnReplicationReceived on each node
   |  The application updates its state machine
   ▼
7. Return
   |  Return RaftReplicationResult to the caller
```

Each step has specific guarantees and failure modes. The sections below cover them in order.

8.2 Step 1: Admission

Before Kommander proposes the entry, it checks two conditions:

Is this node the leader? If not, `ReplicateLogs` returns immediately with `Status = NodeIsNotLeader`. No WAL write occurs. No network traffic is sent.

Is the proposal queue full? Each partition has a bounded queue of pending proposals (default: 2048 entries, controlled by `MaxQueuedClientProposalsPerPartition`). If the queue is full, `ReplicateLogs` returns `Status = ProposalQueueFull`. This is backpressure: the partition is accepting proposals faster than it can replicate and commit them. The application should wait and retry.

```
var result = await raft.ReplicateLogs(partitionId, "SetValue", payload);

if (!result.Success)
{
    switch (result.Status)
    {
        case RaftOperationStatus.NodeIsNotLeader:
            // Redirect to the leader
            break;

        case RaftOperationStatus.ProposalQueueFull:
            // Back off and retry after a delay
            break;
    }
}
```

8.3 Step 2: Propose

The leader assigns the entry a log position (the `Id` field on `RaftLog`) and a proposal ticket (an `HLCTimestamp`). The log position is monotonically increasing within the partition. The ticket identifies this specific proposal through the rest of the lifecycle.

The leader then writes the entry to its local WAL with type `Proposed` and fsyncs the write. The fsync is the durability point on the leader. After this point, the entry survives a leader crash: when the leader restarts, it reads the proposed entry from the WAL and can re-replicate it.

8.3.1 The Propose is the Durability Point

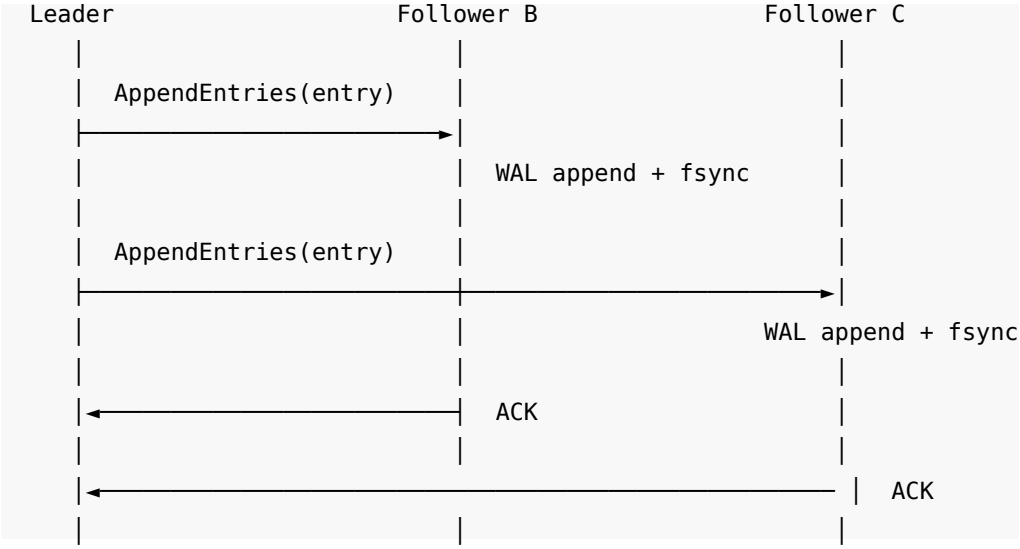
This is important enough to emphasize. The WAL fsync after the propose is the moment the entry becomes durable on the leader. If the leader crashes before the fsync, the entry is lost (but the client has not received a success response, so no data is silently lost). If the leader crashes after the fsync, the entry is on disk and will be recovered.

8.4 Step 3: Replicate

The leader sends the entry to each follower using the AppendEntries RPC. This is the Raft protocol's replication mechanism. The message includes:

- The entry itself (log position, term, type, data).
- The leader's term.
- The previous log position and term (for the Log Matching Property, which ensures followers have the same log prefix as the leader).

Each follower receives the entry, validates it, appends it to its own WAL, and fsyncs. The follower then sends an acknowledgment back to the leader.



The leader does not wait for all followers to acknowledge. It waits for a quorum.

8.5 Step 4: Quorum

A quorum is a majority of the cluster's voters. For a three-node cluster, the quorum is 2. The leader counts itself as one of the acknowledgers (it already wrote the entry to its own WAL). So the leader needs only one follower acknowledgment to reach quorum.

Cluster size	Quorum	Follower ACKs needed
3 nodes	2	1
5 nodes	3	2
7 nodes	4	3

As soon as the quorum is reached, the entry is considered committed. This can happen before all followers have acknowledged. In a three-node cluster, the entry commits as soon as the first follower responds, even if the second follower has not responded yet.

The second follower will receive the entry eventually (through a later AppendEntries or a catch-up mechanism). But the commit decision does not wait for it.

8.6 Step 5: Commit

After quorum is reached, the leader writes a `Committed` record to its own WAL. It also notifies the followers that the entry is committed (via the `leaderCommitIndex` field in subsequent AppendEntries messages).

8.6.1 The Single-Fsync Fast Path

By default, Kommander uses the **single-fsync commit** path (`WalSingleFsyncCommit = true`). In this mode, the client's `ReplicateLogs` call returns as soon as the propose quorum is durable (the leader and at least one follower have fsynced the Proposed entry). The Committed record is written to the WAL afterward, but the client does not wait for that second fsync.

This is safe because the propose-quorum durability is the true Raft commit point. A quorum of nodes holds the entry on disk. The Committed record is bookkeeping: it records a fact that is already guaranteed by the quorum.

The benefit is latency: the client's critical path contains one fsync instead of two. For workloads where WAL fsync dominates the latency (typically 1 to 10 ms per fsync on SSD), this removes a significant chunk.

You can disable this fast path:

```
RaftConfiguration configuration = new()
{
    // ...
    WalSingleFsyncCommit = false
};
```

With `WalSingleFsyncCommit = false`, the client waits for both the Proposed fsync and the Committed fsync. This means the on-disk WAL is always a reliable record of what is committed. The trade-off is higher write latency.

8.7 Step 6: Apply

After the entry is committed, Kommander fires `OnReplicationReceived` on each node that holds the partition. The callback receives the partition id and the `RaftLog` object.

The callback fires in log order. Entry 41 is applied before entry 42. If entry 42 arrives before entry 41 (due to reordering in the replication path), Kommander holds entry 42 until entry 41 is applied.

On the leader, the callback fires after the commit is confirmed. On followers, the callback fires after the follower learns that the entry is committed (via the leader's commit index in a subsequent heartbeat or AppendEntries).

8.8 Step 7: Return

`ReplicateLogs` returns a `RaftReplicationResult` with four properties:

```
RaftReplicationResult result = await raft.ReplicateLogs(
    partitionId, "SetValue", payload
```

```
);

// result.Success    true if the entry was committed
// result.Status     the specific outcome
// result.LogIndex   the log position assigned to this entry
// result.TicketId   the HLC timestamp that identifies this proposal
```

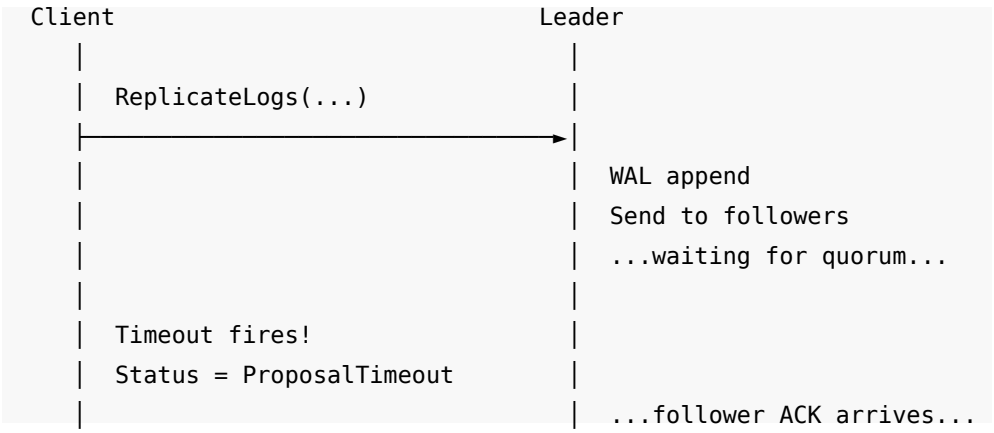
8.8.1 RaftOperationStatus Values

The `Status` field tells you exactly what happened. The most common values:

Status	Meaning	Action
`Success`	The entry was committed.	None.
`NodeIsNotLeader`	This node is not the leader.	Redirect to the leader.
`ProposalQueueFull`	The partition's proposal queue is at capacity.	Back off and retry.
`ProposalTimeout`	The quorum ack did not arrive in time.	The entry may or may not have committed (see below).
`OperationCancelled`	The caller's cancellation token fired.	The entry may or may not have committed (see below).
`RestoreInProgress`	The partition is still replaying WAL entries.	Wait and retry.
`PartitionMoved`	The partition generation changed (split/merge).	Refresh the partition map and retry.

8.9 The Ambiguity Window

Two statuses require special attention: `ProposalTimeout` and `OperationCancelled`. Both indicate that the client gave up waiting, but they do not tell you whether the entry was committed.



```
    |  Client thinks: "failed"          |  Quorum reached → committed
    |                                   |  OnReplicationReceived fires
```

The client received a timeout. The client does not know the outcome. The entry might have been committed a moment after the timeout fired. Or the entry might never have reached quorum.

This is the **ambiguity window**. It exists in every distributed system that separates the client from the commit decision. The network can fail between the commit and the client's receipt of the result.

8.9.1 Handling the Ambiguity

There are two strategies:

1. Idempotent commands. Design commands so that applying them twice produces the same result as applying them once. `SetValue("balance", "500")` is idempotent: applying it twice still sets the balance to 500. `IncrementCounter("visits")` is not idempotent: applying it twice adds 2 instead of 1. Chapter 8 covers this in detail.

2. Check before retry. After a timeout, read the current state to determine whether the command was applied. If `balance` is already `"500"`, the command committed. If not, retry.

The recommended approach is idempotent commands. They make retries safe without extra reads.

8.10 ReplicateLogs Overloads

Kommander provides several overloads of `ReplicateLogs`:

8.10.1 Single entry

```
Task<RaftReplicationResult> ReplicateLogs(
    int partitionId,
    string type,
    byte[] data,
    bool autoCommit = true,
    long expectedGeneration = 0,
    CancellationToken cancellationToken = default
);
```

This is the most common form. It proposes a single entry with a type label and a byte array payload.

8.10.2 Multiple entries (same type)

```
Task<RaftReplicationResult> ReplicateLogs(
    int partitionId,
    string type,
    IEnumerable<byte[]> logs,
    bool autoCommit = true,
    long expectedGeneration = 0,
```

```
    CancellationToken cancellationToken = default
);
```

This proposes multiple entries in a single batch. All entries share the same type label. They are replicated and committed together. The `OnReplicationReceived` callback fires once for each entry, in order.

Batch replication reduces overhead. Instead of one AppendEntries round-trip per entry, the batch sends all entries in one message. For workloads that produce many small entries, batching improves throughput. Chapter 10 covers batch replication in detail.

8.10.3 The autoCommit Parameter

The `autoCommit` parameter defaults to `true`. When true, Kommander commits the entry automatically after the quorum confirms it. This is the standard behavior for most applications.

When `autoCommit` is `false`, Kommander proposes and replicates the entry but does not commit it. The entry sits in a `Proposed` state on all nodes. Your application must call `CommitLogs` or `RollbackLogs` to finalize it.

```
// Propose without committing
var result = await raft.ReplicateLogs(
    partitionId, "Transfer", payload,
    autoCommit: false
);

if (result.Success)
{
    // Decide whether to commit or roll back
    bool shouldCommit = ValidateTransfer(result.TicketId);

    if (shouldCommit)
    {
        await raft.CommitLogs(partitionId, result.TicketId);
    }
    else
    {
        await raft.RollbackLogs(partitionId, result.TicketId);
    }
}
```

Manual commit is useful for two-phase operations where the application needs to validate or coordinate before finalizing. Chapter 10 covers this in full.

8.10.4 The expectedGeneration Parameter

When the cluster uses elastic partitions (Part III), partitions can split or merge. Each split or merge increments the partition's generation number. The `expectedGeneration` parameter lets you detect when a partition has moved:

```
var result = await raft.ReplicateLogs(
    partitionId, "SetValue", payload,
    expectedGeneration: 3
);

if (result.Status == RaftOperationStatus.PartitionMoved)
{
    // The partition has split or merged since generation 3.
    // Refresh the partition map and retry.
}
```

When `expectedGeneration` is 0 (the default), the check is disabled. Chapter 15 covers generation fencing.

8.11 Observing the Lifecycle

Add logging to the ReplicatedStore to trace each step:

```
raft.OnReplicationReceived += (partitionId, log) =>
{
    Console.WriteLine(
        $"[APPLY] partition={partitionId} " +
        $"id={log.Id} type={log.LogType} " +
        $"term={log.Term}"
    );

    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
};

raft.OnReplicationError += (partitionId, log) =>
{
    Console.WriteLine(
        $"[ERROR] partition={partitionId} " +
        $"id={log.Id} type={log.LogType}"
    );
};
```

`OnReplicationError` fires when a committed entry fails to apply (the `OnReplicationReceived` callback returned `false` or threw an exception). This is a diagnostic signal. In a healthy system, it should never fire. If it does, investigate why the callback failed.

8.11.1 Reading the Commit Index

You can query the current commit index for a partition:

```
long commitIndex = raft.GetCommitIndex(partitionId);
Console.WriteLine($"Partition {partitionId} commit index: {commitIndex}");
```

`GetCommitIndex` returns the highest log id that is known to be committed on this node. It is a local in-memory read. The commit index increases monotonically as entries are committed.

This is useful for diagnostics and monitoring. A follower whose commit index lags behind the leader's commit index is catching up. A commit index that stops advancing indicates a problem (no leader, no quorum, or a stuck partition).

8.12 Timeline: Three Nodes Processing One Write

Here is a concrete timeline of a write on a three-node cluster:

```
Time    Leader (A)              Follower (B)           Follower (C)
─────   ──────────              ────────────           ────────────
T=0     ReplicateLogs called
T=1     WAL append (Proposed)
        fsync
T=2     Send AppendEntries ──►  Receive entry
                            ──►                        Receive entry
T=3                             WAL append + fsync
                                Send ACK ──►
T=4     Receive ACK from B
        Quorum: A + B = 2/3 ✓
T=5     Commit (WAL write)
        OnReplicationReceived
        Return Success to client
T=6                                                    WAL append + fsync
                                                       Send ACK ──►
T=7     Receive ACK from C                             (late, but not needed)
T=8                             Learns commit via      Learns commit via
                                next heartbeat         next heartbeat
T=9                             OnReplicationReceived  OnReplicationReceived
```

Key observations:

- The client receives a response at T=5, after quorum (T=4) and the leader's commit.
- Follower C's ACK arrives at T=7, after the entry is already committed. This is fine. The quorum only needed A and B.

- Followers B and C learn about the commit through the next heartbeat (T=8) and apply the entry (T=9).
- The leader's `OnReplicationReceived` fires at T=5. The followers' callbacks fire at T=9. The delay is the heartbeat interval (default: 500 ms).

8.13 What Happens When Things Go Wrong

8.13.1 Leader crashes after propose, before quorum

The entry is in the leader's WAL (fsynced at T=1). No follower received it. If A restarts as leader, it re-replicates the entry. If B or C becomes the new leader, the entry is not in their logs. The new leader's log takes precedence: A's proposed entry is rolled back when A catches up from the new leader. The client received no response (A crashed), so the client retries.

8.13.2 Leader crashes after quorum, before the client receives the response

The entry is committed (a quorum holds it). The client never received the response. The client sees a timeout. The entry is applied on all nodes. The client is in the ambiguity window.

8.13.3 Follower crashes before acknowledging

The leader waits for a quorum. If the other follower acknowledges, quorum is reached (2 of 3). The crashed follower catches up when it restarts. If both followers crash, quorum cannot be reached. The write times out.

8.13.4 Network partition between leader and one follower

The leader still has a quorum (itself + the reachable follower). Writes continue. The unreachable follower catches up when the partition heals.

8.14 Summary

- `ReplicateLogs` proposes a command that goes through admission, propose, replicate, quorum, commit, and apply.
- The propose WAL fsync is the durability point on the leader. After this point, the entry survives a leader crash.
- Quorum means a majority of voters acknowledged. The leader counts itself.
- The single-fsync fast path (`WalSingleFsyncCommit = true`, the default) releases the client after propose-quorum durability, skipping the commit fsync. This reduces write latency without weakening durability.
- `RaftReplicationResult.Status` tells you the exact outcome. Always check it.
- `ProposalTimeout` and `OperationCancelled` are ambiguous: the entry may or may not have committed. Design idempotent commands to handle safe retries.
- `OnReplicationReceived` fires on all nodes, in log order, after the entry is committed. The leader fires it first; followers fire it after the next heartbeat.
- `OnReplicationError` fires when the callback fails. Investigate any occurrence.
- `GetCommitIndex` returns the local commit frontier. Use it for diagnostics.

- The next chapter covers how to design application state machines that process replicated commands correctly.

9 Designing Application State Machines

Kommander replicates commands. Your application interprets them. The callback that processes each committed command is your **state machine**. If the state machine is correct, every node in the cluster arrives at the same state. If the state machine is incorrect, the nodes diverge silently, and no amount of Raft correctness will save you.

This chapter teaches the rules for building a correct state machine. The rules are simple but the consequences of breaking them are severe.

9.1 The Core Rule: Determinism

The state machine must be **deterministic**. Given the same sequence of commands in the same order, every node must produce exactly the same state. No exceptions.

Raft guarantees that every node receives the same commands in the same order. That guarantee is worthless if the nodes interpret those commands differently.

```
Log (same on every node):

Position 1:  SetValue("name", "Alice")
Position 2:  IncrementCounter("visits")
Position 3:  SetValue("role", "admin")
Position 4:  DeleteValue("temp")

State machine result (must be identical on every node):

name    = "Alice"
visits  = 1
role    = "admin"
temp    = (deleted)
```

If Node A processes position 2 and gets `visits = 1`, but Node B processes the same command and gets `visits = 2`, the replicas have diverged. Raft cannot detect or correct this. The divergence is permanent.

9.2 What Breaks Determinism

Three patterns break determinism. Each is easy to introduce and hard to detect.

9.2.1 Pattern 1: Reading External State

The state machine reads something that is not part of the command: the current time, a random number, an environment variable, a file on disk, or a value from an external API.

Incorrect:

```
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
```

```
    if (log.LogType == "SetValue")
    {
        var command = Deserialize<SetValueCommand>(log.LogData);

        // BUG: DateTime.UtcNow is different on each node
        // and different during replay
        store[command.Key] = command.Value;
        store[command.Key + ":updated"] = DateTime.UtcNow.ToString("o");
    }

    return Task.FromResult(true);
}
```

Node A processes this at 14:00:01.123. Node B processes it at 14:00:01.456 (network delay). The `":updated"` value differs between nodes.

On replay after restart, Node A processes it at 15:30:00.000. The `":updated"` value differs from the live value.

Correct: Include the timestamp in the command itself, set by the client or the API handler before replication.

```
record SetValueCommand(string Key, string Value, string UpdatedAt);

// In the API handler (before replication):
var command = new SetValueCommand(
    key,
    value,
    DateTime.UtcNow.ToString("o")   // captured once, replicated to all nodes
);

byte[] payload = JsonSerializer.SerializeToUtf8Bytes(command);
await raft.ReplicateLogs(partitionId, "SetValue", payload);
```

```
// In the state machine (same on all nodes, same on replay):
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "SetValue")
    {
        var command = Deserialize<SetValueCommand>(log.LogData);
        store[command.Key] = command.Value;
        store[command.Key + ":updated"] = command.UpdatedAt;
    }

    return Task.FromResult(true);
}
```

Now the timestamp is part of the replicated command. Every node applies the same value.

The same rule applies to:

- **Random numbers.** Generate them in the API handler and include them in the command.
- **UUIDs/GUIDs.** Generate them before replication.
- **Configuration values.** If the state machine depends on a configuration setting, replicate the setting as a command so that all nodes use the same value.

9.2.2 Pattern 2: Side Effects in the Callback

The state machine sends an email, calls a webhook, writes to a file, or contacts an external API.

Incorrect:

```
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "CreateOrder")
    {
        var order = Deserialize<CreateOrderCommand>(log.LogData);
        orders[order.Id] = order;

        // BUG: fires on every node (3 emails for one order)
        // BUG: fires again on replay after restart
        emailService.SendOrderConfirmation(order.Email, order.Id);
    }

    return Task.FromResult(true);
}
```

This code has two problems:

1. **Multiple executions.** The callback fires on every node. A three-node cluster sends three emails for one order.
2. **Replay.** When a node restarts, `OnLogRestored` replays every committed command. Every historical order triggers another email.

Correct: Separate state changes from side effects. The callback updates state only. Side effects go into a separate queue that runs outside the callback.

```
// Side-effect queue (not replayed)
var pendingNotifications = new ConcurrentQueue<OrderNotification>();
bool isRestoring = false;

raft.OnRestoreStarted += (partitionId) =>
{
    isRestoring = true;
```

```
};

raft.OnRestoreFinished += (partitionId) =>
{
    isRestoring = false;
};

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "CreateOrder")
    {
        var order = Deserialize<CreateOrderCommand>(log.LogData);
        orders[order.Id] = order;

        // Queue the side effect only during live application on the leader
        if (!isRestoring)
        {
            pendingNotifications.Enqueue(
                new OrderNotification(order.Email, order.Id)
            );
        }
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;
```

A separate background task processes the notification queue. Only the leader runs this task. If the leader changes, the new leader picks up from where its state machine left off.

9.2.3 Pattern 3: Conditional Logic Based on Node Identity

The state machine behaves differently depending on which node is running it.

Incorrect:

```
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "SetValue")
    {
        var command = Deserialize<SetValueCommand>(log.LogData);
        store[command.Key] = command.Value;

        // BUG: only the leader tracks metrics
        // other nodes have different state
        if (await raft.AmILeader(partitionId, default))
```

```
        {
            store["metrics:writes"] =
                (int.Parse(store.GetValueOrDefault("metrics:writes", "0")) + 1)
                .ToString();
        }
    }

    return Task.FromResult(true);
}
```

The `metrics:writes` counter exists only on the leader. If leadership changes, the new leader starts from zero (or from whatever stale value it had). The nodes have different state.

Correct: If the metrics counter is part of the state, update it on every node. If it is a local-only diagnostic, do not store it in the replicated state.

9.3 The Two Callbacks

Kommander has two callbacks for applying commands. Both must use the same logic.

9.3.1 OnReplicationReceived

```
raft.OnReplicationReceived += (partitionId, log) =>
{
    // Fires for each newly committed command during normal operation.
    return ApplyCommand(partitionId, log);
};
```

This event fires during normal operation. When a command is committed, each node's `OnReplicationReceived` fires with the committed entry.

9.3.2 OnLogRestored

```
raft.OnLogRestored += (partitionId, log) =>
{
    // Fires for each persisted committed command during startup replay.
    return ApplyCommand(partitionId, log);
};
```

This event fires during startup. When a node starts with a durable WAL (RocksDB or SQLite), Kommander reads all committed entries and fires `OnLogRestored` for each one. This is how the application rebuilds its state machine.

9.3.3 The Pattern: One Function, Two Registrations

```
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    // Pure state change. No side effects.
    // Same result every time, regardless of context.
    switch (log.LogType)
    {
```

```
        case "SetValue":
            var set = Deserialize<SetValueCommand>(log.LogData);
            store[set.Key] = set.Value;
            break;

        case "DeleteValue":
            var del = Deserialize<DeleteValueCommand>(log.LogData);
            store.TryRemove(del.Key, out _);
            break;

        case "IncrementCounter":
            var inc = Deserialize<IncrementCounterCommand>(log.LogData);
            int current = int.Parse(
                store.GetValueOrDefault(inc.Key, "0")
            );
            store[inc.Key] = (current + 1).ToString();
            break;
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;
```

The function is the same for both events. The behavior is identical. Replay produces the same state as live application.

9.3.4 OnRestoreStarted and OnRestoreFinished

Two additional events bracket the restore phase:

```
raft.OnRestoreStarted += (partitionId) =>
{
    Console.WriteLine($"Partition {partitionId}: restore started");
};

raft.OnRestoreFinished += (partitionId) =>
{
    Console.WriteLine($"Partition {partitionId}: restore complete");
};
```

`OnRestoreStarted` fires before the first `OnLogRestored` call for a partition. `OnRestoreFinished` fires after the last one. Use these to:

- Set a flag that distinguishes replay from live application (as shown in the side-effect example above).
- Initialize or finalize data structures before and after replay.

- Log the restore duration for operational monitoring.

Both events receive the partition id as an `int`. They are `Action<int>` delegates (not async).

9.4 Command Serialization

Commands travel as byte arrays through the Raft log. The application must serialize commands before replication and deserialize them in the callback.

9.4.1 Choosing a Serialization Format

Format	Pros	Cons
System.Text.Json	Built into .NET, human-readable, good for debugging	Larger payloads, slower than binary
MessagePack	Compact, fast, schema-based	Requires a NuGet package, not human-readable
Protobuf	Compact, fast, cross-language	Requires .proto files or annotations
Raw bytes	Minimal overhead	Manual parsing, easy to get wrong

For most applications, `System.Text.Json` is the right starting point. The ReplicatedStore examples in this book use it. Switch to a binary format when payload size or serialization speed becomes a bottleneck.

9.4.2 The LogType Field

Every `ReplicateLogs` call includes a `type` string parameter. This becomes the `LogType` field on the `RaftLog` object. The state machine uses it to dispatch to the correct handler.

```
switch (log.LogType)
{
    case "SetValue":
        // ...
        break;
    case "DeleteValue":
        // ...
        break;
}
```

Keep type strings short and stable. They are persisted in the WAL and replayed on every restart. If you rename a type, old entries in the WAL will not match the new name.

9.5 Command Versioning

Over time, the structure of your commands will change. You might add a field, remove a field, or change a field's type. The WAL contains entries from every version of your application. The state machine must handle all of them.

9.5.1 Strategy 1: Tolerant Deserialization

Use a deserializer that ignores unknown fields and uses defaults for missing fields:

```
var options = new JsonSerializerOptions
{
    PropertyNameCaseInsensitive = true,
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull
};

var command = JsonSerializer.Deserialize<SetValueCommand>(
    log.LogData ?? [],
    options
);
```

This handles most additions gracefully. A new field added in version 2 defaults to null when replaying a version 1 entry.

9.5.2 Strategy 2: Explicit Version Field

Include a version number in the command:

```
record SetValueCommand(
    string Key,
    string Value,
    int Version = 1,
    string? UpdatedAt = null   // added in version 2
);
```

The state machine checks the version and adjusts its behavior:

```
if (command.Version >= 2 && command.UpdatedAt is not null)
{
    store[command.Key + ":updated"] = command.UpdatedAt;
}
```

9.5.3 Strategy 3: Separate LogType per Version

Use a different `LogType` for each version: `"SetValue.v1"`, `"SetValue.v2"`. This makes the dispatch explicit but increases the number of cases in the switch statement.

The recommended approach: start with tolerant deserialization. Add explicit versions when the command structure changes in a way that tolerant deserialization cannot handle (such as changing a field's semantics).

9.6 Idempotency

A command is **idempotent** if applying it once produces the same result as applying it two or more times. Idempotent commands are safe to retry after a timeout or a leader change.

9.6.1 Naturally Idempotent Commands

Some commands are idempotent by nature:

- `SetValue("balance", "500")`: applying it twice still sets the balance to 500.
- `DeleteValue("temp")`: deleting a key that does not exist is a no-op.

9.6.2 Non-Idempotent Commands

Other commands are not idempotent:

- `IncrementCounter("visits")`: applying it twice adds 2 instead of 1.
- `AppendToList("log", "entry-42")`: applying it twice adds two copies.
- `Transfer("A", "B", 100)`: applying it twice transfers 200 instead of 100.

9.6.3 Making Commands Idempotent

Add a unique operation id to each command. The state machine tracks which operations have been applied and skips duplicates.

```
record TransferCommand(
    string OperationId,   // unique per operation
    string FromAccount,
    string ToAccount,
    decimal Amount
);
```

```
var appliedOperations = new HashSet<string>();

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "Transfer")
    {
        var cmd = Deserialize<TransferCommand>(log.LogData);

        // Skip if already applied
        if (!appliedOperations.Add(cmd.OperationId))
            return Task.FromResult(true);

        decimal fromBalance = GetBalance(cmd.FromAccount);
        decimal toBalance = GetBalance(cmd.ToAccount);

        store[cmd.FromAccount] = (fromBalance - cmd.Amount).ToString();
        store[cmd.ToAccount] = (toBalance + cmd.Amount).ToString();
    }
```

```
    return Task.FromResult(true);
}
```

The client generates the `OperationId` (typically a GUID) and includes it in every retry of the same operation. The state machine applies the command the first time and skips it on duplicates.

This set of applied operation ids is part of the state machine. It must be rebuilt during replay and must produce the same result on every node.

9.6.4 Managing the Operation Set Size

The set of applied operations grows without bound if you never remove entries. Two strategies:

1. **Time-based expiration.** Remove operations older than a retention period (e.g., 24 hours). Include a timestamp in the command for this purpose. This works if clients do not retry after the retention period.
2. **Sliding window.** Track only the last N operations per client. This bounds memory usage but requires each client to have a stable identity.

9.7 Building the ReplicatedStore State Machine

Here is the evolved state machine that handles multiple command types correctly:

```
// Command records
record SetValueCommand(string Key, string Value, string? UpdatedAt = null);
record DeleteValueCommand(string Key);
record IncrementCounterCommand(string Key);

// State
var store = new ConcurrentDictionary<string, string>();
bool isRestoring = false;

raft.OnRestoreStarted += (_) => isRestoring = true;
raft.OnRestoreFinished += (_) => isRestoring = false;

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    switch (log.LogType)
    {
        case "SetValue":
        {
            var cmd = JsonSerializer.Deserialize<SetValueCommand>(
                log.LogData ?? []
            );
            if (cmd is not null)
            {
                store[cmd.Key] = cmd.Value;
```

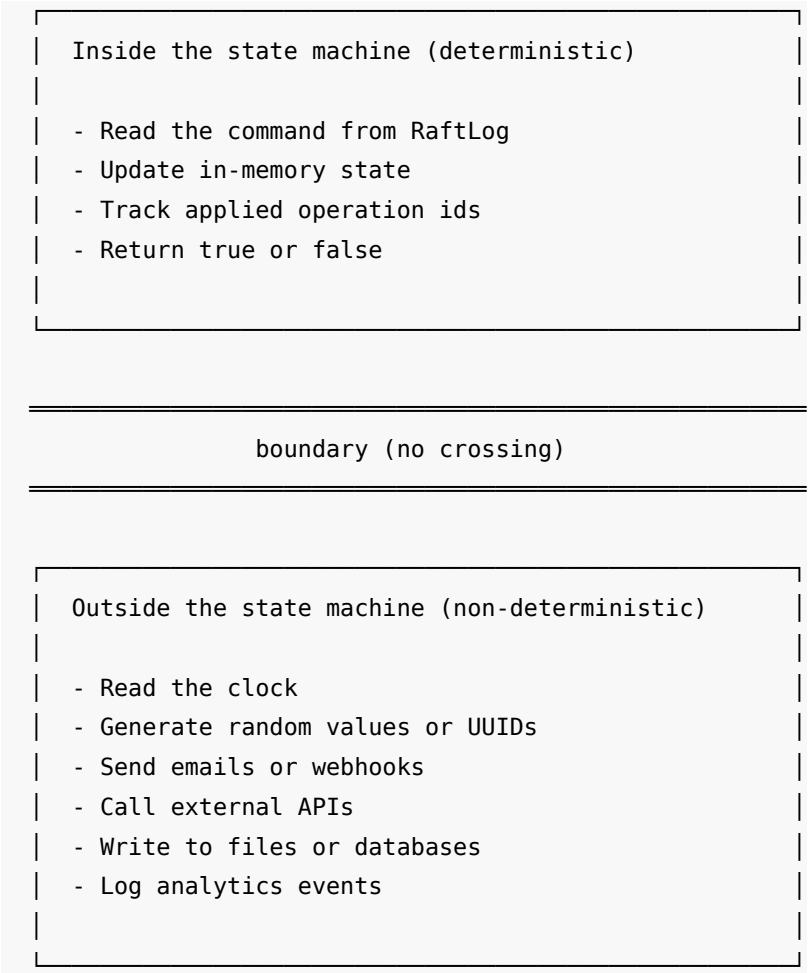
```
                if (cmd.UpdatedAt is not null)
                    store[cmd.Key + ":updated"] = cmd.UpdatedAt;
            }
            break;
        }

        case "DeleteValue":
        {
            var cmd = JsonSerializer.Deserialize<DeleteValueCommand>(
                log.LogData ?? []
            );
            if (cmd is not null)
            {
                store.TryRemove(cmd.Key, out _);
                store.TryRemove(cmd.Key + ":updated", out _);
            }
            break;
        }

        case "IncrementCounter":
        {
            var cmd = JsonSerializer.Deserialize<IncrementCounterCommand>(
                log.LogData ?? []
            );
            if (cmd is not null)
            {
                int current = int.Parse(
                    store.GetValueOrDefault(cmd.Key, "0")
                );
                store[cmd.Key] = (current + 1).ToString();
            }
            break;
        }
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;
```

This state machine:

- Handles three command types.
- Uses one function for both live application and replay.
- Does not read external state (no `DateTime.UtcNow`, no random values).

- Does not trigger side effects.
- Ignores unknown command types (the `default` case falls through silently).
- Uses tolerant deserialization (missing `UpdatedAt` defaults to `null`).

9.8 The Boundary Diagram



Everything above the boundary is the same on every node and on every replay. Everything below the boundary happens once, on one node, outside the callback.

9.9 Checklist

Before shipping a state machine, verify each of these:

1. Does the callback read `DateTime.UtcNow`, `Random`, or `Guid.NewGuid()`? If yes, move the value into the command payload.
2. Does the callback send an email, call an API, or write to a file? If yes, move it to a post-commit queue outside the callback.
3. Does the callback check `AmILeader` or node identity? If yes, remove the check. The state machine must be the same on every node.
4. Does the callback depend on data that is not in the command or in the state machine? If yes, replicate that data as its own command.

5. Is the command idempotent? If not, add an operation id and track applied operations.
6. Can old command formats still be deserialized? If not, add version handling.

9.10 Summary

- The state machine must be deterministic. Same commands in the same order must produce the same state on every node.
- Never read external state (time, random, environment) inside the apply callback. Include those values in the command payload.
- Never trigger side effects (IO, HTTP, email) inside the callback. Use a separate queue processed outside the callback.
- `OnReplicationReceived` and `OnLogRestored` must use the same function. Replay must produce the same state as live application.
- `OnRestoreStarted` and `OnRestoreFinished` bracket the replay phase. Use them to distinguish replay from live operation when controlling side effects.
- Design commands to be idempotent. Add operation ids for commands that are not naturally idempotent.
- Version your command format. Use tolerant deserialization and explicit version fields as the format evolves.
- The next chapter covers failure modes and retry strategies for distributed operations.

10 Batch Replication and Manual Commit Control

Every `ReplicateLogs` call in the previous chapters sent one command. One command means one proposal, one quorum round-trip, and one WAL flush. For a single write, this is fine. For a bulk import of 10,000 keys, it means 10,000 round-trips.

This chapter shows two ways to reduce that cost:

1. **Batch replication.** Send multiple entries in one proposal. One round-trip. One flush.
2. **Manual commit.** Propose entries without committing them. Validate, then commit or roll back as a unit.

10.1 Batch Replication: Homogeneous

The simplest form of batching sends multiple payloads of the same type in one call. Instead of passing a single `byte[]`, pass a list of `byte[]`.

10.1.1 Single-Entry Call (Before)

```
foreach (var item in items)
{
    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(item.Key, item.Value)
    );
```

```
    await raft.ReplicateLogs(partitionId, "SetValue", payload);
}
```

This loop sends N proposals. Each proposal waits for a quorum before the next one starts. For 100 items on a 1 ms round-trip, the total time is at least 100 ms.

10.1.2 Batch Call (After)

```
List<byte[]> payloads = items.Select(item =>
    JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(item.Key, item.Value)
    )
).ToList();

RaftReplicationResult result = await raft.ReplicateLogs(
    partitionId, "SetValue", payloads
);
```

This call sends all entries in one proposal. One quorum round-trip. One WAL flush. For 100 items on a 1 ms round-trip, the total time is approximately 1 ms.

10.1.3 The Batch Overload

```
Task<RaftReplicationResult> ReplicateLogs(
    int partitionId,
    string type,
    IReadOnlyList<byte[]> logs,
    bool autoCommit = true,
    long expectedGeneration = 0,
    CancellationToken cancellationToken = default
);
```

The parameters are the same as the single-entry overload, except that `logs` is a list instead of a single byte array. All entries in the batch share the same `type` string and `autoCommit` flag. This is why it is called a **homogeneous** batch: same type for every entry.

The result is a single `RaftReplicationResult`. If the batch succeeds, all entries committed. If it fails, none committed.

10.1.4 When to Use Homogeneous Batches

Use homogeneous batches when you have multiple commands of the same type that do not depend on each other:

- Bulk imports (many `SetValue` commands)
- Batch deletes (many `DeleteValue` commands)
- Event ingestion (many `AppendEvent` commands)

10.2 Batch Replication: Heterogeneous

Sometimes you need to batch commands of different types. A single transaction might set a key, increment a counter, and delete a temporary flag. The homogeneous batch cannot do this because it forces one `type` string across all entries.

`ReplicateEntries` solves this. Each entry carries its own type.

10.2.1 RaftProposalEntry

```
public readonly record struct RaftProposalEntry(
    string Type,
    byte[] Data,
    bool AutoCommit = true,
    long ExpectedGeneration = 0
);
```

Each entry has:

- **Type.** The command type string (maps to `RaftLog.LogType` in the state machine).
- **Data.** The serialized command payload (maps to `RaftLog.LogData`).
- **AutoCommit.** Whether this entry commits with the batch. Defaults to `true`.
- **ExpectedGeneration.** A generation fence for partition moves (see Chapter 13). Defaults to `0` (no fence).

10.2.2 ReplicateEntries

```
Task<RaftBatchReplicationResult> ReplicateEntries(
    int partitionId,
    IReadOnlyList<RaftProposalEntry> entries,
    CancellationToken cancellationToken = default
);
```

This method sends a heterogeneous batch: one proposal, one round-trip, one flush, but each entry has its own type. The result is a `RaftBatchReplicationResult` that contains a per-entry outcome.

10.2.3 Example: Mixed Command Batch

```
var entries = new List<RaftProposalEntry>
{
    new(
        Type: "SetValue",
        Data: JsonSerializer.SerializeToUtf8Bytes(
            new SetValueCommand("user:123:name", "Alice")
        )
    ),
    new(
        Type: "IncrementCounter",
        Data: JsonSerializer.SerializeToUtf8Bytes(
            new IncrementCounterCommand("stats:user-updates")
        )
```

```
    ),
    new(
        Type: "DeleteValue",
        Data: JsonSerializer.SerializeToUtf8Bytes(
            new DeleteValueCommand("pending:user:123")
        )
    )
};

RaftBatchReplicationResult result = await raft.ReplicateEntries(
    partitionId, entries
);

if (result.Success)
{
    for (int i = 0; i < result.Entries.Count; i++)
    {
        RaftEntryResult entry = result.Entries[i];
        Console.WriteLine(
            $"Entry {i}: status={entry.Status}, logIndex={entry.LogIndex}"
        );
    }
}
```

All three commands travel in one proposal. One quorum round-trip commits the key update, the counter increment, and the delete.

10.2.4 RaftBatchReplicationResult

```
public sealed class RaftBatchReplicationResult
{
    public bool Success { get; }
    public RaftOperationStatus Status { get; }
    public HLCTimestamp TicketId { get; }
    public IReadOnlyList<RaftEntryResult> Entries { get; }
}
```

- **Success.** `true` if at least one entry was admitted and appended. `false` on a batch-level rejection (bad shape or every entry fenced out).
- **Status.** The overall batch status.
- **TicketId.** The ticket for the batch proposal. Use this for diagnostics or for manual commit (see below).
- **Entries.** One `RaftEntryResult` per input entry, aligned by index.

10.2.5 RaftEntryResult

```
public readonly record struct RaftEntryResult(
    RaftOperationStatus Status,
```

```
    long LogIndex,
    HLCTimestamp Ticket
);
```

- **Status.** The outcome for this entry. `Success` for auto-commit entries that committed. `Pending` for manual entries that await explicit commit.
- **LogIndex.** The log index assigned to this entry. `-1` if the entry was not appended.
- **Ticket.** The manual-commit ticket for manual entries. `HLCTimestamp.Zero` for auto-commit entries.

10.2.6 Batch Shape Rules

A heterogeneous batch has a shape rule for the order of auto-commit and manual entries:

1. Auto-commit entries (`AutoCommit = true`) must come first.
2. Manual entries (`AutoCommit = false`) must come after all auto-commit entries.
3. No auto-commit entry may follow a manual entry.

This rule exists because manual entries form one contiguous suffix that the application commits or rolls back as a unit. An auto-commit entry after a manual entry would break that property.

```
Valid batch:

[AutoCommit] [AutoCommit] [Manual] [Manual]
─────────────────────────  ──────────────────

auto group (commits)      manual group (pending)


Invalid batch:

[AutoCommit] [Manual] [AutoCommit]   ← rejected
                       ▲
                       batch-level rejection
```

If a batch violates this rule, `ReplicateEntries` returns a batch-level failure. No entries are appended.

10.3 Manual Commit

By default, every entry commits as soon as it reaches a quorum. The `autoCommit` parameter controls this. When `autoCommit` is `false`, the entry is proposed and replicated but not committed. The application decides later whether to commit or roll back.

10.3.1 The Flow

```
1. Propose with autoCommit: false
     │
     ▼
2. Entry replicates to quorum (durable, but not committed)
```

```
      |
      ▼
  3. Application validates (check inventory, verify balance, etc.)
      |
      ├── Valid:    CommitLogs(partitionId, ticketId)
      |               └── Entry commits. State machine applies it.
      |
      └── Invalid:  RollbackLogs(partitionId, ticketId)
                      └── Entry is discarded. Never applied.
```

10.3.2 Step 1: Propose Without Committing

```
byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
    new ReserveInventoryCommand("SKU-42", quantity: 5)
);

RaftReplicationResult result = await raft.ReplicateLogs(
    partitionId,
    "ReserveInventory",
    payload,
    autoCommit: false,
    cancellationToken: timeout.Token
);

if (!result.Success)
{
    // Proposal failed. Handle the error.
    return Results.Json(
        new { error = result.Status.ToString() },
        statusCode: 503
    );
}

// The entry is proposed and durable, but NOT committed.
// result.TicketId identifies this proposal.
HLCTimestamp ticketId = result.TicketId;
```

The `TicketId` on the result is an `HLCTimestamp` value. It identifies the proposal for later commit or rollback.

10.3.3 Step 2: Validate

The application runs its validation logic. This might check inventory levels, verify account balances, or apply business rules. The validation reads from the current committed state.

```
int available = GetAvailableInventory("SKU-42");
bool valid = available >= 5;
```

10.3.4 Step 3: Commit or Roll Back

```
if (valid)
{
    var (success, status, commitLogId) = await raft.CommitLogs(
        partitionId,
        ticketId,
        cancellationToken: timeout.Token
    );

    if (success)
    {
        return Results.Ok(new { reserved = true, commitLogId });
    }

    return Results.Json(
        new { error = status.ToString() },
        statusCode: 503
    );
}
else
{
    var (success, status, _) = await raft.RollbackLogs(
        partitionId,
        ticketId,
        cancellationToken: timeout.Token
    );

    return Results.Json(
        new { reserved = false, reason = "Insufficient inventory" },
        statusCode: 409
    );
}
```

10.3.5 CommitLogs

```
Task<(bool success, RaftOperationStatus status, long commitLogId)> CommitLogs(
    int partitionId,
    HLCTimestamp ticketId,
    CancellationToken cancellationToken = default
);
```

`CommitLogs` commits the entries associated with the given ticket. After the call succeeds, the entries are committed and the state machine applies them through `OnReplicationReceived`.

The return tuple contains:

- **success.** `true` if the commit succeeded.

- **status.** The operation status.
- **commitLogId.** The log id of the commit record.

Committing an already-committed ticket is a no-op. This makes retries safe.

10.3.6 RollbackLogs

```
Task<(bool success, RaftOperationStatus status, long commitLogId)> RollbackLogs(
    int partitionId,
    HLCTimestamp ticketId,
    CancellationToken cancellationToken = default
);
```

`RollbackLogs` discards the entries associated with the given ticket. The entries are removed from the log. The state machine never sees them.

Rolling back an already-rolled-back ticket is also a no-op.

10.3.7 Cancellation Behavior

Both `CommitLogs` and `RollbackLogs` handle cancellation the same way. If the `CancellationToken` fires before the operation completes, the method returns `(false, OperationCancelled, 0)` instead of throwing. The caller should back off and retry the same ticket. The queued work may still apply, and re-issuing the same ticket is safe.

10.4 The Reserve-and-Confirm Pattern

Here is a complete example that uses manual commit for an inventory reservation:

```
record ReserveInventoryCommand(string Sku, int Quantity, string OperationId);
record ConfirmReservationCommand(string Sku, int Quantity, string OperationId);

app.MapPost("/reserve", async (HttpRequest request) =>
{
    using var reader = new StreamReader(request.Body);
    string body = await reader.ReadToEndAsync();
    var input = JsonSerializer.Deserialize<ReserveRequest>(body);

    if (input is null)
        return Results.BadRequest();

    string operationId = Guid.NewGuid().ToString();

    // Step 1: Propose the reservation without committing
    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new ReserveInventoryCommand(input.Sku, input.Quantity, operationId)
    );

    using var timeout = new CancellationTokenSource(
        TimeSpan.FromSeconds(5)
    );
```

```
var result = await raft.ReplicateLogs(
    partitionId,
    "ReserveInventory",
    payload,
    autoCommit: false,
    cancellationToken: timeout.Token
);

if (!result.Success)
{
    return Results.Json(
        new { error = result.Status.ToString() },
        statusCode: 503
    );
}

HLCTimestamp ticketId = result.TicketId;

// Step 2: Validate against current committed state
int available = GetAvailableInventory(input.Sku);

if (available < input.Quantity)
{
    // Not enough inventory. Roll back.
    await raft.RollbackLogs(partitionId, ticketId, timeout.Token);

    return Results.Json(
        new { reserved = false, reason = "Insufficient inventory" },
        statusCode: 409
    );
}

// Step 3: Commit the reservation
var (success, status, commitLogId) = await raft.CommitLogs(
    partitionId,
    ticketId,
    timeout.Token
);

if (!success)
{
    return Results.Json(
        new { error = status.ToString() },
```

```
            statusCode: 503
        );
    }

    return Results.Ok(new
    {
        reserved = true,
        sku = input.Sku,
        quantity = input.Quantity,
        commitLogId
    });
});

record ReserveRequest(string Sku, int Quantity);
```

This pattern gives the application a decision point between proposal and commit. The entry is durable on a quorum of nodes, but the state machine does not apply it until the explicit commit.

10.5 Manual Commit with Heterogeneous Batches

`ReplicateEntries` supports mixing auto-commit and manual entries in one batch. The auto-commit entries commit immediately. The manual entries form a trailing group that the application commits or rolls back later.

```
var entries = new List<RaftProposalEntry>
{
    // Auto-commit: record the attempt (always commits)
    new(
        Type: "AuditLog",
        Data: JsonSerializer.SerializeToUtf8Bytes(
            new AuditLogEntry("reservation-attempt", sku, quantity)
        ),
        AutoCommit: true
    ),
    // Manual: the reservation itself (commit or rollback)
    new(
        Type: "ReserveInventory",
        Data: JsonSerializer.SerializeToUtf8Bytes(
            new ReserveInventoryCommand(sku, quantity, operationId)
        ),
        AutoCommit: false
    )
};

RaftBatchReplicationResult result = await raft.ReplicateEntries(
    partitionId, entries
```

```
);

if (result.Success)
{
    // The audit log entry committed immediately (status = Success).
    // The reservation entry is pending (status = Pending).
    RaftEntryResult auditResult = result.Entries[0];
    RaftEntryResult reserveResult = result.Entries[1];

    // The manual ticket for commit/rollback
    HLCTimestamp ticketId = reserveResult.Ticket;

    // Validate, then commit or rollback ticketId...
}
```

The audit log entry commits with the batch. The reservation entry waits for an explicit `CommitLogs` or `RollbackLogs`.

10.6 What Happens When the Leader Crashes

If the leader crashes after a manual proposal but before the application calls `CommitLogs`:

1. The entry is durable on a quorum of nodes (it was proposed and acknowledged).
2. The new leader does not automatically commit the entry. It remains uncommitted.
3. The client's `CommitLogs` call fails (the connection breaks or times out).
4. The entry stays in the log as an uncommitted proposal until a new leader either includes it in its own term or overwrites it.

The client must detect the failure and re-propose the entire operation:

```
var (success, status, _) = await raft.CommitLogs(
    partitionId, ticketId, timeout.Token
);

if (!success)
{
    if (status == RaftOperationStatus.OperationCancelled ||
        status == RaftOperationStatus.NodeIsNotLeader)
    {
        // Leader changed. Re-propose from scratch.
        // The old uncommitted entry will be overwritten
        // by the new leader.
    }
}
```

This is why manual commit is a more advanced pattern. It adds a failure window between proposal and commit where a leader crash requires re-proposal. Auto-commit does not have this window because the entry commits as part of the same proposal.

10.7 Checkpoints

`ReplicateCheckpoint` writes a special marker entry to the log. This marker tells the WAL compaction process that all entries before this point are safe to truncate.

```
RaftReplicationResult result = await raft.ReplicateCheckpoint(
    partitionId,
    cancellationToken: timeout.Token
);
```

The checkpoint is a replicated entry, not a local-only marker. It goes through the same proposal and quorum process as any other entry. This ensures that all nodes agree on the checkpoint position.

10.7.1 When to Use Checkpoints

Use checkpoints when:

- The WAL grows large and you want to allow compaction.
- Your application has a natural "save point" (e.g., after a snapshot of the state machine).
- You want to bound the replay time on restart.

Without checkpoints, the WAL retains every entry from the beginning. Each restart replays the entire log through `OnLogRestored`. A checkpoint lets the WAL discard entries before the checkpoint, so replay starts from the checkpoint position instead of the beginning.

10.7.2 Checkpoint Flow

```
Log:  [1] [2] [3] [4] [5] [CP] [6] [7] [8]
                          ▲
                          checkpoint at index 5

After compaction:

Log:  [CP] [6] [7] [8]

On restart, replay starts at [6], not [1].
```

The checkpoint does not include a snapshot of the state machine. It is a marker only. The application must ensure that the state machine can reconstruct its state from the entries after the checkpoint. This typically means the application writes a snapshot to disk at the same time it issues the checkpoint.

10.8 Building a Bulk Import Endpoint

Here is a practical endpoint that uses batch replication:

```
app.MapPost("/store/bulk", async (HttpRequest request) =>
{
    bool isLeader = await raft.AmILeader(partitionId, default);
```

```
if (!isLeader)
{
    string? leader = raft.GetPartitionLeaderHint(partitionId);
    return Results.Json(
        new { error = "NotLeader", leader },
        statusCode: 503
    );
}

using var reader = new StreamReader(request.Body);
string body = await reader.ReadToEndAsync();
var items = JsonSerializer.Deserialize<List<KeyValueItem>>(body);

if (items is null || items.Count == 0)
    return Results.BadRequest();

// Build the batch
var entries = items.Select(item => new RaftProposalEntry(
    Type: "SetValue",
    Data: JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(item.Key, item.Value)
    )
)).ToList();

using var timeout = new CancellationTokenSource(
    TimeSpan.FromSeconds(10)
);

RaftBatchReplicationResult result = await raft.ReplicateEntries(
    partitionId, entries, timeout.Token
);

if (result.Success)
{
    int committed = result.Entries.Count(
        e => e.Status == RaftOperationStatus.Success
    );

    return Results.Ok(new { imported = committed, total = items.Count });
}

return Results.Json(
    new { error = result.Status.ToString() },
    statusCode: 503
```

```
    );
});

record KeyValueItem(string Key, string Value);
```

This endpoint accepts a JSON array of key-value pairs and imports all of them in one proposal. The response reports how many entries committed.

10.8.1 Performance Comparison

Approach	100 items, 1 ms RTT	1000 items, 1 ms RTT
Single-entry loop	~100 ms	~1000 ms
Homogeneous batch	~1 ms	~1 ms
Heterogeneous batch	~1 ms	~1 ms

The batch approaches reduce the time from O(N) round-trips to O(1). The WAL also benefits: one flush for the entire batch instead of one flush per entry.

10.9 Choosing the Right Approach

Need	Use
Many commands, same type	`ReplicateLogs` with `IReadOnlyList<byte[]>`
Many commands, different types	`ReplicateEntries` with `RaftProposalEntry` list
Validate before committing	`autoCommit: false` + `CommitLogs` / `RollbackLogs`
Mark a WAL compaction point	`ReplicateCheckpoint`

Start with auto-commit. Move to manual commit only when you need the validate-then-decide pattern. Manual commit adds complexity and a failure window between proposal and commit.

10.10 Summary

- Batch replication sends multiple entries in one proposal. One quorum round-trip and one WAL flush replace N round-trips and N flushes.
- Homogeneous batches (`ReplicateLogs` with a list) require all entries to share the same type.
- Heterogeneous batches (`ReplicateEntries`) allow each entry to carry its own type and auto-commit flag.
- `RaftBatchReplicationResult` contains per-entry results aligned by index to the input list.
- Manual commit (`autoCommit: false`) proposes entries without committing. The application calls `CommitLogs` to commit or `RollbackLogs` to discard.
- The `TicketId` (an `HLCTimestamp`) connects a proposal to its commit or rollback.

- If the leader crashes before a manual commit, the entry is not automatically committed. The client must re-propose.
- `ReplicateCheckpoint` marks a point in the log for WAL compaction. Entries before the checkpoint can be truncated.
- The next chapter explains why a single Raft group is not sufficient for large-scale systems and introduces partitioned Raft.

11 Why One Raft Group Is Not Enough

Every chapter so far used a single partition: partition 1. All writes went to one leader. All reads came from one state machine. This works for small workloads. It does not scale.

This chapter explains why and introduces partitioned Raft: independent Raft groups with independent leaders that distribute the write load across all nodes in the cluster.

11.1 The Single-Leader Bottleneck

In a Raft cluster with one partition, only the leader accepts writes. The followers replicate entries and serve as standby copies. They do not accept proposals.

```
Three-node cluster, one partition:

Node A (Leader)
   │
   ├── All writes go here
   ├── WAL flush for every batch
   ├── Send AppendEntries to B and C
   └── Wait for quorum ACK
       │
Node B (Follower)        Node C (Follower)
   │                        │
   └── Idle (replicating)   └── Idle (replicating)
```

Node A does all the work. Nodes B and C receive entries and store them, but they do not propose or order commands. The cluster has the capacity of three nodes, but the write throughput is limited to what one node can handle.

11.1.1 Measuring the Bottleneck

Assume each node can handle 50,000 writes per second. With one partition:

Metric	Value
Write throughput	50,000 ops/sec (one node)
Leader CPU usage	High
Follower CPU usage	Low
Cluster utilization	33% (1 of 3 nodes busy)

Two-thirds of the cluster's capacity sits idle. Adding more nodes does not help. A six-node cluster with one partition still funnels all writes through one leader. The write throughput stays at 50,000 ops/sec.

11.1.2 The Read Side

Reads are less affected. Every node has a copy of the state. A read that tolerates eventual consistency can go to any node (Chapter 6). A consistent read still needs a quorum confirmation, but the confirmation overhead is small compared to a write.

The bottleneck is writes. Raft is a write-path protocol. Its core job is to order and replicate write proposals. When all proposals go through one leader, that leader is the ceiling.

11.2 Partitioned Raft

The solution: run multiple independent Raft groups in the same cluster. Each group is a **partition**. Each partition has its own leader, its own term, its own log, and its own state machine.

```
Three-node cluster, three partitions:

Node A               Node B               Node C
┌──────────┐         ┌──────────┐         ┌──────────┐
│ P1 LEADER │         │ P1 follow │         │ P1 follow │
│ P2 follow │         │ P2 LEADER │         │ P2 follow │
│ P3 follow │         │ P3 follow │         │ P3 LEADER │
└──────────┘         └──────────┘         └──────────┘

Writes to partition 1 → Node A
Writes to partition 2 → Node B
Writes to partition 3 → Node C
```

Now each node leads one partition. Write load spreads across all three nodes. The cluster throughput scales with the number of nodes and partitions.

Metric	One partition	Three partitions
Write throughput	50,000 ops/sec	150,000 ops/sec
Leader CPU usage	1 node at high	3 nodes at moderate
Cluster utilization	33%	~100%

11.2.1 Independence

Each partition is fully independent. The leader of partition 1 does not coordinate with the leader of partition 2. Their logs are separate. Their terms are separate. Their elections are separate.

If partition 2's leader crashes, partition 2 runs a new election. Partitions 1 and 3 continue without interruption. The failure scope is one partition, not the entire cluster.

```
Node B crashes:

Node A                                  Node C
┌─────────────┐                         ┌─────────────┐
│ P1 LEADER │  (unaffected)             │ P1 follow │
│ P2 follow │  ← election starts →      │ P2 LEADER │  (new leader)
│ P3 follow │  (unaffected)             │ P3 LEADER │
└─────────────┘                         └─────────────┘

Partition 1: no interruption
Partition 2: brief pause, then resumes on Node C
Partition 3: no interruption
```

11.3 Configuring Partitions

The `InitialPartitions` property on `RaftConfiguration` sets the number of application partitions at cluster bootstrap:

```
RaftConfiguration configuration = new()
{
    NodeName = "node-a",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 8
};
```

This creates 8 application partitions (partition 1 through partition 8) plus the system partition (partition 0). The system partition is always present. The `InitialPartitions` count does not include it.

11.3.1 Choosing the Number of Partitions

The number of partitions determines the maximum parallelism for writes:

Partitions	Maximum leaders	Write parallelism
1	1	Single node
3	3	All 3 nodes (on a 3-node cluster)
8	3	All 3 nodes, some lead multiple partitions
24	3	All 3 nodes, 8 partitions each

More partitions than nodes is fine. Kommander distributes leadership across the available nodes. With 8 partitions on 3 nodes, one node leads approximately 3 partitions and the other two lead approximately 2 or 3 each.

Guidelines:

- **Minimum:** At least as many partitions as nodes. This ensures every node can lead at least one partition.

- **Practical default:** 2 to 4 times the number of nodes. This gives the scheduler room to balance leadership even when a node goes down.
- **Too many:** Each partition has overhead (WAL files, memory for the state machine, election timers). Hundreds of partitions on a 3-node cluster waste resources for no throughput gain.

11.3.2 InitialPartitions Is a Bootstrap Setting

`InitialPartitions` takes effect only during the first cluster bootstrap (the first leader election on partition 0). The value is written into the committed partition map. On subsequent restarts, the value in the configuration is ignored. The cluster uses the committed map.

To add partitions to a running cluster, use the partition management APIs (Chapter 13). To remove partitions, use the partition lifecycle APIs. `InitialPartitions` does not change the partition count of an existing cluster.

11.4 Partition 0: The System Partition

Partition 0 is the **system partition**. Kommander reserves it for internal cluster configuration: the partition map, membership changes, and other system-level operations.

```
Partition 0 (system):   cluster configuration
Partition 1 (application): your data
Partition 2 (application): your data
Partition 3 (application): your data
...
```

Rules for partition 0:

- Do not replicate application data to partition 0.
- Do not register application callbacks on partition 0.
- Kommander manages partition 0 internally.
- System entries use the `_RaftSystem` log type.

Your application works with partitions 1 and above. When the previous chapters used `int partitionId = 1;`, that was partition 1, the first application partition.

11.5 Routing Keys to Partitions

With multiple partitions, the application must decide which partition holds which data. Kommander provides `GetPartitionKey` for this:

```
int partitionId = raft.GetPartitionKey("user:123");
```

`GetPartitionKey` hashes the input string and maps the hash to a partition id. The same key always maps to the same partition. Different keys may map to different partitions.

The method returns a partition id in the application range (1 and above). It never returns 0 (the system partition).

11.5.1 How the Routing Works

```
Key: "user:123"
     |
     ▼
Hash function (consistent hash)
     |
     ▼
Hash value: 0x4A2F...
     |
     ▼
Map to partition range
     |
     ▼
Partition id: 3
```

The partition map defines ranges. Each partition owns a range of hash values. `GetPartitionKey` hashes the key, finds which range contains the hash, and returns that partition's id.

11.5.2 Prefix Routing

`GetPrefixPartitionKey` works the same way but uses only a prefix of the key for routing:

```
int partitionId = raft.GetPrefixPartitionKey("user:123:profile");
```

This is useful when you want all keys with the same prefix to land on the same partition. For example, all keys starting with `"user:123"` go to the same partition, regardless of the suffix.

11.5.3 Updated ReplicatedStore

Here is the ReplicatedStore evolved from one partition to multiple partitions:

```
// Configuration: 8 partitions across 3 nodes
RaftConfiguration configuration = new()
{
    NodeName = args[0],
    Host = "localhost",
    Port = int.Parse(args[1]),
    InitialPartitions = 8
};

// Per-partition state machines
var stores = new ConcurrentDictionary<int, ConcurrentDictionary<string,
string>>();

ConcurrentDictionary<string, string> GetStore(int partitionId)
{
    return stores.GetOrAdd(partitionId, _ => new ConcurrentDictionary<string,
```

```
string>());
}

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    var store = GetStore(partitionId);

    switch (log.LogType)
    {
        case "SetValue":
            var cmd = JsonSerializer.Deserialize<SetValueCommand>(
                log.LogData ?? []
            );
            if (cmd is not null)
                store[cmd.Key] = cmd.Value;
            break;

        case "DeleteValue":
            var del = JsonSerializer.Deserialize<DeleteValueCommand>(
                log.LogData ?? []
            );
            if (del is not null)
                store.TryRemove(del.Key, out _);
            break;
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;
```

The state machine now receives the `partitionId` parameter and maintains a separate dictionary for each partition. This is important: each partition is an independent Raft group with its own log. The state machine must keep the data separate.

11.5.4 Routing Writes

```
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    // Route the key to a partition
    int partitionId = raft.GetPartitionKey(key);

    // Check if this node leads the partition
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)
```

```
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "NotLeader", leader, partitionId },
            statusCode: 503
        );
    }

    using var reader = new StreamReader(request.Body);
    string value = await reader.ReadToEndAsync();

    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(key, value)
    );

    using var timeout = new CancellationTokenSource(
        TimeSpan.FromSeconds(5)
    );

    var result = await raft.ReplicateLogs(
        partitionId, "SetValue", payload,
        cancellationToken: timeout.Token
    );

    if (result.Success)
        return Results.Ok(new { key, value, partitionId });

    string? newLeader = raft.GetPartitionLeaderHint(partitionId);
    return Results.Json(
        new { error = result.Status.ToString(), leader = newLeader },
        statusCode: 503
    );
});
```

The key change: `partitionId` is no longer hardcoded to 1. It comes from `GetPartitionKey(key)`. Different keys route to different partitions. Different partitions may have different leaders. The error response now includes `partitionId` so the client knows which partition to retry.

11.5.5 Routing Reads

```
app.MapGet("/store/{key}", async (string key, HttpRequest request) =>
{
    int partitionId = raft.GetPartitionKey(key);
    var store = GetStore(partitionId);
```

```
    bool consistent = request.Query.ContainsKey("consistent");

    if (consistent)
    {
        using var timeout = new CancellationTokenSource(
            TimeSpan.FromSeconds(3)
        );

        bool confirmed = await raft.ConfirmLeadershipAsync(
            partitionId, timeout.Token
        );

        if (!confirmed)
        {
            string? leader = raft.GetPartitionLeaderHint(partitionId);
            return Results.Json(
                new { error = "LeadershipNotConfirmed", leader, partitionId },
                statusCode: 503
            );
        }
    }

    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value, partitionId })
        : Results.NotFound();
});
```

The read handler uses the same routing. Consistent reads confirm leadership on the correct partition.

11.6 The Partition Map

`GetPartitionMap` returns the current partition map:

```
IReadOnlyList<RaftPartitionRange> map = raft.GetPartitionMap();

foreach (RaftPartitionRange range in map)
{
    Console.WriteLine(
        $"Partition {range.PartitionId}: " +
        $"range [{range.StartRange}, {range.EndRange}], " +
        $"generation {range.Generation}, " +
        $"state {range.State}"
    );
}
```

Each `RaftPartitionRange` has:

Property	Type	Description
`PartitionId`	`int`	The partition's id
`StartRange`	`int`	Start of the hash range
`EndRange`	`int`	End of the hash range
`Generation`	`long`	Incremented on each map mutation
`State`	`RaftPartitionState`	Lifecycle state (active, splitting, etc.)
`RoutingMode`	`RaftRoutingMode`	Whether included in `GetPartitionKey` routing
`Replicas`	`List<RaftReplica>`	The nodes hosting this partition

The partition map is replicated through partition 0 (the system partition). All nodes see the same map.

11.7 Failure Isolation

With multiple partitions, a single node failure affects only the partitions that node led. Other partitions continue without interruption.

11.7.1 Example: Node B Fails

```
Before failure:

Node A: leads P1, P4, P7
Node B: leads P2, P5, P8
Node C: leads P3, P6

Node B crashes.

After election:

Node A: leads P1, P4, P7, P5 (new)
Node C: leads P3, P6, P2 (new), P8 (new)

Affected partitions: P2, P5, P8 (brief pause during election)
Unaffected partitions: P1, P3, P4, P6, P7
```

Clients that wrote to partitions P1, P3, P4, P6, or P7 never noticed the failure. Clients that wrote to P2, P5, or P8 saw a brief timeout (the election timeout, 2 to 4 seconds), then resumed on the new leaders.

This is the core advantage of partitioned Raft: **failure isolation**. One node failure does not stop the entire cluster. It stops only the partitions that node led, and those partitions recover through election within seconds.

11.8 A Note on the Word "Partition"

The word "partition" appears in three contexts in distributed systems:

1. **Data partition.** A subset of the data, assigned to a Raft group. This is the meaning in this chapter.
2. **Raft group / partition.** The Raft consensus group that manages a data partition. In Kommander, these are the same thing: one Raft group per data partition.
3. **Network partition.** A failure that splits the network into isolated groups (Chapter 9).

Context makes the meaning clear. When this book uses "partition" without a qualifier, it means a Raft group (senses 1 and 2). "Network partition" always has the "network" qualifier.

11.9 Summary

- One Raft group means one leader. One leader is a write throughput bottleneck.
- Partitioned Raft runs multiple independent Raft groups in the same cluster. Each partition has its own leader, term, log, and state machine.
- `InitialPartitions` sets the number of application partitions at bootstrap. Partition 0 is reserved for system use.
- `GetPartitionKey` hashes a string key to a partition id. The same key always maps to the same partition.
- Each partition must have its own state machine (separate data store). The `partitionId` parameter in the callbacks identifies which partition the entry belongs to.
- A node failure affects only the partitions that node led. Other partitions continue without interruption.
- More partitions allow better distribution of leadership across nodes. A practical default is 2 to 4 times the node count.
- The next chapter builds a fully partitioned ReplicatedStore with per-partition routing and state machines.

12 Partitioned Raft

Chapter 11 explained why a single Raft group cannot scale. This chapter builds the full picture: a partitioned ReplicatedStore with per-partition state machines, per-partition routing, and proper handling of the system partition, the partition map, and the exceptions that arise when a partition is not hosted on the local node.

12.1 The Two Kinds of Partitions

Kommander separates partitions into two categories:

12.1.1 Partition 0: The System Partition

Partition 0 is reserved. Kommander uses it to replicate cluster-wide configuration: the partition map, membership changes, and internal coordination entries. System entries use the `_RaftSystem` log type.

Rules:

- Do not call `ReplicateLogs` on partition 0 with application data.
- Do not register application logic in `OnReplicationReceived` for partition 0.
- Do not route application keys to partition 0.
- `GetPartitionKey` never returns 0.

12.1.2 Partitions 1 and Above: Application Partitions

All application data goes into partitions 1 and above. These are the partitions your code controls. Each one is an independent Raft group with its own:

- Leader and term
- Log (WAL entries)
- Election timer
- State machine

```
Partition 0 (system)     managed by Kommander
──────────────────────────────────────────────

Partition 1 (application) ── your state machine
Partition 2 (application) ── your state machine
Partition 3 (application) ── your state machine
...
Partition N (application) ── your state machine
```

12.2 Per-Partition State Machines

Each partition has its own log. The state machine must keep each partition's data separate. A command committed on partition 3 must not change the state of partition 5.

12.2.1 The Pattern

```
var stores = new ConcurrentDictionary<int, ConcurrentDictionary<string,
string>>();

ConcurrentDictionary<string, string> GetStore(int partitionId)
{
    return stores.GetOrAdd(
        partitionId,
        _ => new ConcurrentDictionary<string, string>()
    );
}

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
```

```
if (partitionId == 0)
    return Task.FromResult(true);

var store = GetStore(partitionId);

switch (log.LogType)
{
    case "SetValue":
    {
        var cmd = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );
        if (cmd is not null)
            store[cmd.Key] = cmd.Value;
        break;
    }

    case "DeleteValue":
    {
        var cmd = JsonSerializer.Deserialize<DeleteValueCommand>(
            log.LogData ?? []
        );
        if (cmd is not null)
            store.TryRemove(cmd.Key, out _);
        break;
    }

    case "IncrementCounter":
    {
        var cmd = JsonSerializer.Deserialize<IncrementCounterCommand>(
            log.LogData ?? []
        );
        if (cmd is not null)
        {
            int current = int.Parse(
                store.GetValueOrDefault(cmd.Key, "0")
            );
            store[cmd.Key] = (current + 1).ToString();
        }
        break;
    }
}

return Task.FromResult(true);
```

```
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;
```

Key points:

- The first parameter (`partitionId`) identifies the partition. The same callback fires for every partition on this node.
- The guard `if (partitionId == 0) return` skips system partition entries. Kommander handles those internally.
- `GetStore(partitionId)` returns a separate dictionary for each partition. Data stays isolated.

12.2.2 Why Isolation Matters

If the state machine uses a single shared dictionary for all partitions, two problems arise:

1. **Key collisions.** Partition 2 and partition 5 could both contain a key named `"balance"`. Without per-partition dictionaries, one overwrites the other.
2. **Cross-partition corruption.** A replay of partition 3's log would see entries from partition 7 in the same dictionary. The state would be wrong.

Per-partition isolation is not optional. It is a correctness requirement.

12.3 The Partition Map

The partition map is the cluster's record of which partitions exist, their hash ranges, and their lifecycle states. `GetPartitionMap` returns the current map:

```
IReadOnlyList<RaftPartitionRange> map = raft.GetPartitionMap();

foreach (RaftPartitionRange range in map)
{
    Console.WriteLine(
        $"Partition {range.PartitionId}: " +
        $"[{range.StartRange}..{range.EndRange}] " +
        $"gen={range.Generation} state={range.State}"
    );
}
```

12.3.1 RaftPartitionRange Properties

Property	Type	Description
`PartitionId`	`int`	Unique identifier for this partition
`StartRange`	`int`	Start of the hash range (inclusive)
`EndRange`	`int`	End of the hash range (inclusive)
`Generation`	`long`	Incremented on each map mutation
`State`	`RaftPartitionState`	`Active`, `Splitting`, `Draining`, or `Removed`
`RoutingMode`	`RaftRoutingMode`	Whether `GetPartitionKey` includes this partition
`Replicas`	`List<RaftReplica>`	Nodes hosting this partition
`ReplicationFactor`	`int`	Desired voter replicas (0 = use cluster default)

12.3.2 Partition States

The `RaftPartitionState` enum has four values:

State	Meaning
`Active`	Normal operation. Accepts reads and writes.
`Splitting`	A split is in progress. The partition still serves traffic.
`Draining`	The partition is being removed. Traffic is moving away.
`Removed`	The partition is gone. Its id is never reused.

During normal operation, all application partitions are `Active`. The other states appear during partition lifecycle operations (Chapter 13).

12.3.3 Hash Range Layout

At bootstrap with `InitialPartitions = 4`, the hash space is divided evenly:

```
Hash space: 0 ──────────────────────────────────── 2^31

Partition 1: [0 ............. 536,870,911]
```

```
  Partition 2: [536,870,912 ... 1,073,741,823]
  Partition 3: [1,073,741,824 . 1,610,612,735]
  Partition 4: [1,610,612,736 . 2,147,483,647]
```

`GetPartitionKey("some-key")` hashes the string, takes the result modulo the hash space, and returns the partition that owns that range.

12.4 OnPartitionMapChanged

The `OnPartitionMapChanged` event fires whenever the partition map changes:

```
raft.OnPartitionMapChanged += (map) =>
{
    Console.WriteLine($"Partition map changed: {map.Count} partitions");

    foreach (RaftPartitionRange range in map)
    {
        Console.WriteLine(
            $"  P{range.PartitionId}: state={range.State}, " +
            $"gen={range.Generation}"
        );
    }
};
```

The callback receives a snapshot of the partition map at the time of the change. The snapshot is a point-in-time copy. Changing the snapshot does not affect the live map.

Use this event for:

- **Routing table updates.** Refresh cached routing information when partitions are added, split, or removed.
- **State machine cleanup.** When a partition is removed, clean up the per-partition data store.
- **Logging.** Record partition map changes for operational visibility.

The callback runs on the system coordinator's single-consumer loop. Do not block inside it. Do not call Kommander APIs that re-enter the coordinator.

12.5 HostsPartition

With replica placement (Chapter 13), not every node hosts every partition. A seven-node cluster with a replication factor of 3 hosts each partition on only three nodes. The other four nodes do not have that partition's data.

`HostsPartition` checks whether a partition is materialized on the local node:

```
bool hosted = raft.HostsPartition(partitionId);
```

If `hosted` is `true`, the partition is on this node. You can call `AmILeader`, `ReplicateLogs`, and other per-partition APIs.

If `hosted` is `false`, the partition is not on this node. Per-partition API calls will throw `PartitionNotHostedException`.

12.5.1 Advisory, Not a Lock

`HostsPartition` reads the local materialized-partition dictionary. It is accurate at the time of the read. But hosting can change immediately after the call: a replica move can un-host a partition at any time.

Use `HostsPartition` as a fast-path routing check. Do not treat it as a guarantee. Always handle `PartitionNotHostedException` even after a `true` result.

12.6 PartitionNotHostedException

When you call a per-partition API on a partition that is not hosted on this node, Kommander throws `PartitionNotHostedException`:

```
try
{
    var result = await raft.ReplicateLogs(
        partitionId, "SetValue", payload
    );
}
catch (PartitionNotHostedException ex)
{
    // This partition is not on this node.
    // Route to a node that hosts it.
    IReadOnlyList<RaftReplica> replicas =
        raft.GetPartitionReplicas(ex.PartitionId);

    // Pick a replica and forward the request.
}
```

This exception is **not an error**. Under replica placement, most partitions on most nodes are not hosted. The exception is a normal routing signal. Catch it and forward the request to a node that hosts the partition.

12.6.1 Which APIs Throw It

These per-partition APIs throw `PartitionNotHostedException` when the partition is not materialized on this node:

- `ReplicateLogs`
- `ReplicateEntries`
- `ReplicateCheckpoint`
- `CommitLogs`
- `RollbackLogs`
- `ConfirmLeadershipAsync`
- `ConfirmLocalApplicationAsync`
- `AmILeader`
- `WaitForLeader`

These APIs do **not** throw it:

- `GetPartitionKey` (hash-only, no partition needed)
- `GetPartitionMap` (reads the global map)
- `GetPartitionReplicas` (reads the global map)
- `GetPartitionLeaderHint` (works for non-hosted partitions via gossip)
- `HostsPartition` (the check itself)

12.7 The Routing Decision Tree

When a request arrives, the application must decide how to handle it:

```
Request arrives with key
     |
     ▼
partitionId = GetPartitionKey(key)
     |
     ▼
HostsPartition(partitionId)?
     |
     ├── No
     |     └── Forward to a node from GetPartitionReplicas(partitionId)
     |         or use GetPartitionLeaderHint(partitionId)
     |
     └── Yes
          |
          ▼
      AmILeader(partitionId)?
          |
          ├── No
          |     └── Return 503 with GetPartitionLeaderHint(partitionId)
          |
          └── Yes
               |
               ▼
           Handle the request locally
```

12.7.1 Implementation

```
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    int partitionId = raft.GetPartitionKey(key);

    // Check if this node hosts the partition
    if (!raft.HostsPartition(partitionId))
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "PartitionNotHosted", leader, partitionId },
            statusCode: 503
```

```
    );
}

// Check leadership
bool isLeader;
try
{
    isLeader = await raft.AmILeader(partitionId, default);
}
catch (PartitionNotHostedException)
{
    string? leader = raft.GetPartitionLeaderHint(partitionId);
    return Results.Json(
        new { error = "PartitionNotHosted", leader, partitionId },
        statusCode: 503
    );
}

if (!isLeader)
{
    string? leader = raft.GetPartitionLeaderHint(partitionId);
    return Results.Json(
        new { error = "NotLeader", leader, partitionId },
        statusCode: 503
    );
}

// Handle the write
using var reader = new StreamReader(request.Body);
string value = await reader.ReadToEndAsync();

byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
    new SetValueCommand(key, value)
);

using var timeout = new CancellationTokenSource(
    TimeSpan.FromSeconds(5)
);

try
{
    var result = await raft.ReplicateLogs(
        partitionId, "SetValue", payload,
        cancellationToken: timeout.Token
```

```
        );

        if (result.Success)
            return Results.Ok(new { key, value, partitionId });

        return Results.Json(
            new { error = result.Status.ToString(), partitionId },
            statusCode: 503
        );
    }
    catch (PartitionNotHostedException)
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "PartitionMoved", leader, partitionId },
            statusCode: 503
        );
    }
});
```

The handler catches `PartitionNotHostedException` in two places:

1. After `AmILeader`. The partition could have moved between the `HostsPartition` check and the `AmILeader` call.
2. After `ReplicateLogs`. The partition could have moved between the `AmILeader` check and the replication attempt.

Both catches return the same response: a 503 with a leader hint so the client can retry on the correct node.

12.8 The Read Handler

```
app.MapGet("/store/{key}", async (string key, HttpRequest request) =>
{
    int partitionId = raft.GetPartitionKey(key);

    if (!raft.HostsPartition(partitionId))
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "PartitionNotHosted", leader, partitionId },
            statusCode: 503
        );
    }

    var store = GetStore(partitionId);
    bool consistent = request.Query.ContainsKey("consistent");
```

```
    if (consistent)
    {
        using var timeout = new CancellationTokenSource(
            TimeSpan.FromSeconds(3)
        );

        try
        {
            bool confirmed = await raft.ConfirmLeadershipAsync(
                partitionId, timeout.Token
            );

            if (!confirmed)
            {
                string? leader = raft.GetPartitionLeaderHint(partitionId);
                return Results.Json(
                 new { error = "LeadershipNotConfirmed", leader, partitionId },
                    statusCode: 503
                );
            }
        }
        catch (PartitionNotHostedException)
        {
            string? leader = raft.GetPartitionLeaderHint(partitionId);
            return Results.Json(
                new { error = "PartitionNotHosted", leader, partitionId },
                statusCode: 503
            );
        }
    }

    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value, partitionId })
        : Results.NotFound();
});
```

12.9 Wrong-Partition Routing

Routing correctness is the application's responsibility. Kommander routes keys to partitions through `GetPartitionKey`. If the application bypasses this and sends a key to the wrong partition, the data is misplaced.

12.9.1 The Mistake

```
// WRONG: hardcoded partition id
int partitionId = 3;
```

```
byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
    new SetValueCommand("customer:42", "Alice")
);

await raft.ReplicateLogs(partitionId, "SetValue", payload);
```

`GetPartitionKey("customer:42")` returns partition 5. But the code wrote to partition 3. The write succeeds on partition 3. But a read using `GetPartitionKey("customer:42")` queries partition 5 and finds nothing.

The data exists but is invisible to any code that uses `GetPartitionKey` for reads.

12.9.2 The Fix

Always use `GetPartitionKey` for routing:

```
// CORRECT: let GetPartitionKey decide
int partitionId = raft.GetPartitionKey("customer:42");

byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
    new SetValueCommand("customer:42", "Alice")
);

await raft.ReplicateLogs(partitionId, "SetValue", payload);
```

Now writes and reads use the same routing function. The data goes to the correct partition.

12.10 The Complete Partitioned ReplicatedStore

Here is the full program with 8 partitions across 3 nodes:

```
using Kommander;
using Kommander.Communication.Grpc;
using Kommander.Discovery;
using Kommander.WAL;
using Kommander.Data;
using System.Collections.Concurrent;
using System.Text.Json;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddKommanderGrpc();

var app = builder.Build();

string nodeName = args[0];
int port = int.Parse(args[1]);

RaftConfiguration configuration = new()
```

```
{
    NodeName = nodeName,
    Host = "localhost",
    Port = port,
    InitialPartitions = 8
};

List<RaftNode> nodes = [
    new("localhost:8001"),
    new("localhost:8002"),
    new("localhost:8003")
];

StaticDiscovery discovery = new(nodes);
RocksDbWAL wal = new($"data/{nodeName}", "v1", app.Logger);
GrpcCommunication communication = new();
HybridLogicalClock clock = new();

RaftManager raft = new(
    configuration, discovery, wal, communication, clock,
    app.Logger
);

// Per-partition state machines
var stores = new ConcurrentDictionary<int, ConcurrentDictionary<string,
string>>();

ConcurrentDictionary<string, string> GetStore(int partitionId)
{
    return stores.GetOrAdd(
        partitionId,
        _ => new ConcurrentDictionary<string, string>()
    );
}

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (partitionId == 0)
        return Task.FromResult(true);

    var store = GetStore(partitionId);

    switch (log.LogType)
    {
```

```
        case "SetValue":
        {
            var cmd = JsonSerializer.Deserialize<SetValueCommand>(
                log.LogData ?? []
            );
            if (cmd is not null)
                store[cmd.Key] = cmd.Value;
            break;
        }

        case "DeleteValue":
        {
            var cmd = JsonSerializer.Deserialize<DeleteValueCommand>(
                log.LogData ?? []
            );
            if (cmd is not null)
                store.TryRemove(cmd.Key, out _);
            break;
        }
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;

// Write endpoint
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    int partitionId = raft.GetPartitionKey(key);

    if (!raft.HostsPartition(partitionId))
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "PartitionNotHosted", leader, partitionId },
            statusCode: 503
        );
    }

    try
    {
        bool isLeader = await raft.AmILeader(partitionId, default);
```

```
        if (!isLeader)
        {
            string? leader = raft.GetPartitionLeaderHint(partitionId);
            return Results.Json(
                new { error = "NotLeader", leader, partitionId },
                statusCode: 503
            );
        }

        using var reader = new StreamReader(request.Body);
        string value = await reader.ReadToEndAsync();

        byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
            new SetValueCommand(key, value)
        );

        using var timeout = new CancellationTokenSource(
            TimeSpan.FromSeconds(5)
        );

        var result = await raft.ReplicateLogs(
            partitionId, "SetValue", payload,
            cancellationToken: timeout.Token
        );

        if (result.Success)
            return Results.Ok(new { key, value, partitionId });

        return Results.Json(
            new { error = result.Status.ToString(), partitionId },
            statusCode: 503
        );
    }
    catch (PartitionNotHostedException)
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "PartitionMoved", leader, partitionId },
            statusCode: 503
        );
    }
});

// Read endpoint
```

```
app.MapGet("/store/{key}", async (string key, HttpRequest request) =>
{
    int partitionId = raft.GetPartitionKey(key);

    if (!raft.HostsPartition(partitionId))
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "PartitionNotHosted", leader, partitionId },
            statusCode: 503
        );
    }

    var store = GetStore(partitionId);

    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value, partitionId })
        : Results.NotFound();
});

app.MapGrpcRaftRoutes(raft);
await raft.JoinCluster();
app.Run($"http://localhost:{port + 1000}");

record SetValueCommand(string Key, string Value);
record DeleteValueCommand(string Key);
```

12.10.1 Running It

```
# Terminal 1
dotnet run -- node-a 8001

# Terminal 2
dotnet run -- node-b 8002

# Terminal 3
dotnet run -- node-c 8003
```

12.10.2 Testing Partition Routing

```
# Write to different keys (may go to different partitions)
curl -X PUT http://localhost:9001/store/user:1 -d '"Alice"'
curl -X PUT http://localhost:9001/store/user:2 -d '"Bob"'
curl -X PUT http://localhost:9001/store/order:100 -d '"pending"'

# If a key routes to a partition this node does not lead,
```

```
# the response includes the leader hint:
# {"error":"NotLeader","leader":"localhost:8002","partitionId":5}
```

Different keys hash to different partitions. Different partitions have different leaders. The client uses the `leader` and `partitionId` fields in error responses to route retries to the correct node.

12.11 Summary

- Partition 0 is the system partition. Kommander manages it. Do not store application data in it.
- Partitions 1 and above are application partitions. Each has its own leader, log, and state machine.
- Per-partition state machines must keep data isolated. One dictionary per partition. A shared dictionary causes key collisions and cross-partition corruption.
- `OnPartitionMapChanged` fires when partitions are added, split, moved, or removed. Use it to refresh routing.
- `HostsPartition` checks whether a partition is materialized on this node. It is a fast-path check, not a guarantee.
- `PartitionNotHostedException` is a normal routing signal under replica placement, not an error. Catch it and forward to a hosting node.
- Always use `GetPartitionKey` for routing. Hardcoded partition ids lead to misplaced data.
- The next chapter introduces replica placement and replication factors for controlling which nodes host which partitions.

13 Replica Placement and Replication Factor

In every example so far, every node stored every partition. A three-node cluster with 8 partitions had 24 partition copies (8 per node). This is **full replication**: safe, simple, and wasteful at scale.

A seven-node cluster with 100 partitions under full replication stores 700 partition logs. Each node runs 100 election timers and 100 state machines. Most of that work is unnecessary. If each partition lives on only 3 nodes (replication factor 3), the total drops to 300 partition copies. Each node holds about 43 partitions instead of 100.

This chapter explains how to configure the replication factor, how Kommander places replicas on specific nodes, and how the placement controller repairs under-replicated partitions.

13.1 Full Replication vs. Fixed Replication Factor

Mode	Storage per node	Quorum basis	Node cost to add
Full replication (RF=0)	All partitions	All roster voters	All partition data
RF=3 on 7 nodes	~43% of partitions	3 voters per range	Only assigned partitions

With full replication, quorum for each partition is computed over the entire cluster. In a 7-node cluster, quorum is 4 of 7. A write must reach 4 nodes before it commits.

With RF=3, quorum for each partition is computed over its 3 replicas. Quorum is 2 of 3. A write must reach 2 of the 3 hosting nodes. The other 4 nodes do not participate at all.

```
Full replication (7 nodes):

P1: [A] [B] [C] [D] [E] [F] [G]    quorum = 4 of 7
P2: [A] [B] [C] [D] [E] [F] [G]    quorum = 4 of 7

RF = 3 (7 nodes):

P1: [A] [B] [C]                     quorum = 2 of 3
P2: [C] [D] [E]                     quorum = 2 of 3
P3: [E] [F] [G]                     quorum = 2 of 3
P4: [G] [A] [B]                     quorum = 2 of 3
```

RF=3 uses fewer resources and commits faster (2 acks instead of 4). The trade-off: if 2 of the 3 replicas for a partition fail, that partition is unavailable. With full replication, 4 of the 7 nodes must fail before any partition becomes unavailable.

13.2 Configuring the Replication Factor

Set `ReplicationFactor` on `RaftConfiguration`:

```
RaftConfiguration configuration = new()
{
    NodeName = "node-a",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 16,
    ReplicationFactor = 3
};
```

When `ReplicationFactor` is 0 (the default), every roster voter hosts every partition. This is full replication, the behavior in all previous chapters.

When `ReplicationFactor` is greater than 0, initial placement assigns each partition a replica set of that size. The placement controller spreads replicas evenly across the available nodes.

13.2.1 Odd Values

Prefer odd values for the replication factor:

RF	Quorum	Tolerates failures
2	2 of 2	0 (any failure blocks writes)
3	2 of 3	1
4	3 of 4	1
5	3 of 5	2

RF=4 tolerates only 1 failure, just like RF=3. The extra replica costs storage and network without increasing fault tolerance. Use odd values: 3, 5, or 7.

13.2.2 Cluster Size and Replication Factor

If the cluster has fewer voters than the configured factor, Kommander falls back to full replication. A 2-node cluster with RF=3 replicates to both nodes. No data is lost. When a third node joins, the placement controller assigns replicas normally.

13.3 Replica Sets

Each partition has a **replica set**: the specific nodes that host it. `GetPartitionReplicas` returns the set:

```
IReadOnlyList<RaftReplica> replicas =
    raft.GetPartitionReplicas(partitionId);

foreach (RaftReplica replica in replicas)
{
    Console.WriteLine(
        $"  {replica.Endpoint} role={replica.Role}"
    );
}
```

13.3.1 RaftReplica

```
public sealed class RaftReplica
{
    public string Endpoint { get; set; }
    public int NodeId { get; set; }
    public RaftReplicaRole Role { get; set; }
    public long SinceGeneration { get; set; }
}
```

Property	Description
`Endpoint`	The node's `host:port` address
`NodeId`	The node's numeric id
`Role`	`Voter`, `Learner`, or `Removing`
`SinceGeneration`	The partition generation when this replica entered the set

13.3.2 RaftReplicaRole

The `RaftReplicaRole` enum has three values:

Role	Quorum	Elections	Description
`Voter`	Counts toward quorum	Can campaign	Full voting replica
`Learner`	Excluded from quorum	Cannot campaign	Catch-up replica, promoted to Voter when caught up
`Removing`	Excluded from quorum	Cannot campaign	Marked for removal, still hosts data until dropped

13.3.3 Empty Replica List

`GetPartitionReplicas` returns an empty list for partitions under full replication. An empty list means "every roster voter hosts this range." This is the legacy mode and the default when `ReplicationFactor` is 0.

13.4 Routing with Replica Sets

Under full replication, every node hosts every partition. The routing from Chapter 12 checks `HostsPartition` and `AmILeader`. Under a fixed replication factor, most nodes do not host most partitions. The routing must account for this.

13.4.1 GetPartitionLeaderHint

`GetPartitionLeaderHint` works even for partitions the local node does not host:

```
string? leader = raft.GetPartitionLeaderHint(partitionId);
```

For a locally hosted partition, the hint is the node's own leader belief. For a remote partition, the hint comes from gossiped load reports. Each node advertises the partitions it believes it leads. `GetPartitionLeaderHint` aggregates these reports and returns the freshest claim.

The hint is best-effort. It may be stale or null. Use it as a first attempt. If the target rejects the request (`NotLeader` or `PartitionNotHostedException`), fall back to `GetPartitionReplicas` and try another node.

13.4.2 Routing Pattern

```
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    int partitionId = raft.GetPartitionKey(key);

    // Fast path: handle locally if hosted and leading
    if (raft.HostsPartition(partitionId))
    {
        try
        {
            bool isLeader = await raft.AmILeader(partitionId, default);
            if (isLeader)
            {
                // Handle write locally...
                return await HandleWrite(partitionId, key, request);
            }
        }
        catch (PartitionNotHostedException) { }
    }

    // Not hosted or not leader. Return routing info.
    string? leader = raft.GetPartitionLeaderHint(partitionId);

    if (leader is null)
    {
        // No leader hint. Give the client the replica set.
        var replicas = raft.GetPartitionReplicas(partitionId);
        var endpoints = replicas
            .Where(r => r.Role == RaftReplicaRole.Voter)
            .Select(r => r.Endpoint)
            .ToList();

        return Results.Json(
            new { error = "NoLeaderHint", replicas = endpoints, partitionId },
            statusCode: 503
        );
    }

    return Results.Json(
        new { error = "NotLeader", leader, partitionId },
        statusCode: 503
    );
});
```

13.5 GetEffectiveReplicationFactor

Each partition can have its own replication factor. `GetEffectiveReplicationFactor` returns the active value:

```
int rf = raft.GetEffectiveReplicationFactor(partitionId);
```

If the partition has a per-range override, this returns the override. Otherwise, it returns the global `RaftConfiguration.ReplicationFactor`. A return value of 0 means full replication.

13.5.1 SetReplicationFactorAsync

Change the replication factor for a single partition at runtime:

```
var result = await raft.SetReplicationFactorAsync(
    partitionId,
    replicationFactor: 5,
    ct: cancellationToken
);
```

This commits a per-range override through partition 0 (the system partition). The change is leader-only. The placement controller adjusts the replica count on subsequent passes.

Setting the value to 0 clears the override. The partition inherits the global `ReplicationFactor` again.

This does not immediately add or remove replicas. It sets the target. The placement controller moves replicas toward the target over time.

13.6 The Placement Controller

The placement controller runs on the partition 0 leader. It compares the current replica sets against the target replication factors and plans moves to converge.

13.6.1 What It Does

On each pass, the controller:

1. Reads the committed partition map and the current node roster.
2. For each partition, compares the actual voter count against the target replication factor.
3. Plans moves to reach the target:
   - **Under-replicated:** Add a learner on a node that does not yet host the partition. After the learner catches up, promote it to voter.
   - **Over-replicated:** Mark the excess voter as removing. After a subsequent commit, drop it from the set.
   - **Misplaced (zone-aware):** If two voters share a zone while another zone has no voter, move one.

13.6.2 The Repair Sequence

When a node fails, its partitions become under-replicated:

```
Before failure:

P1: [A Voter] [B Voter] [C Voter]   RF=3, quorum=2 ✓

Node C crashes:

P1: [A Voter] [B Voter]             RF=3, only 2 voters ✗

Placement controller detects under-replication.

Step 1: Add learner on Node D.
P1: [A Voter] [B Voter] [D Learner]

Step 2: D catches up (receives all log entries from the leader).

Step 3: Promote D to voter.
P1: [A Voter] [B Voter] [D Voter]   RF=3, quorum=2 ✓
```

During the repair:

- The partition stays available (A and B form a quorum).
- The learner does not count toward quorum, so it cannot break consensus.
- Once promoted, D is a full voter and the partition tolerates one more failure.

13.7 Placement Configuration

Several properties control the speed and scope of placement moves:

13.7.1 PlacementPassInterval

```
PlacementPassInterval = TimeSpan.FromSeconds(5)    // default
```

How often the controller runs a pass. A relocation takes about 3 passes: add learner, observe it caught up, promote. With a 5-second interval, a replica move takes approximately 15 seconds (plus data transfer time).

13.7.2 MaxReplicaMovesPerPass

```
MaxReplicaMovesPerPass = 4    // default
```

The maximum number of new moves started per pass (across all priorities). This bounds the blast radius of a bad plan. Set this to at least `MaxConcurrentReplicaRepairs` + `MaxConcurrentReplicaTransfers`. Otherwise it starves repairs.

13.7.3 MaxConcurrentReplicaRepairs

```
MaxConcurrentReplicaRepairs = 3    // default
```

The maximum number of in-flight **repair** moves. Repairs restore under-replicated partitions and remove replicas stranded on evicted nodes. This budget is separate from the balance budget so that repairs are not serialized behind cosmetic rebalancing.

13.7.4 MaxConcurrentReplicaTransfers

```
MaxConcurrentReplicaTransfers = 1    // default
```

The maximum number of in-flight **balance** moves (cosmetic skew spreading). These are lower priority than repairs. If 3 repairs are in flight, balance moves pause until a repair slot frees up.

13.7.5 ReplicaCountDeadband

```
ReplicaCountDeadband = 1    // default
```

The minimum per-node replica-count imbalance before the planner creates balancing moves. This prevents unnecessary moves when the distribution is nearly even. Under-replicated repairs bypass the deadband. Restoring the replication factor is always the highest priority.

13.7.6 EnablePlacementRebalancer

```
EnablePlacementRebalancer = true
```

The master switch for the continual rebalancer. When `false` (the default), the controller drives in-flight transitions to completion (learner promotions and removal drops) but does not plan new balancing moves.

When `true`, the controller actively rebalances: it spreads replicas evenly across nodes, respects zone constraints, and moves replicas away from overloaded nodes.

Initial placement at bootstrap happens regardless of this flag. `EnablePlacementRebalancer` controls only the ongoing rebalancing.

13.8 Zone-Aware Placement

Set `Zone` on each node to group nodes by failure domain (data center, rack, availability zone):

```
// Node in zone "us-east-1a"
RaftConfiguration configuration = new()
{
    NodeName = "node-a",
    Host = "10.0.1.10",
    Port = 8001,
    ReplicationFactor = 3,
    EnablePlacementRebalancer = true,
    Zone = "us-east-1a"
};
```

When zones are configured, the placement planner spreads a partition's replicas across distinct zones. It prefers to place each voter in a different zone.

```
Without zone awareness:

P1: [A us-east-1a] [B us-east-1a] [C us-east-1a]
```

```
If us-east-1a fails, P1 loses all 3 voters. Unavailable.


With zone awareness:

P1: [A us-east-1a] [D us-east-1b] [F us-east-1c]

If us-east-1a fails, P1 still has 2 voters in other zones. Available.
```

The rebalancer also repairs zone violations. If a partition has two voters in the same zone while another zone has a free node, the rebalancer moves one voter to the uncovered zone.

13.8.1 Zone Gossip

Zone information travels through load-report gossip. Each node advertises its zone on its load reports. The placement planner on the P0 leader reads these reports to see remote nodes' zones.

Gossip starts after membership is seeded, which happens after the initial partition map is committed. The initial placement at bootstrap cannot see remote zones. The rebalancer converges the zone spread shortly after bring-up.

13.9 Example: Seven-Node Cluster with RF=3

```
RaftConfiguration configuration = new()
{
    NodeName = nodeName,
    Host = host,
    Port = port,
    InitialPartitions = 12,
    ReplicationFactor = 3,
    EnablePlacementRebalancer = true,
    Zone = zone   // "zone-a", "zone-b", or "zone-c"
};
```

After bootstrap, each partition has 3 replicas spread across 7 nodes:

```
Node A (zone-a):  P1, P4, P7, P10          (4 partitions)
Node B (zone-a):  P2, P5, P8               (3 partitions)
Node C (zone-b):  P1, P3, P6, P9           (4 partitions)
Node D (zone-b):  P2, P5, P11              (3 partitions)
Node E (zone-c):  P3, P6, P10, P12         (4 partitions)
Node F (zone-c):  P4, P7, P8, P11          (4 partitions)
Node G (zone-a):  P9, P12                  (2 partitions)

Each partition is on 3 nodes, spread across 2-3 zones.
```

13.9.1 Node Loss

Node C crashes. Partitions P1, P3, P6, P9 lose one replica each:

```
  P1: [A zone-a] [C zone-b] → [A zone-a]        1 voter left → under-replicated
  P3: [C zone-b] [E zone-c] → [E zone-c]         need repair
  P6: [C zone-b] [E zone-c] → [E zone-c]         need repair
  P9: [C zone-b] [G zone-a] → [G zone-a]         need repair
```

The placement controller detects 4 under-replicated partitions. With `MaxConcurrentReplicaRepairs = 3`, it starts 3 repairs in parallel (the 4th waits for a slot). For each repair:

1. Add a learner on a node in a different zone from the existing voters.
2. Wait for the learner to catch up.
3. Promote the learner to voter.

Within a few passes (15 to 30 seconds depending on data size), all 4 partitions are restored to RF=3 with proper zone spread.

13.10 Querying the Effective Factor

```
// Read the effective replication factor for partition 5
int rf = raft.GetEffectiveReplicationFactor(5);
Console.WriteLine($"Partition 5: RF={rf}");

// Change it at runtime
await raft.SetReplicationFactorAsync(5, 5, cancellationToken);

// Verify
rf = raft.GetEffectiveReplicationFactor(5);
Console.WriteLine($"Partition 5: RF={rf}");   // now 5
```

The placement controller adds 2 learners on the next passes and promotes them when they catch up.

13.11 Summary

- `ReplicationFactor` controls how many nodes store each partition. 0 means full replication (the default). 3, 5, or 7 are common production values.
- Quorum is computed per partition over its voter replicas, not over the entire cluster. RF=3 means quorum is 2 of 3.
- `GetPartitionReplicas` returns the replica set for a partition. An empty list means full replication.
- `RaftReplicaRole` has three values: `Voter` (counts toward quorum), `Learner` (catching up), and `Removing` (marked for removal).
- `GetEffectiveReplicationFactor` returns the active factor for a partition. `SetReplicationFactorAsync` changes it at runtime.
- The placement controller runs on the P0 leader and repairs under-replicated partitions by adding learners and promoting them.
- `EnablePlacementRebalancer` turns on ongoing rebalancing. Without it, only in-flight transitions and repairs run.

- `Zone` groups nodes by failure domain. The planner spreads replicas across distinct zones.
- `PartitionNotHostedException` is the normal routing signal when a partition is not on this node. Catch it and forward to a hosting node.
- The next chapter covers elastic partitions: splitting and merging partitions at runtime.

14 Generation Fencing and Routing Safety

Chapter 14 showed how to split and merge partitions at runtime. Each structural change increments the partition's **generation**, a monotonically increasing counter. This chapter explains how to use that counter to prevent a dangerous class of bugs: stale routing.

14.1 The Problem: Stale Routing

A client (or a server-side proxy) caches the partition map. It reads the map at time T=0 and learns that partition 1 covers the full hash range `[0, 2,147,483,647]` at generation 7.

At time T=1, the cluster splits partition 1. The lower half stays with partition 1. The upper half goes to partition 9. Both partitions advance to generation 8.

At time T=2, the client sends a write for a key that hashes into the upper range. The client still believes partition 1 covers the full range. It sends the write to partition 1.

14.1.1 Without Generation Fencing

The write succeeds on partition 1. The key lands in the wrong partition. Reads that correctly route to partition 9 will not find it. The data is misplaced, and no error was reported.

```
T=0   Client reads map: P1 [0..MAX], gen=7
T=1   Cluster splits: P1 [0..MID] gen=8, P9 [MID+1..MAX] gen=8
T=2   Client writes key (hash in upper range) to P1
      P1 accepts the write ← WRONG
T=3   Correct reads go to P9 → key not found
```

The data exists but is invisible to correctly-routed readers. This is silent data misplacement. No exception was thrown. No error status was returned. The client believed the write succeeded, and it did, but in the wrong place.

14.1.2 With Generation Fencing

The client passes `expectedGeneration: 7` with the write. Partition 1's generation is now 8. The mismatch causes Kommander to reject the write with `PartitionMoved`.

```
T=0   Client reads map: P1 [0..MAX], gen=7
T=1   Cluster splits: P1 [0..MID] gen=8, P9 [MID+1..MAX] gen=8
T=2   Client writes key to P1 with expectedGeneration=7
      P1 generation is 8 → REJECTED (PartitionMoved)
```

```
T=3   Client refreshes map, sees P9, retries on P9 with gen=8
      P9 accepts → CORRECT
```

The fence turns silent misplacement into an explicit rejection. The client learns that its map is stale, refreshes it, and retries on the correct partition.

14.2 The expectedGeneration Parameter

Every `ReplicateLogs` overload has an `expectedGeneration` parameter:

```
Task<RaftReplicationResult> ReplicateLogs(
    int partitionId,
    string type,
    byte[] data,
    bool autoCommit = true,
    long expectedGeneration = 0,
    CancellationToken cancellationToken = default
);
```

The behavior is:

- **`expectedGeneration = 0` (default).** No fence. The write proceeds regardless of the partition's generation. This is the behavior in all previous chapters.
- **`expectedGeneration > 0`.** The proposal is checked against the partition's committed generation. If they match, the write proceeds. If they do not match, the write is rejected with `RaftOperationStatus.PartitionMoved`.

14.2.1 When the Fence Fires

The generation check happens at proposal admission, before the entry is appended to the WAL. A rejected entry never enters the log. It is as if the write never happened.

```
Client calls ReplicateLogs with expectedGeneration=7
     │
     ▼
Partition executor checks:
     partition.Generation == 7?
     │
     ├── Yes → proceed with proposal
     │
     └── No  → return PartitionMoved immediately
```

14.3 GetPartitionGeneration

Query the current generation for a partition:

```
long generation = raft.GetPartitionGeneration(partitionId);
```

This reads from the in-memory partition dictionary. No WAL I/O. The cost is a dictionary lookup.

Returns 0 if the partition does not exist.

14.4 Using Generation Fencing in ReplicatedStore

14.4.1 Server-Side Fencing

The server reads the generation before each write and passes it to `ReplicateLogs`:

```
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    int partitionId = raft.GetPartitionKey(key);

    if (!raft.HostsPartition(partitionId))
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "PartitionNotHosted", leader, partitionId },
            statusCode: 503
        );
    }

    try
    {
        bool isLeader = await raft.AmILeader(partitionId, default);
        if (!isLeader)
        {
            string? leader = raft.GetPartitionLeaderHint(partitionId);
            return Results.Json(
                new { error = "NotLeader", leader, partitionId },
                statusCode: 503
            );
        }

        using var reader = new StreamReader(request.Body);
        string value = await reader.ReadToEndAsync();

        byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
            new SetValueCommand(key, value)
        );

        // Read the generation and pass it as a fence
        long generation = raft.GetPartitionGeneration(partitionId);

        using var timeout = new CancellationTokenSource(
            TimeSpan.FromSeconds(5)
        );

        var result = await raft.ReplicateLogs(
```

```
            partitionId, "SetValue", payload,
            expectedGeneration: generation,
            cancellationToken: timeout.Token
        );

        if (result.Success)
            return Results.Ok(new { key, value, partitionId, generation });

        if (result.Status == RaftOperationStatus.PartitionMoved)
        {
            // The partition map changed between the GetPartitionKey call
            // and the ReplicateLogs call. Re-route.
            int newPartitionId = raft.GetPartitionKey(key);
            string? leader = raft.GetPartitionLeaderHint(newPartitionId);
            return Results.Json(
                new
                {
                    error = "PartitionMoved",
                    leader,
                    partitionId = newPartitionId,
                   newGeneration = raft.GetPartitionGeneration(newPartitionId)
                },
                statusCode: 503
            );
        }

        return Results.Json(
            new { error = result.Status.ToString(), partitionId },
            statusCode: 503
        );
    }
    catch (PartitionNotHostedException)
    {
        int newPartitionId = raft.GetPartitionKey(key);
        string? leader = raft.GetPartitionLeaderHint(newPartitionId);
        return Results.Json(
         new { error = "PartitionMoved", leader, partitionId = newPartitionId },
            statusCode: 503
        );
    }
});
```

The key addition: `raft.GetPartitionGeneration(partitionId)` reads the generation, and `expectedGeneration: generation` passes it to `ReplicateLogs`. If a split or merge

happened between the `GetPartitionKey` call and the `ReplicateLogs` call, the fence catches it.

14.4.2 Client-Side Fencing

When a client caches the partition map, it can include the generation in each request. The server validates the generation and rejects stale requests:

```
// Client sends: PUT /store/balance
// Headers: X-Partition-Id: 3, X-Expected-Generation: 7

app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    int partitionId;
    long expectedGeneration = 0;

    // Client-supplied routing (optional)
    if (request.Headers.TryGetValue("X-Partition-Id", out var pidHeader) &&
        int.TryParse(pidHeader, out int clientPartition))
    {
        partitionId = clientPartition;

          if (request.Headers.TryGetValue("X-Expected-Generation", out var
genHeader) &&
            long.TryParse(genHeader, out long clientGen))
        {
            expectedGeneration = clientGen;
        }
    }
    else
    {
        partitionId = raft.GetPartitionKey(key);
        expectedGeneration = raft.GetPartitionGeneration(partitionId);
    }

    // ... proceed with write using expectedGeneration
});
```

This pattern lets the client make routing decisions. The server validates them with the generation fence. If the client's map is stale, the server rejects the write and the client refreshes.

14.5 Per-Entry Fencing in Heterogeneous Batches

`ReplicateEntries` supports per-entry generation fencing through the `ExpectedGeneration` field on `RaftProposalEntry`:

```
var entries = new List<RaftProposalEntry>
{
```

```
    new(
        Type: "SetValue",
        Data: payload1,
        ExpectedGeneration: 7    // fenced
    ),
    new(
        Type: "SetValue",
        Data: payload2,
        ExpectedGeneration: 0    // not fenced
    ),
    new(
        Type: "SetValue",
        Data: payload3,
        ExpectedGeneration: 7    // fenced
    )
};

RaftBatchReplicationResult result = await raft.ReplicateEntries(
    partitionId, entries
);
```

Each entry is fenced independently. If the partition's generation is 8:

- Entry 0: fenced at 7, partition is 8. **Rejected** with `PartitionMoved`. Dropped from the batch.
- Entry 1: not fenced (0). **Accepted.**
- Entry 2: fenced at 7, partition is 8. **Rejected** with `PartitionMoved`. Dropped from the batch.

The rejected entries appear as `PartitionMoved` in the per-entry results. Their siblings proceed normally. The batch `Success` flag is `true` as long as at least one entry was accepted.

```
for (int i = 0; i < result.Entries.Count; i++)
{
    RaftEntryResult entry = result.Entries[i];

    if (entry.Status == RaftOperationStatus.PartitionMoved)
    {
        Console.WriteLine($"Entry {i}: fenced out (stale generation)");
    }
    else if (entry.Status == RaftOperationStatus.Success)
    {
        Console.WriteLine($"Entry {i}: committed at index {entry.LogIndex}");
    }
}
```

This is useful when a batch contains entries routed by different keys. Some keys may still belong to this partition (generation matches), while others may have moved to a new partition after a split.

14.6 The Refresh-on-Rejection Pattern

The standard pattern for handling `PartitionMoved`:

```
Client writes with expectedGeneration
     |
     ▼
Server: PartitionMoved (generation mismatch)
     |
     ▼
Client refreshes partition map
     |
     ▼
Client re-routes key using the new map
     |
     ▼
Client retries on the correct partition with new generation
```

14.6.1 Client Implementation

```
public async Task<bool> SetValueAsync(string key, string value)
{
    int maxRetries = 3;

    for (int attempt = 0; attempt < maxRetries; attempt++)
    {
        int partitionId = cachedMap.GetPartitionForKey(key);
        long generation = cachedMap.GetGeneration(partitionId);
        string endpoint = cachedMap.GetLeader(partitionId);

        var response = await SendWrite(
            endpoint, key, value, partitionId, generation
        );

        if (response.Success)
            return true;

        if (response.Error == "PartitionMoved")
        {
            // Refresh the cached map from any cluster node
            cachedMap = await FetchPartitionMap();
            continue;
        }
```

```
        if (response.Error == "NotLeader" && response.Leader is not null)
        {
            cachedMap.UpdateLeader(partitionId, response.Leader);
            continue;
        }

        await Task.Delay(100 * (attempt + 1));
    }

    return false;
}
```

The client maintains a cached copy of the partition map. On `PartitionMoved`, it fetches a fresh map from the cluster and retries. The retry uses the new partition id and generation.

14.7 Proactive Map Updates with OnPartitionMapChanged

Instead of waiting for a rejection, the server can proactively update its routing information when the partition map changes:

```
var routingCache = new ConcurrentDictionary<int, long>();

raft.OnPartitionMapChanged += (map) =>
{
    foreach (RaftPartitionRange range in map)
    {
        routingCache[range.PartitionId] = range.Generation;
    }
};
```

This reduces the window where a stale map can cause a `PartitionMoved` rejection. The server-side routing table updates as soon as the map change commits.

However, `OnPartitionMapChanged` does not eliminate the race entirely. A split can commit between the `GetPartitionKey` call and the `ReplicateLogs` call. The generation fence is the safety net for this unavoidable race.

14.8 When to Use Generation Fencing

Scenario	Fence needed?	Why
Static partition layout (no splits or merges)	No	The generation never changes
Partitions split or merge at runtime	Yes	Routing can become stale
Client caches partition map	Yes	The cache can be stale after a split
Server routes keys with `GetPartitionKey` per request	Optional	The race window is small but exists
Multiple services share a Kommander cluster	Yes	Each service has its own routing cache

14.8.1 The Trade-Off

Generation fencing adds one dictionary lookup per write (`GetPartitionGeneration`). This is sub-microsecond. The cost is negligible compared to the quorum round-trip.

The benefit is correctness. Without fencing, a split during a write can silently misplace data. With fencing, the same scenario produces a clear `PartitionMoved` rejection and a retry on the correct partition.

14.9 When Not to Pass expectedGeneration

Pass `expectedGeneration = 0` (the default) when:

- The partition layout is static and never changes at runtime.
- You are writing to an unrouted partition (one created with `RaftRoutingMode.Unrouted`) that is not part of hash-range routing.
- You are writing internal control entries that do not depend on the hash-range assignment.

In all other cases, pass a non-zero generation for safety.

14.10 The Race Window

Even with generation fencing, a small race window exists:

```
T=0   GetPartitionKey("key") → partition 3
T=1   GetPartitionGeneration(3) → 7
T=2   Split happens: partition 3 generation becomes 8
T=3   ReplicateLogs(3, ..., expectedGeneration: 7)
      → PartitionMoved (fence catches it)
```

The fence catches the race at T=3. Without the fence, the write at T=3 would succeed in the wrong partition.

The window between T=1 and T=3 is typically sub-millisecond (the time to serialize the payload and call `ReplicateLogs`). The chance of a split happening in that window is extremely small. But in a system that processes millions of writes per day, extremely small is not zero. The fence makes the outcome safe regardless of timing.

14.11 Summary

- Each partition has a generation counter that increments on every structural change (split, merge, create, remove, replica move).
- `expectedGeneration` on `ReplicateLogs` rejects a write if the partition's generation has changed. The rejection status is `PartitionMoved`.
- `GetPartitionGeneration` reads the current generation. The cost is one dictionary lookup.
- Without generation fencing, a stale client can write data to the wrong partition after a split. The data is silently misplaced.
- With generation fencing, the same scenario produces a clear `PartitionMoved` rejection. The client refreshes its map and retries.
- `ReplicateEntries` supports per-entry fencing. Each entry in a batch can carry its own `ExpectedGeneration`. Fenced-out entries are dropped while their siblings proceed.
- `OnPartitionMapChanged` lets the server proactively update its routing cache, but the generation fence is still needed for the unavoidable race between routing and replication.
- The next chapter covers WAL internals, backend selection, and durability configuration.

15 Dynamic Cluster Membership

Your cluster started with three nodes. Traffic grows and you need a fourth. Later, you decommission a node for maintenance. Both operations must happen at runtime. No downtime. No quorum violation.

This chapter covers how nodes join and leave a running cluster, how Kommander detects dead members, and how the committed roster differs from service discovery.

15.1 Discovery vs. Committed Membership

These are two separate concepts:

- **Discovery** tells a node how to find other nodes. It provides endpoint addresses. A discovery mechanism (DNS, a static seed list, a service registry) says "these endpoints exist."
- **Committed membership** determines who participates in consensus. The committed roster is stored on partition 0 (the system partition) as a Raft log entry. It says "these nodes vote."

Discovery is input. Membership is output. A node that appears in a DNS record is not a cluster member. It becomes a member only when the P0 leader commits an `AddMember` entry to the roster.

```
Discovery (DNS, seeds, registry):
  "node-a:8001, node-b:8002, node-c:8003, node-d:8004"

Committed roster (partition 0):
  node-a  Voter   (version 1)
  node-b  Voter   (version 1)
  node-c  Voter   (version 1)

node-d is discoverable but NOT a member.
It does not vote. It does not receive replicated entries.
```

Why does this distinction matter? If discovery alone controlled membership, adding a DNS record could change the quorum requirement instantly. Three nodes with quorum 2 would become four nodes with quorum 3. If the fourth node is not yet running, quorum is broken. The cluster stalls.

Committed membership avoids this. The roster changes only through the Raft log, one member at a time, in a controlled sequence.

15.2 ClusterMembership

The committed roster is represented by the `ClusterMembership` type:

```
public sealed class ClusterMembership
{
    public long MembershipVersion { get; set; }
    public List<ClusterMember> Members { get; set; } = [];
}
```

`MembershipVersion` is a monotonically increasing counter. It increments by 1 on every add, promote, or remove. Zero means no roster was committed yet (a pre-seed transient).

15.2.1 ClusterMember

```
public sealed class ClusterMember
{
    public string Endpoint { get; set; } = "";
    public int NodeId { get; set; }
    public ClusterMemberRole Role { get; set; }
    public long JoinedVersion { get; set; }
}
```

Property	Description
`Endpoint`	The `host:port` address of the node
`NodeId`	The numeric node id
`Role`	`Voter`, `Learner`, `Leaving`, or `NotMember`
`JoinedVersion`	The membership version when this node first joined as a Learner

15.2.2 ClusterMemberRole

Role	Votes	Campaigns	Description
`Voter`	Yes	Yes	Full voting member. Counts toward quorum.
`Learner`	No	No	Catching up. Receives entries but does not vote.
`Leaving`	No	No	Draining replicas before removal.
`NotMember`	No	No	Not in the committed roster.

Liveness state (Alive, Suspect, Dead) lives in the gossip layer, not in the roster. Gossip state never touches the Raft log. This separation prevents liveness churn from creating unnecessary log entries.

15.3 Querying Membership

15.3.1 GetMembership

```
ClusterMembership membership = raft.GetMembership();

Console.WriteLine($"Roster version: {membership.MembershipVersion}");

foreach (ClusterMember member in membership.Members)
{
    Console.WriteLine(
        $"  {member.Endpoint} role={member.Role} "
        + $"joined at version {member.JoinedVersion}"
    );
}
```

Returns a point-in-time snapshot. The returned object is immutable. Future roster changes do not affect it.

15.3.2 LocalRole

```
ClusterMemberRole role = raft.LocalRole;
```

Returns the local node's role in the committed roster. During the pre-seed transient (roster version 0), it returns `Voter` so existing behavior is preserved on new clusters.

`LocalRole` also returns `Leaving` locally (without a committed role change) once teardown has begun. This suppresses election campaigns immediately on shutdown.

15.3.3 OnMembershipChanged

```
raft.OnMembershipChanged += (membership) =>
{
    Console.WriteLine(
        $"Roster changed: version {membership.MembershipVersion}, "
        + $"{membership.Members.Count} members"
    );

    foreach (ClusterMember member in membership.Members)
    {
        Console.WriteLine(
            $"  {member.Endpoint} role={member.Role}"
        );
    }
};
```

Fires whenever the committed roster advances to a new version. This includes joins, promotions, departures, and SWIM-driven evictions.

The event fires on the system coordinator consumer loop. Handlers must not block or re-enter the coordinator.

15.4 Joining a Cluster

15.4.1 The Join Flow

```
1. New node calls JoinCluster(seeds)
     |
     ▼
2. Node contacts a seed endpoint
     |
     ▼
3. Seed forwards the join request to the P0 leader
     |
     ▼
4. P0 leader commits AddMember(endpoint, Learner)
     |
     ▼
5. New node enters as a Learner
   Receives entries but does not vote
     |
     ▼
6. Leader monitors learner's lag on all partitions
     |
     ▼
7. Learner within LearnerPromotionLag entries
   for LearnerPromotionStableWindow
```

```
      |
      ▼
  8. P0 leader commits PromoteMember(endpoint, Voter)
      |
      ▼
  9. Node is now a Voter. Counts toward quorum.
```

15.4.2 JoinCluster

Two overloads:

```
Task JoinCluster(CancellationToken cancellationToken = default);

Task JoinCluster(
    IEnumerable<string> seeds,
    CancellationToken cancellationToken = default
);
```

The first overload uses the configured discovery mechanism. The second overload contacts the given seed endpoints directly, and bypasses discovery. The second form is useful when discovery is unavailable or when you add a node programmatically.

15.4.3 Example: Adding a Fourth Node

```
// Node D configuration
RaftConfiguration configuration = new()
{
    NodeName = "node-d",
    Host = "localhost",
    Port = 8004,
    InitialPartitions = 8,
    ReplicationFactor = 3
};

IRaft raft = new RaftManager(configuration);

// Register callbacks before joining
raft.OnReplicationReceived += async (partitionId, log) =>
{
    // Apply entries as they arrive
    return true;
};

raft.OnLogRestored += async (partitionId, log) =>
{
    // Replay on restart
    return true;
};
```

```
// Join using a seed endpoint
await raft.JoinCluster(
    seeds: ["localhost:8001"],
    cancellationToken: default
);
```

The node contacts `localhost:8001` (an existing member). That member forwards the request to the P0 leader. The leader commits the learner entry.

15.4.4 Learner Promotion

The P0 leader monitors every learner's lag on all partitions. Two properties control when promotion happens:

```
LearnerPromotionLag = 10   // default (entries)
LearnerPromotionStableWindow = TimeSpan.FromSeconds(3)   // default
```

LearnerPromotionLag. The maximum number of committed entries a learner may trail the leader on any partition and still be considered caught up. The promotion driver checks this threshold on every tick.

LearnerPromotionStableWindow. How long the learner must remain within the lag threshold on all partitions before promotion. This prevents premature promotion of nodes that are briefly caught up but still unstable (for example, during a burst of writes).

When the learner is within `LearnerPromotionLag` entries on every partition for `LearnerPromotionStableWindow` continuously, the P0 leader commits a `PromoteMember` entry. The learner becomes a Voter.

15.4.5 Why Learners Matter

Adding a new node directly as a Voter is dangerous. The new node has no data. It immediately counts toward quorum. If another existing node fails at the same time, the cluster may lose quorum.

The learner stage prevents this. The new node receives and applies entries while it catches up. It does not affect quorum calculations. Only after it is fully caught up does it become a Voter.

```
Before node-d joins:
Voters: [A, B, C]    quorum = 2 of 3

Node-d joins as Learner:
Voters: [A, B, C]    quorum = 2 of 3   (unchanged)
Learners: [D]        (does not count toward quorum)

After promotion:
Voters: [A, B, C, D]  quorum = 3 of 4
```

Quorum changes only when the promotion commits, and only then the new node is caught up.

15.5 Leaving a Cluster

Kommander provides two ways to remove a node:

15.5.1 LeaveCluster

```
await raft.LeaveCluster(dispose: false, cancellationToken: default);
```

Commits a `RemoveMember(self)` entry on partition 0. Surviving nodes drop this node from their roster. The node stops participating immediately.

When `dispose` is `true`, the manager is disposed after stopping.

When the committed roster contains no other Voter (a standalone node), the membership round-trip is skipped. The node stops immediately.

15.5.2 RequestLeaveAsync

```
LeaveClusterResult result = await raft.RequestLeaveAsync(cancellationToken);

if (result.Left)
{
    Console.WriteLine("Node removed from roster");

    if (result.Drained)
        Console.WriteLine("All replicas evacuated");

    // Safe to stop the process
}
else
{
    Console.WriteLine($"Leave outcome: {result.Outcome}");
}
```

`RequestLeaveAsync` asks the cluster to remove this node, but does not stop it. The caller decides what to do based on the outcome.

15.5.3 LeaveClusterResult

```
public readonly record struct LeaveClusterResult(
    LeaveClusterOutcome Outcome,
    long MembershipVersion,
    bool Drained = false
)
{
    public bool Left => Outcome is
        LeaveClusterOutcome.Committed or
        LeaveClusterOutcome.NotAMember;
```

```
    public bool Terminal => Outcome is
        LeaveClusterOutcome.RefusedInsufficientVoters;
}
```

15.5.4 LeaveClusterOutcome

Outcome	Description
`Committed`	The removal committed. The node is out of the roster.
`NotAMember`	The node was already absent from the roster.
`RefusedInsufficientVoters`	Removing this node would leave zero voters. Permanent.
`NotInitialized`	The cluster has not finished initialization yet.
`NoLeader`	The P0 leader could not be reached. Retry.
`Timeout`	The deadline expired before the removal committed. Check the roster.
`RefusedDrainInProgress`	Another node is already draining. Only one drain at a time. Retry later.
`DrainTimedOut`	The drain did not complete in time. The role was rolled back to Voter.

15.5.5 The Difference

`LeaveCluster` is for shutdown. It removes the node and stops it. Use it when you are shutting down the process.

`RequestLeaveAsync` is for decommissioning. It removes the node and returns the outcome. The node keeps running until the caller decides to stop it. Use it when you want to verify the removal before stopping.

15.6 Decommission Drain

When per-partition replica placement is active (`ReplicationFactor > 0` and `EnablePlacementRebalancer = true`), `RequestLeaveAsync` runs a **drain** before removal:

```
1. RequestLeaveAsync() called
     |
     ▼
2. P0 leader commits ClusterMemberRole.Leaving
     |
     ▼
3. Node enters Leaving state
   - No longer counts toward roster-level quorums
```

```
    - No longer a placement candidate
    - Still receives entries (evacuating learners catch up from it)
    - Still serves reads for partitions it hosts
      |
      ▼
4. Placement controller evacuates replicas
    - For each partition the node hosts:
      Add learner on a survivor → catch up → promote → remove old replica
      |
      ▼
5. All replicas evacuated (Drained = true)
      |
      ▼
6. P0 leader commits RemoveMember
      |
      ▼
7. Node is out of the roster
```

15.6.1 DecommissionDrainTimeout

```
DecommissionDrainTimeout = TimeSpan.FromMinutes(2)    // default
```

The maximum time `RequestLeaveAsync` waits for the placement pass to evacuate all replicas. If the drain does not complete in time:

1. The `Leaving` role is rolled back to `Voter`.
2. The node resumes normal operation.
3. The outcome is `DrainTimedOut`.

Replicas already evacuated stay evacuated. A retry resumes the drain from where it left off.

Size this from the data. An evacuation costs about 3 placement passes per partition (add learner, observe catch-up, promote). With `MaxConcurrentReplicaRepairs = 3` and many partitions, the drain can take minutes.

15.6.2 Drain Constraints

Only one drain can run at a time. If another node is already draining, `RequestLeaveAsync` returns `RefusedDrainInProgress`. Two concurrent drains would shrink the voter set by two before either evacuation completes.

15.7 SWIM Failure Detector

Kommander uses SWIM (Scalable Weakly-consistent Infection-style process group Membership protocol) to detect dead nodes.

15.7.1 How SWIM Works

```
Every PingInterval (default 1s):

1. Prober picks one random peer
```

```
     │
     ▼
2. Sends a direct ping
     │
     ├── Response received → peer is Alive
     │
     └── PingTimeout (500ms) expires, no response
           │
           ▼
3. Indirect probing: pick IndirectPingFanout (2) random peers
   Ask them to ping the target on your behalf
     │
     ├── Any indirect response → peer is Alive
     │
     └── All indirect probes time out
           │
           ▼
4. Peer enters Suspect state
     │
     ▼
5. SuspicionTimeout (5s): peer can refute via gossip
     │
     ├── Refuted → back to Alive
     │
     └── Not refuted
           │
           ▼
6. Peer enters Dead state
     │
     ▼
7. DeadMemberEvictionGrace (2 minutes): wait before eviction
     │
     ├── Peer comes back → rejoin
     │
     └── Still dead
           │
           ▼
8. P0 leader commits RemoveMember
   OnMembershipChanged fires on all nodes
```

15.7.2 SWIM Configuration

Property	Default	Description
`PingInterval`	1 second	Time between SWIM probe rounds. Each round probes one random peer.
`PingTimeout`	500 ms	Time to wait for a direct or indirect ping response.
`IndirectPingFanout`	2	Number of intermediaries for indirect probing.
`SuspicionTimeout`	5 seconds	Time a node stays Suspect before promotion to Dead.
`DeadMemberEvictionGrace`	2 minutes	Time a Dead node stays before eviction from the roster.

15.7.3 PingInterval

```
PingInterval = TimeSpan.FromSeconds(1)   // default
```

The time between SWIM probe rounds. Each round picks one random peer and probes it. All built-in transports (InMemory, gRPC, REST) implement the SWIM ping protocol. Set to `TimeSpan.Zero` to disable the failure detector entirely (for tests that control membership explicitly).

15.7.4 PingTimeout

```
PingTimeout = TimeSpan.FromMilliseconds(500)   // default
```

How long to wait for a ping response before the probe counts as failed. Smaller values detect failures faster but produce more false positives on slow networks.

15.7.5 IndirectPingFanout

```
IndirectPingFanout = 2   // default
```

After a direct ping times out, the prober asks this many random peers to ping the target on its behalf. This reduces false positives caused by a single faulty network path. If any indirect probe succeeds, the target is Alive.

15.7.6 SuspicionTimeout

```
SuspicionTimeout = TimeSpan.FromSeconds(5)   // default
```

How long a node stays in the Suspect state before it advances to Dead. During this window, the suspected node can refute the suspicion through gossip (it sends a message that says "I am alive"). If no refutation arrives, the node is declared Dead.

15.7.7 DeadMemberEvictionGrace

```
DeadMemberEvictionGrace = TimeSpan.FromMinutes(2)   // default
```

How long a Dead node stays before the P0 leader commits a `RemoveMember` entry. This grace period absorbs transient issues that resolve before eviction. A node restarting (container restart, cold storage open) can take tens of seconds. The grace period must exceed that.

The original default was 30 seconds. That was shorter than a cold RocksDB start and evicted nodes during routine restarts. The current default of 2 minutes provides a comfortable margin.

Eviction is a one-way door. A node that misses the window must run the full join flow again (unless auto-rejoin is enabled).

15.8 Gossip Anti-Entropy

Gossip is the background mechanism that converges membership across all nodes. Each round, a node contacts random peers and exchanges membership versions. If a peer holds a newer committed roster version, the node pulls it.

15.8.1 Gossip Configuration

```
GossipInterval = TimeSpan.FromSeconds(5)   // default
GossipFanout = 2   // default
```

GossipInterval. Time between gossip rounds. Default: 5 seconds.

GossipFanout. Number of random peers contacted per round. Higher values converge stale caches faster at the cost of more network traffic. Setting to 0 disables gossip entirely. Default: 2.

Gossip is not the path that commits membership changes. Raft commits membership changes through partition 0. Gossip is the convergence mechanism that ensures every node sees the latest committed roster, even if it missed the replication broadcast (for example, during a network partition that healed).

15.9 Auto-Rejoin

```
EnableAutoRejoin = true   // default
```

When enabled, a node that discovers it was removed from the roster (typically through dead-member eviction during a slow restart) automatically re-runs the join flow against the remaining members.

Without auto-rejoin, a node evicted during a restart would park as `NotMember` forever. It would suppress pre-votes on every partition, return terminal errors to clients, and never recover.

Auto-rejoin is suppressed during a graceful `LeaveCluster` (the node intended to leave) and before the node has finished its first join (first-time joins own their admission loop).

Disable auto-rejoin only if an operator workflow removes live nodes remotely and expects them to stay out while still running.

15.10 Failure During Membership Transition

15.10.1 Learner Crash During Promotion

```
T=0   Learner D is caught up
T=1   P0 leader commits PromoteMember(D, Voter)
T=2   Node D crashes before receiving the promotion entry
T=3   Node D restarts
T=4   Node D catches up and sees the committed promotion
T=5   Node D is a Voter
```

No inconsistency occurs. The committed log is the source of truth. Node D did not need to receive the promotion entry in real time. It received it during catch-up after restart.

15.10.2 Leader Crash During Drain

```
T=0   Node C calls RequestLeaveAsync
T=1   P0 leader commits Leaving(C)
T=2   Placement starts evacuating C's replicas
T=3   P0 leader crashes
T=4   New P0 leader is elected
T=5   New leader sees C in Leaving state
T=6   New leader continues the evacuation
T=7   Evacuation completes, removal commits
```

The Leaving state is in the committed log. A new leader picks up the drain from where the old leader stopped.

15.10.3 Node Evicted While Restarting

```
T=0    Node B crashes
T=1    SWIM declares B as Suspect
T=6    SWIM promotes B to Dead
T=126  DeadMemberEvictionGrace expires (2 minutes)
T=127  P0 leader commits RemoveMember(B)
T=130  Node B finishes restarting
T=131  Node B discovers it is not in the roster
T=132  EnableAutoRejoin → Node B runs the join flow
T=133  Node B re-enters as a Learner
T=140  Node B is promoted to Voter
```

Without auto-rejoin, Node B would stay as `NotMember` at step 131.

15.11 Adding a Node: Complete Example

```
// Existing three-node cluster: node-a (8001), node-b (8002), node-c (8003)

// New node configuration
RaftConfiguration configuration = new()
{
    NodeName = "node-d",
    Host = "localhost",
```

```
    Port = 8004,
    InitialPartitions = 8,
    ReplicationFactor = 3,
    LearnerPromotionLag = 10,
    LearnerPromotionStableWindow = TimeSpan.FromSeconds(3)
};

IRaft raft = new RaftManager(configuration);

// Set up state machine callbacks
var stores = new ConcurrentDictionary<int, ConcurrentDictionary<string,
string>>();

raft.OnRestoreStarted += (partitionId) =>
{
    stores[partitionId] = new ConcurrentDictionary<string, string>();
};

raft.OnLogRestored += async (partitionId, log) =>
{
    if (log.Type == "SetValue")
    {
        var cmd = JsonSerializer.Deserialize<SetValueCommand>(log.LogData);
        if (cmd is not null)
            stores[partitionId][cmd.Key] = cmd.Value;
    }
    return true;
};

raft.OnReplicationReceived += async (partitionId, log) =>
{
    if (log.Type == "SetValue")
    {
        var cmd = JsonSerializer.Deserialize<SetValueCommand>(log.LogData);
        if (cmd is not null)
            stores[partitionId][cmd.Key] = cmd.Value;
    }
    return true;
};

// Monitor membership changes
raft.OnMembershipChanged += (membership) =>
{
    ClusterMember? self = membership.Members.FirstOrDefault(
```

```
        m => m.Endpoint == "localhost:8004"
    );

    if (self is not null)
    {
        Console.WriteLine($"My role: {self.Role}");

        if (self.Role == ClusterMemberRole.Voter)
            Console.WriteLine("Promoted to Voter!");
    }
};

// Join the cluster
await raft.JoinCluster(
    seeds: ["localhost:8001"],
    cancellationToken: default
);

// The node is now a Learner, catching up.
// After LearnerPromotionStableWindow, it becomes a Voter.
```

15.12 Removing a Node: Complete Example

```
// On the node to decommission:

using var timeout = new CancellationTokenSource(TimeSpan.FromMinutes(5));

LeaveClusterResult result = await raft.RequestLeaveAsync(
    timeout.Token
);

switch (result.Outcome)
{
    case LeaveClusterOutcome.Committed:
        Console.WriteLine("Removed from roster");
        if (result.Drained)
            Console.WriteLine("All replicas evacuated safely");
        // Safe to stop the process
        break;

    case LeaveClusterOutcome.NotAMember:
        Console.WriteLine("Already not a member");
        break;

    case LeaveClusterOutcome.RefusedInsufficientVoters:
        Console.WriteLine("Cannot remove: last voter");
```

```
        // Permanent. Do not retry.
        break;

    case LeaveClusterOutcome.RefusedDrainInProgress:
        Console.WriteLine("Another node is draining. Retry later.");
        break;

    case LeaveClusterOutcome.DrainTimedOut:
        Console.WriteLine("Drain timed out. Role rolled back to Voter.");
        Console.WriteLine("Retry to resume the drain.");
        break;

    case LeaveClusterOutcome.Timeout:
        Console.WriteLine("Timeout. Check the roster before deciding.");
        break;

    case LeaveClusterOutcome.NoLeader:
        Console.WriteLine("No P0 leader available. Retry.");
        break;
}
```

15.13 Configuration Summary

Property	Default	Description
`LearnerPromotionLag`	10	Max lag (entries) for a learner to be considered caught up.
`LearnerPromotionStableWindow`	3 seconds	How long a learner must stay within the lag threshold.
`PingInterval`	1 second	Time between SWIM probe rounds.
`PingTimeout`	500 ms	Ping response timeout.
`IndirectPingFanout`	2	Intermediary nodes for indirect probing.
`SuspicionTimeout`	5 seconds	Time in Suspect before promotion to Dead.
`DeadMemberEvictionGrace`	2 minutes	Time in Dead before eviction from the roster.
`EnableAutoRejoin`	`true`	Auto-rejoin after eviction during a slow restart.
`GossipInterval`	5 seconds	Time between gossip rounds.
`GossipFanout`	2	Peers contacted per gossip round.
`DecommissionDrainTimeout`	2 minutes	Max wait for replica evacuation during decommission.

15.14 Summary

- Discovery tells nodes where to find each other. The committed roster (on partition 0) determines who votes. They are separate mechanisms.
- `ClusterMembership` holds the roster with a monotonically increasing `MembershipVersion`. Each change increments it by 1.
- New nodes join as `Learner` through `JoinCluster`. They receive entries but do not vote. After catching up within `LearnerPromotionLag` for `LearnerPromotionStableWindow`, the P0 leader promotes them to `Voter`.
- `RequestLeaveAsync` is the recommended way to decommission a node. Under replica placement, it drains replicas before removal.
- `LeaveCluster` is for shutdown. It removes the node and stops it.

- SWIM detects dead nodes through direct and indirect pinging. Dead nodes are evicted after `DeadMemberEvictionGrace` (default: 2 minutes).
- `EnableAutoRejoin` (default: `true`) lets evicted nodes automatically re-join.
- Gossip converges stale roster caches in the background. It is not the path that commits membership changes.
- Only one drain can run at a time. Two concurrent drains would reduce the voter set unsafely.
- The next chapter covers leadership balancing: how to distribute write load evenly across nodes.

16 Leadership Balancing

Elections are random. When a cluster starts or recovers from a failure, the node that wins each partition's election becomes its leader. Over time, one node can end up leading most partitions while the others sit idle. All write traffic flows through the leader, so an uneven leadership distribution creates an uneven write load.

The leader balancer watches how leaderships are distributed and transfers them to equalize the load.

16.1 The Problem: Uneven Leadership

A three-node cluster with 12 partitions after random elections:

```
Node A: leads P1, P2, P3, P4, P5, P6, P7    (7 partitions)
Node B: leads P8, P9                          (2 partitions)
Node C: leads P10, P11, P12                   (3 partitions)

Node A handles 58% of all write traffic.
Nodes B and C are underutilized.
```

The ideal distribution is 4 partitions per node. Node A leads 3 more than it should. The balancer detects this and transfers 3 leaderships from A to B and C.

After balancing:

```
Node A: leads P1, P2, P3, P4                  (4 partitions)
Node B: leads P5, P8, P9, P10                 (4 partitions)
Node C: leads P6, P7, P11, P12                (4 partitions)

Each node handles ~33% of write traffic.
```

16.2 Enabling the Leader Balancer

The balancer is off by default. Enable it in the configuration:

```
RaftConfiguration configuration = new()
{
    NodeName = "node-a",
    Host = "localhost",
    Port = 8001,
```

```
    InitialPartitions = 12,
    EnableLeaderBalancer = true,
    LeaderBalancerInterval = TimeSpan.FromSeconds(30)
};
```

`EnableLeaderBalancer` is the master switch. When `false`, the balancer loop never runs and no transfer suggestions are sent.

Enable it only after all nodes in the cluster run a version that supports the feature. The flag is revertible at runtime through a rolling restart.

16.3 How Load Reports Work

Each node periodically builds a **NodeLoadReport** and sends it through gossip. The report contains:

- The node's endpoint.
- A monotonically increasing version counter.
- A list of partitions the node leads, with per-partition load metrics.
- The node's zone (if configured).

```
public sealed class NodeLoadReport
{
    public string Endpoint { get; set; } = "";
    public long ReportVersion { get; set; }
    public HLCTimestamp Time { get; set; }
    public string? Zone { get; set; }
    public List<PartitionLoad> Leaderships { get; set; } = [];
}
```

Load reports are never committed to the Raft log. They are high-churn and purely advisory. A stale or dropped report only delays a balancing decision. It never violates Raft safety.

The receiver keeps only the highest `ReportVersion` per endpoint. Out-of-order arrivals are discarded.

16.3.1 PartitionLoad

Each entry in the report carries per-partition metrics:

```
public sealed class PartitionLoad
{
    public int PartitionId { get; set; }
    public double Load { get; set; }
    public long LeaderSinceMs { get; set; }
    public double LogOpsPerSecond { get; set; }
    public int WalQueueDepth { get; set; }
    public double CommitWaitMs { get; set; }
}
```

Field	Description
`Load`	Composite load scalar (computed from ops/sec and queue depth)
`LeaderSinceMs`	Milliseconds since this node became leader of the partition
`LogOpsPerSecond`	EWMA rate of `ReplicateLogs` operations per second
`WalQueueDepth`	WAL write operations queued for this partition
`CommitWaitMs`	EWMA of enqueue-to-durable commit-wait latency

16.3.2 The Load Formula

The composite load score per partition is:

```
Load = LeaderBalancerOpsWeight x OpsPerSecond
     + LeaderBalancerQueueWeight x (clientDepth + walDepth)
```

The default weights are:

```
LeaderBalancerOpsWeight = 1.0    // default
LeaderBalancerQueueWeight = 0.5  // default
```

Higher `OpsWeight` makes the balancer more sensitive to throughput differences. Higher `QueueWeight` makes it more sensitive to backlog buildup.

16.4 The GlobalLeadershipView

The P0 leader aggregates all load reports into a `GlobalLeadershipView`. This is the planner's input. It contains:

Field	Description
`LeadersByNode`	Maps each endpoint to the partition ids it leads
`LoadByNode`	Maps each endpoint to its aggregate load (sum of partition loads)
`PartitionOwner`	Maps each partition id to the endpoint that leads it
`PartitionLoad`	Maps each partition id to its load score
`PartitionLeaderSinceMs`	Maps each partition id to its leadership tenure
`LiveVoters`	The set of Voter-role members that are alive
`FreshReportEndpoints`	Endpoints with non-expired reports

16.4.1 Completeness Check

Before planning any moves, the planner calls `IsComplete()`. This check passes only when every live voter has a fresh report. If any live voter is silent (no report within the TTL), the planner emits zero moves.

This is a safety valve. An incomplete view means the planner cannot see the full leadership distribution. It could make a bad decision. Instead, it waits for the missing reports.

16.5 The Two-Tier Planning Algorithm

The `LeaderBalancePlanner` uses two tiers. They run in order. The second tier runs only when the first tier produces no moves.

16.5.1 Tier 1: Count Balancing

Equalize the number of leaderships per node.

```
Ideal distribution:  total_partitions / live_nodes
  12 partitions / 3 nodes = 4 per node

Over-loaded:  count > ceil + CountDeadband - 1
Under-loaded: count < floor
```

The count tier picks the hottest eligible partition from the most over-loaded node and moves it to the most under-loaded node. It repeats until no over-loaded or under-loaded nodes remain, or until `MaxMovesPerPass` is reached.

"Hottest" means the partition with the highest load score. The balancer moves the busiest partition first, because that move has the largest effect on load equalization.

```
Before count balancing:
```

```
Node A: [P1=100, P2=50, P3=30, P4=20, P5=10, P6=5, P7=2]  count=7
Node B: [P8=40, P9=15]                                      count=2
Node C: [P10=25, P11=10, P12=5]                             count=3

Ideal = 4.  CountDeadband = 1.
Over threshold = ceil(4) + 1 - 1 = 4.  Under threshold = floor(4) = 4.

Node A (7) is over 4 → over-loaded
Node B (2) is under 4 → under-loaded

Move 1: P1 (load=100) from A to B     → A=6, B=3
Move 2: P2 (load=50)  from A to B     → A=5, B=4
Move 3: P3 (load=30)  from A to C     → A=4, C=4

After:
Node A: [P4, P5, P6, P7]   count=4
Node B: [P1, P2, P8, P9]   count=4
Node C: [P3, P10, P11, P12] count=4
```

16.5.2 Tier 2: Load Balancing

When counts are already balanced but aggregate load is skewed, the load tier fires.

Load skew is measured as:

```
skew = (maxLoad - minLoad) / maxLoad
```

If the skew exceeds `LoadImbalanceThreshold` (default: 25%), the load tier emits a **count-neutral swap**: one partition moves from the hot node to the cold node, and one moves in the opposite direction. The count stays the same on both nodes.

```
Counts are balanced (4 each), but:

Node A: total load = 200  (P1=100, P4=50, P7=30, P10=20)
Node B: total load = 70   (P2=25, P5=20, P8=15, P11=10)

Skew = (200 - 70) / 200 = 0.65 → exceeds 0.25 threshold

Swap: P1 (load=100) from A to B
      P11 (load=10) from B to A

After swap:
Node A: total load = 110   (P11=10, P4=50, P7=30, P10=20)
Node B: total load = 160   (P1=100, P2=25, P5=20, P8=15)
```

The swap only fires if it strictly reduces the load spread. If the swap would not improve the distribution, it is skipped.

At most one swap per pass. The balancer re-evaluates on the next interval after the transferred partitions have settled.

16.6 Per-Move Safety Filters

Every candidate move must pass all of these filters:

1. **Partition is Active.** Partitions in `Splitting`, `Draining`, or `Removed` state are excluded.
2. **Leadership is stable.** The leader must have held the partition for at least `MinLeaderStabilityMs` (default: 5 seconds). This prevents the balancer from immediately shuffling a partition that just won an election.
3. **Target is a live voter.** The target endpoint must be a Voter in the committed roster and must be alive according to SWIM.
4. **Partition is not in cooldown.** After a transfer, the partition is excluded from further moves for `MoveCooldown` (default: 60 seconds).

16.7 The Transfer Mechanism

A `LeaderMove` is an advisory suggestion. The P0 leader sends the suggestion to the current leader of the partition. The current leader validates the suggestion and executes it through Raft's **leadership transfer** mechanism.

```
P0 leader (balancer):
  "Move partition 5 from node-a to node-b"
     |
     ▼
Suggestion sent to node-a
     |
     ▼
node-a validates:
  - Am I still the leader of partition 5?
  - Is node-b alive and caught up?
     |
     ├── No → drop the suggestion (safe)
     |
     └── Yes → TransferLeadershipAsync(partitionId=5, to=node-b)
           |
           ▼
Raft leadership transfer:
  - node-a stops accepting new proposals
  - node-a sends remaining log entries to node-b
  - node-a sends a TimeoutNow to node-b
  - node-b starts an election and wins
  - node-b is now the leader of partition 5
```

Dropping or ignoring a `LeaderMove` never violates Raft safety. The worst outcome is a wasted or skipped move. The balancer tries again on the next pass.

16.8 Preventing Oscillation

Without safeguards, the balancer can oscillate: move a partition from A to B, then on the next pass, move it back from B to A.

Three mechanisms prevent this:

16.8.1 MoveCooldown

```
MoveCooldown = TimeSpan.FromSeconds(60)    // default
```

After a transfer suggestion is confirmed or times out, the partition is excluded from further moves for 60 seconds. This gives the new leader time to establish a stable load profile.

16.8.2 MinLeaderStabilityMs

```
MinLeaderStabilityMs = 5000    // default (5 seconds)
```

A partition must have been on the same leader for at least 5 seconds before the balancer considers moving it. A freshly elected leader's load profile is not representative.

16.8.3 CountDeadband

```
CountDeadband = 1    // default
```

The minimum count imbalance before the count tier emits moves. With a deadband of 1, a node must exceed the natural ceiling by 2 or more before the balancer considers it over-loaded. This prevents flip-flopping around a perfectly balanced distribution.

16.9 Load Metrics

Kommander exposes three per-partition load metrics. These metrics work for both locally led partitions and remote ones:

16.9.1 GetPartitionLogOpsPerSecond

```
double opsPerSec = raft.GetPartitionLogOpsPerSecond(partitionId);
```

Returns the EWMA rate of `ReplicateLogs` operations per second.

- **Local path (this node leads the partition):** Reads directly from the in-process accumulator. No gossip lag.
- **Remote path (another node leads the partition):** Reads from the most recent gossip load report. May trail by up to one `LeaderBalancerReportInterval` plus propagation delay.

16.9.2 GetPartitionWalQueueDepth

```
int queueDepth = raft.GetPartitionWalQueueDepth(partitionId);
```

Returns the number of WAL write operations queued for this partition. When `LogOpsPerSecond` plateaus while this value rises, the partition is fsync-bound.

16.9.3 GetPartitionCommitWaitMs

```
double commitWaitMs = raft.GetPartitionCommitWaitMs(partitionId);
```

Returns the EWMA of enqueue-to-durable commit-wait latency in milliseconds. This complements queue depth: depth counts operations; commit-wait measures how long each operation waits. Under fsync pressure, both rise together while ops/sec plateaus.

16.10 Timing Configuration

Property	Default	Description
`LeaderBalancerReportInterval`	5 seconds	How often each node emits a load report through gossip.
`LeaderBalancerInterval`	30 seconds	How often the P0 leader runs a planning pass.
`LeaderBalancerReportTtl`	20 seconds	Maximum age of a report before it is expired from the view.
`SuggestionTimeout`	15 seconds	How long the P0 leader waits for a move to be confirmed.

LeaderBalancerReportInterval. Shorter intervals give fresher data at the cost of more gossip traffic.

LeaderBalancerInterval. Shorter intervals react faster to leadership skew but add coordinator overhead.

LeaderBalancerReportTtl. Must be greater than `LeaderBalancerReportInterval` to tolerate at least one missed emission. A node that goes silent is excluded from the planning view. Its phantom leaderships are not carried forward.

SuggestionTimeout. After timeout, the partition is eligible again (subject to `MoveCooldown`). The move may or may not have completed.

16.11 Concurrency Limits

```
MaxMovesPerPass = 4           // default
MaxConcurrentTransfers = 2    // default
```

MaxMovesPerPass. The maximum number of `LeaderMove` suggestions emitted in a single planner pass. Limits the blast radius of a bad planning decision.

MaxConcurrentTransfers. The maximum number of in-flight transfer suggestions the P0 leader tracks simultaneously. Once this limit is reached, no new moves are dispatched until outstanding ones confirm or time out.

16.12 Configuration Summary

Property	Default	Description
`EnableLeaderBalancer`	`false`	Master switch for the balancer.
`LeaderBalancerReportInterval`	5 seconds	Load report emission interval.
`LeaderBalancerInterval`	30 seconds	Planning pass interval.
`LeaderBalancerReportTtl`	20 seconds	Report expiry.
`CountDeadband`	1	Minimum count imbalance before count-tier moves.
`LoadImbalanceThreshold`	0.25	Fractional load skew before load-tier swaps.
`MinLeaderStabilityMs`	5,000 ms	Minimum leadership tenure before eligibility.
`MoveCooldown`	60 seconds	Post-move cooldown per partition.
`MaxMovesPerPass`	4	Moves per planning pass.
`MaxConcurrentTransfers`	2	In-flight transfer limit.
`LeaderBalancerOpsWeight`	1.0	Ops/sec weight in the load formula.
`LeaderBalancerQueueWeight`	0.5	Queue depth weight in the load formula.
`SuggestionTimeout`	15 seconds	In-flight suggestion expiry.

16.13 Complete Example

```
RaftConfiguration configuration = new()
{
    NodeName = "node-a",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 16,
    ReplicationFactor = 3,
    EnableLeaderBalancer = true,
    LeaderBalancerInterval = TimeSpan.FromSeconds(30),
    CountDeadband = 1,
    LoadImbalanceThreshold = 0.25,
    MaxMovesPerPass = 4,
    MaxConcurrentTransfers = 2,
    MoveCooldown = TimeSpan.FromSeconds(60),
```

```
    MinLeaderStabilityMs = 5000,
    LeaderBalancerOpsWeight = 1.0,
    LeaderBalancerQueueWeight = 0.5
};
```

16.13.1 Monitoring Load

```
IReadOnlyList<RaftPartitionRange> map = raft.GetPartitionMap();

foreach (RaftPartitionRange range in map)
{
    int pid = range.PartitionId;
    double ops = raft.GetPartitionLogOpsPerSecond(pid);
    int depth = raft.GetPartitionWalQueueDepth(pid);
    double waitMs = raft.GetPartitionCommitWaitMs(pid);

    Console.WriteLine(
        $"P{pid}: ops/s={ops:F1}  queue={depth}  "
        + $"commitWait={waitMs:F1}ms"
    );
}
```

16.13.2 Detecting Hotspots

```
double avgOps = 0;
int count = 0;

foreach (RaftPartitionRange range in map)
{
    avgOps += raft.GetPartitionLogOpsPerSecond(range.PartitionId);
    count++;
}

avgOps /= count;

foreach (RaftPartitionRange range in map)
{
    double ops = raft.GetPartitionLogOpsPerSecond(range.PartitionId);
    if (ops > avgOps * 3)
    {
        Console.WriteLine(
            $"Hotspot: P{range.PartitionId} at {ops:F1} ops/s "
            + $"(avg={avgOps:F1})"
        );
    }
}
```

A partition with 3 times the average ops/sec is a hotspot candidate. The leader balancer handles count-level redistribution automatically. For extreme hotspots, consider splitting the partition (Chapter 14).

16.14 Summary

- The leader balancer is off by default. Enable it with `EnableLeaderBalancer = true`.
- Each node emits a `NodeLoadReport` through gossip. The P0 leader aggregates these into a `GlobalLeadershipView`.
- The planner uses a two-tier strategy: count tier (equalize leadership count) then load tier (equalize aggregate load through count-neutral swaps).
- The planner requires a complete view. If any live voter has no fresh report, zero moves are emitted.
- Moves use Raft's leadership transfer mechanism. A dropped or ignored suggestion never violates safety.
- `MoveCooldown` (default: 60 seconds) prevents oscillation. `MinLeaderStabilityMs` (default: 5 seconds) prevents premature moves.
- Load is computed as `OpsWeight x OpsPerSecond + QueueWeight x QueueDepth`. Tune the weights for your workload.
- `GetPartitionLogOpsPerSecond`, `GetPartitionWalQueueDepth`, and `GetPartitionCommitWaitMs` expose per-partition load metrics for monitoring.
- The next chapter covers transport security and node authentication.

17 Observability and Diagnostics

Your cluster is running. How do you know it is healthy? Is a follower lagging? Is the WAL under pressure? Is the leader balancer oscillating? You need observability to answer these questions.

Kommander exposes three layers of observability:

1. **Diagnostic APIs.** Methods on the `IRaft` interface that return per-partition and per-node health data.
2. **OpenTelemetry metrics.** Counters, histograms, and gauges emitted through `System.Diagnostics.Metrics`.
3. **Structured logging.** High-performance log messages generated with `[LoggerMessage]` source generators.

This chapter covers all three layers and shows how to build a health endpoint that combines them.

17.1 The Three Primary Health Signals

Three per-partition methods form the core of runtime monitoring:

Method	Returns	What it measures
`GetPartitionLogOpsPerSecond(int)`	`double`	Throughput: the EWMA rate of replicated operations per second.
`GetPartitionWalQueueDepth(int)`	`int`	Saturation: the number of WAL write operations queued for the partition.
`GetPartitionCommitWaitMs(int)`	`double`	Latency: the EWMA time from enqueue to durable commit, in milliseconds.

Each method follows a two-tier resolution path:

1. **Local fast path.** If the partition is hosted on this node and this node is the leader, the value comes from the in-process executor and WAL scheduler. No network call.
2. **Gossip path.** If this node is not the leader, the value comes from the most recent `NodeLoadReport` received through gossip. The leader broadcasts its load report periodically.

All three return `0` when the partition is not hosted or no data is available.

```
Healthy partition:

OpsPerSecond = 1200.5     (throughput is steady)
WalQueueDepth = 0         (WAL is keeping up)
CommitWaitMs = 1.8        (fsync latency is low)

Partition under pressure:

OpsPerSecond = 3500.0     (high throughput)
WalQueueDepth = 42        (writes are queuing)
CommitWaitMs = 18.7       (fsync latency is rising)
```

17.1.1 EWMA Accumulators

The throughput and latency values use exponentially weighted moving averages (EWMA). These smooth out short spikes and reflect recent behavior.

PartitionLoadAccumulator tracks ops/sec with continuous-time exponential decay. The half-life is approximately 6.9 seconds. A burst of operations raises the rate quickly. When the burst stops, the rate decays toward zero over several half-lives.

PartitionWaitAccumulator tracks commit-wait latency with geometric per-sample decay (alpha = 0.3). Each new sample blends into the running average. The accumulator converges in 4 to 5 samples.

17.2 Commit Index

```
long commitIndex = raft.GetCommitIndex(partitionId);
```

Returns the highest contiguously committed log index for the partition. This is the gap-aware frontier: all entries up to and including this index are committed and applied. Returns `-1` when the partition is not hosted or nothing is committed.

17.2.1 Replication Lag

Compare the leader's commit index to a follower's commit index to measure replication lag:

```
Leader:    GetCommitIndex(5) = 48230
Follower:  GetCommitIndex(5) = 48215
Lag:       15 entries

Leader:    GetCommitIndex(5) = 48230
Follower:  GetCommitIndex(5) = 48230
Lag:       0 (caught up)
```

A small lag (a few entries) is normal during active writes. A growing lag means the follower cannot keep up. Possible causes:

- Slow storage on the follower (high fsync latency).
- Network congestion between the leader and the follower.
- The follower is processing a snapshot install.

17.3 Snapshot and Backfill Status

17.3.1 GetSnapshotStatuses

```
IReadOnlyList<RaftSnapshotStatus> statuses
= raft.GetSnapshotStatuses(partitionId);
```

Returns the leader-side snapshot transfer status for each follower that needs a snapshot. The list is empty when no snapshot transfers are in progress. It is also empty when this node is not the leader.

Each `RaftSnapshotStatus` contains:

Property	Type	Description
`FollowerEndpoint`	`string`	The follower receiving the snapshot.
`FailedAttempts`	`int`	Number of failed transfer attempts.
`LastError`	`string?`	The most recent error message.
`Unproducible`	`bool`	`true` if the application cannot produce a snapshot.
`InFlight`	`bool`	`true` if a transfer is in progress now.
`InFlightFor`	`TimeSpan?`	How long the current transfer has been running.
`FirstFailureAt`	`DateTimeOffset?`	When the first failure occurred.
`LastFailureAt`	`DateTimeOffset?`	When the most recent failure occurred.
`RetryBackoffRemaining`	`TimeSpan`	Time until the next retry attempt.

Watch for:

- `FailedAttempts > 0`: The transfer failed and is retrying with exponential backoff.
- `Unproducible = true`: The application's state transfer callback returned no data. Check your `IRaftPartitionStateTransfer` implementation.
- `InFlightFor` growing beyond a few seconds: The transfer is slow. Large snapshots on constrained networks cause this.

17.3.2 GetBackfillStatuses

```
IReadOnlyList<RaftBackfillStatus> statuses
= raft.GetBackfillStatuses(partitionId);
```

Returns the leader-side backfill refusal status for each follower that requested entries the leader no longer holds. The list is empty when no refusals occurred.

Each `RaftBackfillStatus` contains:

Property	Type	Description
`FollowerEndpoint`	`string`	The follower that requested entries.
`AnchorIndex`	`long`	The log index the follower asked for.
`FirstAvailableIndex`	`long`	The oldest index the leader still holds.
`LastCheckpoint`	`long`	The leader's last committed checkpoint.
`Occurrences`	`int`	How many times this follower was refused.
`FirstRefusedAt`	`DateTimeOffset`	When the first refusal occurred.
`LastRefusedAt`	`DateTimeOffset`	When the most recent refusal occurred.

A backfill refusal means the follower fell so far behind that the leader compacted away the entries it needs. The follower must receive a full snapshot instead of incremental backfill. A persistent pattern of refusals suggests one of these problems:

- Compaction is too aggressive for the cluster's catch-up speed.
- The follower was offline for a long time.
- Network bandwidth between the leader and follower is not sufficient.

17.4 Stale Proposed Count

```
long count = raft.GetStaleProposedSkippedCount(partitionId);
```

Returns the number of duplicate `Proposed` entries that the WAL handler refused since the last partition start. A stale proposal occurs when a WAL completion arrives for the wrong partition, term, or operation ID.

- Returns `-1` when the partition is not hosted.
- Returns `0` when hosted but no duplicates occurred.

A non-zero count is not an error. It can happen during leader transitions when proposals from the old term arrive after the new term started. A steadily growing count at high rate may indicate a configuration problem or a network partition that causes repeated retries.

17.5 Node Activity

17.5.1 GetActiveNodes

```
IReadOnlyList<string>                activeNodes
= raft.GetActiveNodes(TimeSpan.FromSeconds(30));
```

Returns all non-local node endpoints that had activity within the specified window. The list is sorted alphabetically. Use this to verify cluster health: the list should contain all peer endpoints.

17.5.2 GetLastNodeActivity

```
HLCTimestamp lastSeen = raft.GetLastNodeActivity("https://node-b:8001");
```

Returns the most recent activity timestamp for the specified endpoint across all partitions. Returns `HLCTimestamp.Zero` if the node was never seen.

17.5.3 NodeActivityTracker

The `NodeActivityTracker` is the internal component that records all node activity. It uses a two-level `ConcurrentDictionary`: endpoint to partition to timestamp.

Every Raft RPC (vote request, append entries, heartbeat) updates the tracker. The tracker stores both general activity timestamps and heartbeat timestamps separately. This lets you distinguish between "the node sent us data" and "the node sent us a heartbeat."

```
NodeActivityTracker internal state:

lastActivity:
  "https://node-b:8001" → { partition 0: T+5.2, partition 3: T+4.8 }
  "https://node-c:8001" → { partition 0: T+5.0, partition 1: T+3.1 }

lastHeartbeat:
  "https://node-b:8001" → { partition 0: T+5.2, partition 3: T+4.7 }
  "https://node-c:8001" → { partition 0: T+4.9, partition 1: T+3.0 }
```

17.6 Cluster Membership and Partition Map

Two methods provide the cluster topology:

```
ClusterMembership membership = raft.GetMembership();
IReadOnlyList<RaftPartitionRange> partitionMap = raft.GetPartitionMap();
```

`GetMembership()` returns a point-in-time snapshot of the committed cluster membership. Each `ClusterMember` contains:

- `Endpoint`: The node's address.
- `NodeId`: The node's numeric identifier.
- `Role`: One of `Voter`, `Learner`, `Leaving`, or `NotMember`.
- `JoinedVersion`: The membership version at which the node joined.

`GetPartitionMap()` returns a copy of the current partition map. Each `RaftPartitionRange` contains the partition ID, hash range, generation, state, routing mode, replicas, and replication factor.

17.7 KommanderMetrics: OpenTelemetry Integration

The `KommanderMetrics` class registers a `System.Diagnostics.Metrics.Meter` named `"Kommander"`. All instruments are static and shared across the process. When no metric listener is attached, the overhead is zero.

17.7.1 Counters

Instrument	Unit	Description
`raft.executor.operations_total`	ops	Total operations processed by partition executors, tagged by class.
`raft.executor.rejections_total`	ops	Client proposals rejected because the per-partition queue was full.
`raft.wal.batches_total`	batches	WAL write batches dispatched to the storage adapter.
`raft.wal.operations_total`	ops	Individual WAL write operations completed.
`raft.wal.compaction_passes_total`	passes	Compaction passes invoked, tagged by outcome.
`raft.wal.compaction_blocked_by_durability_floor_total`	passes	Compaction passes blocked by the application durability floor.
`raft.snapshot.transfer_failures_total`	failures	Failed snapshot transfer attempts, tagged by cause.
`raft.stale_completions_total`	ops	WAL completions discarded as stale.
`raft.elections_started_total`	elections	Election attempts (transitions to candidate), tagged by partition.
`raft.heartbeats_sent_total`	heartbeats	Heartbeat messages sent by leader partitions.
`raft.balancer.moves_total`	moves	Leadership transfer moves, tagged by outcome (`planned`, `succeeded`, `timed_out`).
`raft.balancer.skipped_passes_total`	passes	Balancer passes skipped because the global view was incomplete.

17.7.2 Histograms

Instrument	Unit	Description
`raft.executor.operation_duration_ms`	ms	Per-operation dispatch latency in the partition executor, tagged by class.
`raft.wal.batch_size`	ops	Distribution of WAL write batch sizes.
`raft.wal.durability_floor_lag`	entries	Entries between the application durability floor and the last checkpoint.
`raft.heartbeat_delay_ms`	ms	Interval between consecutive heartbeats sent by a leader partition.
`raft.election_delay_ms`	ms	Time since last heartbeat when an election was triggered.

17.7.3 Observable Gauges

Instrument	Unit	Description
`raft.executor.client_queue_depth`	ops	Current client proposals pending in each partition executor's queue.
`raft.wal.queue_depth`	ops	Current pending or in-flight WAL operations per partition.
`raft.balancer.count_imbalance`	ratio	Max node leadership count minus target (P0 leader only).
`raft.balancer.load_imbalance`	ratio	Fractional load imbalance: `(maxLoad / meanLoad) - 1`.

The observable gauges use weak references to registered executor and scheduler instances. This prevents the metrics system from keeping disposed partitions alive. The gauge callback iterates live instances and reports their current state.

17.7.4 Connecting to OpenTelemetry

Register the `Kommander` meter with your OpenTelemetry pipeline:

```
builder.Services.AddOpenTelemetry()
    .WithMetrics(metrics =>
    {
```

```
        metrics.AddMeter("Kommander");
        metrics.AddPrometheusExporter();
    });
```

All Kommander instruments are then available in your metrics backend (Prometheus, Grafana, Datadog, or any OpenTelemetry-compatible collector).

17.8 Structured Logging

Kommander uses `[LoggerMessage]` source-generated log methods throughout the codebase. These generate zero allocations when the log level is not enabled. The library defines over 60 message templates that cover:

- WAL restore progress and completion.
- Voting decisions and election outcomes.
- Proposal submission and commit acknowledgment.
- Rollback events.
- Heartbeat send and receive.
- Backfill progress and refusals.
- Snapshot install start and completion.
- Compaction pass results.
- Membership changes (join, leave, promotion).
- Replica placement decisions.
- Transport authentication results.

17.8.1 What to Watch For

Warning level. These messages indicate conditions that need attention but do not stop the cluster:

- Configuration validation warnings at startup (for example, a setting that reduces durability).
- Compaction passes that find no checkpoint.
- Snapshot transfer failures.
- Backfill refusals.
- Clock skew warnings during authentication.

Error level. These messages indicate failures:

- WAL write failures.
- Transport errors between nodes.
- Unhandled exceptions in partition executors.

Information level. These messages are useful for understanding cluster behavior:

- Election started and completed.
- Leader transitions.
- Node join and leave events.
- Snapshot transfers started and completed.

17.8.2 Configuration

Pass your logger through `RaftConfiguration`:

```
RaftConfiguration configuration = new()
{
    NodeName = "node-a",
    Host = "localhost",
    Port = 8001,
    Logger = app.Logger
};
```

Kommander uses the standard `ILogger` interface. Set the log level through your logging framework's configuration.

17.9 WAL Phase Instrumentation

The `WalPhaseInstrumentation` class provides detailed latency decomposition for the WAL write path. It is disabled by default because it adds per-operation overhead.

When enabled, it records three phases:

Phase	What it measures
`LeaderPropose`	Time to write the `Proposed` entry and fsync.
`LeaderCommit`	Time to write the `Committed` entry (with or without fsync).
`FollowerAppend`	Time for a follower to write an appended entry.

For each phase, the instrumentation reports:

- Enqueued count.
- Mean latency.
- P50 latency (median).
- P99 latency.

Enable it only for targeted performance investigations. The overhead is small but not zero.

17.10 Building a Health Endpoint

Combine the diagnostic APIs into a health endpoint that reports cluster status:

```
app.MapGet("/health", (IRaft raft) =>
{
    ClusterMembership membership = raft.GetMembership();
    IReadOnlyList<RaftPartitionRange> partitions = raft.GetPartitionMap();
                                IReadOnlyList<string>       activeNodes       =
raft.GetActiveNodes(TimeSpan.FromSeconds(30));
```

```
    List<object> partitionHealth = [];

    foreach (RaftPartitionRange partition in partitions)
    {
        int pid = partition.PartitionId;

        partitionHealth.Add(new
        {
            partitionId = pid,
            generation = partition.Generation,
            state = partition.State.ToString(),
            replicas = partition.Replicas.Count,
            replicationFactor = partition.ReplicationFactor,
            commitIndex = raft.GetCommitIndex(pid),
           opsPerSecond = Math.Round(raft.GetPartitionLogOpsPerSecond(pid), 1),
            walQueueDepth = raft.GetPartitionWalQueueDepth(pid),
            commitWaitMs = Math.Round(raft.GetPartitionCommitWaitMs(pid), 1),
            staleProposedSkipped = raft.GetStaleProposedSkippedCount(pid),
            snapshotTransfers = raft.GetSnapshotStatuses(pid).Select(s => new
            {
                follower = s.FollowerEndpoint,
                failedAttempts = s.FailedAttempts,
                lastError = s.LastError,
                inFlight = s.InFlight,
                retryBackoffRemaining = s.RetryBackoffRemaining.TotalSeconds
            }),
            backfillRefusals = raft.GetBackfillStatuses(pid).Select(b => new
            {
                follower = b.FollowerEndpoint,
                anchorIndex = b.AnchorIndex,
                firstAvailableIndex = b.FirstAvailableIndex,
                occurrences = b.Occurrences
            })
        });
    }

    return Results.Ok(new
    {
        membership = new
        {
            version = membership.MembershipVersion,
            members = membership.Members.Select(m => new
            {
                endpoint = m.Endpoint,
```

```
                nodeId = m.NodeId,
                role = m.Role.ToString()
            })
        },
        activeNodes,
        partitions = partitionHealth
    });
});
```

17.10.1 Reading the Health Response

A healthy response looks like this (three nodes, two partitions):

```
{
  "membership": {
    "version": 3,
    "members": [
      { "endpoint": "https://node-a:8001", "nodeId": 1, "role": "Voter" },
      { "endpoint": "https://node-b:8001", "nodeId": 2, "role": "Voter" },
      { "endpoint": "https://node-c:8001", "nodeId": 3, "role": "Voter" }
    ]
  },
  "activeNodes": [
    "https://node-b:8001",
    "https://node-c:8001"
  ],
  "partitions": [
    {
      "partitionId": 0,
      "commitIndex": 15230,
      "opsPerSecond": 450.2,
      "walQueueDepth": 0,
      "commitWaitMs": 1.2,
      "staleProposedSkipped": 0,
      "snapshotTransfers": [],
      "backfillRefusals": []
    },
    {
      "partitionId": 1,
      "commitIndex": 12810,
      "opsPerSecond": 380.7,
      "walQueueDepth": 0,
      "commitWaitMs": 1.5,
      "staleProposedSkipped": 0,
      "snapshotTransfers": [],
      "backfillRefusals": []
```

```
    }
  ]
}
```

Signs of trouble:

- `walQueueDepth > 0` on a sustained basis means the WAL cannot keep up with the write rate.
- `commitWaitMs` rising means fsync latency is increasing. Check disk I/O.
- `snapshotTransfers` not empty means followers need snapshot catch-up. Check `failedAttempts` and `lastError`.
- `backfillRefusals` not empty means followers fell behind past the compaction horizon.
- `activeNodes` missing a peer means that node has not communicated within the window. Check network connectivity.
- A member with role `Leaving` means a decommission is in progress. Wait for it to complete.

17.11 Metric Alerting Guidelines

Metric	Condition	Suggested action
`raft.executor.rejections_total`	Increasing	The partition queue is full. Reduce write rate or increase queue capacity.
`raft.executor.client_queue_depth`	Sustained > 100	Backpressure is building. Check commit latency.
`raft.wal.queue_depth`	Sustained > 0	WAL writes are queuing. Check disk throughput.
`raft.snapshot.transfer_failures_total`	Increasing	Snapshot transfers are failing. Check network and `IRaftPartitionStateTransfer` implementation.
`raft.elections_started_total`	Burst of elections	Frequent elections indicate heartbeat timeouts. Check network latency and `HeartbeatTimeout`.
`raft.heartbeat_delay_ms`	P99 > heartbeat interval	The leader is slow to send heartbeats. Check CPU saturation.
`raft.balancer.moves_total` (timed_out)	Increasing	Leadership transfers are timing out. Check follower readiness.
`raft.balancer.skipped_passes_total`	Increasing	The balancer cannot get a complete global view. Check gossip connectivity.
`raft.balancer.count_imbalance`	Sustained > 2	Leadership is unevenly distributed. Check balancer configuration.

17.12 Diagnostic Queries in Context

The following table maps common operational questions to the API call that answers them:

Question	API call
How many ops/sec is this partition processing?	`GetPartitionLogOpsPerSecond(partitionId)`
Is the WAL backed up?	`GetPartitionWalQueueDepth(partitionId)`
How long does a commit take?	`GetPartitionCommitWaitMs(partitionId)`
How far has this partition committed?	`GetCommitIndex(partitionId)`
Are any followers receiving snapshots?	`GetSnapshotStatuses(partitionId)`
Did any followers fall behind compaction?	`GetBackfillStatuses(partitionId)`
Are duplicate proposals being rejected?	`GetStaleProposedSkippedCount(partitionId)`
Which nodes are alive?	`GetActiveNodes(TimeSpan)`
When was this node last seen?	`GetLastNodeActivity(endpoint)`
Who is in the cluster?	`GetMembership()`
What is the partition layout?	`GetPartitionMap()`

17.13 Summary

- `GetPartitionLogOpsPerSecond`, `GetPartitionWalQueueDepth`, and `GetPartitionCommitWaitMs` are the three primary per-partition health signals. They use EWMA accumulators for smooth, recent-biased values.
- `GetCommitIndex` shows the committed frontier. Compare it across nodes to measure replication lag.
- `GetSnapshotStatuses` and `GetBackfillStatuses` reveal stuck followers and catch-up problems.
- `GetStaleProposedSkippedCount` tracks duplicate proposal rejections.
- `GetActiveNodes` and `GetLastNodeActivity` provide node-level liveness data through the `NodeActivityTracker`.
- `KommanderMetrics` emits counters, histograms, and observable gauges through a `System.Diagnostics.Metrics.Meter` named `"Kommander"`. Connect it to OpenTelemetry for export.
- Structured logging uses `[LoggerMessage]` source generators for zero-allocation log messages. Watch Warning and Error levels in production.
- Build a health endpoint that combines membership, partition status, and diagnostic queries into a single JSON response.
- The next chapter covers the repository architecture and component map for contributors.

18 Anatomy of a Raft Partition

Each partition is a miniature Raft instance. It has its own term, voted-for, log, commit index, leader, and follower progress. When you debug a stuck election, trace a replication

stall, or contribute to the consensus engine, you need to understand the state inside one partition.

This chapter opens the partition and examines every piece of state it holds.

18.1 The Two Shells

Two classes wrap a partition:

RaftPartition is the outer shell. It owns the executor, the state machine, and the WAL handler. It provides `Post` (fire-and-forget) and `Ask` (reply-awaiting) dispatch to the executor. It holds a volatile `Leader` reference that any thread can read without going through the executor.

RaftPartitionStateMachine is the inner shell. It holds all the consensus collaborators and dispatches typed `RaftRequest` messages to the correct one. All consensus logic flows through this class, but it contains almost none of that logic itself. It delegates to 14 collaborators.

```
RaftPartition (outer shell)
│
├── executor          RaftPartitionExecutor
│                     Serial, single-owner concurrency
│
├── stateMachine      RaftPartitionStateMachine (inner shell)
│   │
│   ├── coreState         RaftPartitionCoreState
│   ├── election          ElectionCoordinator
│   ├── replicator        LogReplicator
│   ├── ackProcessor      ReplicationAckProcessor
│   ├── heartbeat         HeartbeatDriver
│   ├── backfillSender    BackfillSender
│   ├── followerHandler   FollowerAppendHandler
│   ├── applicator        LogApplicator
│   ├── readIndex         ReadIndexCoordinator
│   ├── completionRouter  WalCompletionRouter
│   ├── proposals         ProposalRegistry
│   ├── tracker           ReplicationTracker
│   ├── snapshotSender    SnapshotSender
│   ├── snapshotInstaller SnapshotInstaller
│   └── backfillTracker   NonContiguousBackfillTracker
│
└── walHandler        RaftWriteAhead
```

The executor guarantees that only one operation runs at a time for a given partition. This means the collaborators do not need locks. They read and write shared state freely because only the executor thread accesses them.

18.2 RaftPartitionCoreState

`RaftPartitionCoreState` is the consensus nucleus. Every collaborator reads from it. It holds the fields that define the partition's position in the Raft protocol.

18.2.1 Node State

```
public enum RaftNodeState
{
    Follower = 0,
    Candidate = 1,
    Leader = 2
}
```

The `NodeState` property is the only volatile field in the core state. It can be read from any thread. All other fields are accessed only from the executor thread.

```
State transitions:

Follower ──► Candidate ──► Leader
   ▲             │            │
   │             │            │
   └─────────────┘            │
   └──────────────────────────┘


Follower → Candidate:  election timeout expired, pre-vote succeeded
Candidate → Leader:    quorum of votes received
Candidate → Follower:  election timed out, or higher term discovered
Leader → Follower:     higher term discovered, or check-quorum failed
```

The setter for `NodeState` invalidates the `LeadershipLease` on any non-Leader assignment. This ensures that a deposed leader's lease is immediately revoked.

18.2.2 Term and Voting

Field	Type	Description
`CurrentTerm`	`long`	The current Raft term. The setter invalidates the `LeadershipLease` on any change.
`LastVotation`	`HLCTimestamp`	HLC timestamp of the last vote this node granted.
`LastVotationTicks`	`long`	Monotonic shadow of `LastVotation`. Drives the recent-vote cooldown that prevents immediate re-election.
`VotingStartedAt`	`HLCTimestamp`	HLC timestamp when this node became a candidate.
`VotingStartedTicks`	`long`	Monotonic shadow. Bounds how long a candidacy may stall.
`ElectionTimeout`	`TimeSpan`	Current randomized election timeout. Re-drawn each election round.

18.2.3 Commit and Apply Frontiers

Field	Type	Description
`LocalCommittedIndex`	`long`	The highest log index committed on this node. Volatile-backed for off-thread reads. Reset to -1 on leader-to-follower transitions.
`LiveCommitFloor`	`long`	The commit frontier at the moment this node became leader. Re-anchored at every promotion.
`LastAppliedIndex`	`long`	The highest log index delivered to the application consumer. Volatile-backed.

The gap between `LocalCommittedIndex` and `LastAppliedIndex` represents entries that are committed but not yet applied. In steady state, this gap is zero or near zero.

18.2.4 Leadership Lease

```
internal sealed record LeadershipLease(long Term, long ConfirmedTicks);
```

The `LeadershipLease` is an immutable snapshot of a completed leadership confirmation. It is published through a volatile reference so that off-thread readers (the fast-path leadership check) can use it without going through the executor.

- `Term`: the term in which the confirmation completed.
- `ConfirmedTicks`: the monotonic timestamp of the quorum confirmation.

A lease is fresh when `elapsed(ConfirmedTicks, now) < HeartbeatInterval`. The fast-path check uses this to confirm leadership without an executor round-trip.

The lease is invalidated (set to `null`) in three places:

1. The `NodeState` setter, on any non-Leader assignment.
2. The `CurrentTerm` setter, on any change.
3. The `ReadIndexCoordinator`, when it fails all waiters.

18.2.5 Heartbeat and Quiescence

Field	Type	Description
`LastHeartbeat`	`HLCTimestamp`	HLC timestamp of the last accepted heartbeat from a leader.
`LastHeartbeatTicks`	`long`	Monotonic shadow. Drives the "time since last leader contact" check.
`Quiesced`	`bool`	Whether this partition is quiesced (idle, not heartbeating).
`LastProposalAt`	`HLCTimestamp`	HLC timestamp of the last real proposal.
`LastProposalAtTicks`	`long`	Monotonic shadow. Drives the quiesce-after gate.
`Restored`	`bool`	Set once WAL restore is complete.

18.2.6 Promotion Barrier

Field	Type	Description
`LeadershipBarrierTicket`	`HLCTimestamp`	Ticket of the in-flight promotion-barrier no-op.
`LeadershipBarrierTerm`	`long`	Term in which the barrier was proposed.
`LeadershipBarrierArmedTicks`	`long`	Monotonic timestamp when the barrier was armed. Drives the timeout revert.

When a node wins an election, it proposes a no-op entry (the promotion barrier). Until this entry commits, the node reports itself as `Candidate` to external callers, even though internally it is `Leader`. This prevents the node from serving reads before its commit index is confirmed by a quorum.

18.3 The Election Coordinator

The `ElectionCoordinator` handles everything about becoming or refusing to become leader: pre-vote probes, real elections, and vote granting.

18.3.1 Election Phase

```
public enum RaftElectionPhase
{
    None,       // no election activity
    PreVote,    // side-effect-free pre-vote round for CurrentTerm + 1
    Election    // real election in progress (term bumped, self-vote cast)
}
```

18.3.2 Pre-Vote (Raft Section 9.6)

Before starting a real election, the node runs a pre-vote round. Pre-votes do not change any persistent state. They ask peers: "Would you vote for me if I started an election?"

```
Pre-vote sequence:

1. Node A's election timeout expires.
2. Node A sends PreVoteRequest to all peers.
   (Hypothetical term = CurrentTerm + 1)
3. Each peer checks:
   - Is the candidate's log at least as fresh as mine?
   - Am I satisfied that the current leader is not alive?
   A quiesced follower denies pre-votes if the expected leader
   is SWIM-Alive.
4. If a quorum grants pre-votes:
   Node A starts a real election.
```

```
5. If not:
   Node A stays as follower. No term was bumped.
   No state was changed on any node.
```

Pre-votes prevent a partitioned node from bumping its term. Without pre-votes, a node isolated from the leader would increment its term repeatedly. When it rejoins, its high term would force the leader to step down, even though the leader was healthy.

18.3.3 Real Election

```
Real election sequence:

1. Node A transitions to Candidate.
2. Node A increments CurrentTerm.
3. Node A persists hard state (term + vote-for-self).
4. Node A sends RequestVote to all peers.
5. Each peer checks:
   - Is the candidate's term >= my term?
   - Have I already voted in this term?
   - Is the candidate's log at least as fresh as mine?
   If all checks pass: grant vote, persist hard state.
6. If Node A receives a quorum of votes:
   Node A calls BecomeLeaderAsync.
   Node A sends an immediate heartbeat.
7. If Node A's election times out:
   Node A reverts to Follower.
   Node A randomizes its election timeout.
```

For a single-node cluster, the pre-vote round promotes directly to a real election, and the real election promotes directly to leader (no network round-trips needed).

18.3.4 Log Freshness Check

The vote-granting side checks that the candidate's log is at least as fresh as the local log. The comparison is lexicographic on (last log term, max log index):

1. If the candidate's last log term is greater, the candidate is fresher.
2. If the terms are equal, the candidate with the higher max log index is fresher.
3. If both are equal, the candidate is as fresh as the local node.

A node with a stale log cannot become leader. This prevents a node that missed entries from overwriting committed data.

18.3.5 Vote Record

The `ElectionCoordinator` tracks two data structures:

- `votes`: a dictionary from term to set of voting endpoints. Used to count real election tallies.
- `expectedLeaders`: a dictionary from term to endpoint. Records who this node voted for in each term.

Both are cleared on leader transitions.

18.3.6 Election Timeout Randomization

Each election round draws a new timeout from the range `[StartElectionTimeout, EndElectionTimeout)`. The randomization prevents synchronized elections where all nodes time out at the same moment. When `ElectionTimeoutSeed` is configured, the random generator is seeded with a deterministic hash of the seed, partition ID, and node ID. This makes elections reproducible in simulation tests.

18.4 The Replication Tracker

The `ReplicationTracker` maintains per-follower replication bookkeeping. It is owned by the leader and accessed only from the executor thread.

18.4.1 Six Maps

Map	Key	Value	Description
`lastCommitIndexes`	endpoint	log index	The peer's committed frontier as reported in its acknowledgment. Last-writer-wins.
`startCommitIndexes`	endpoint	log index	Where the peer's log started, from handshakes or vote replies.
`nextIndex`	endpoint	log index	The next entry to send to this follower. Seeded to `leaderMaxLog + 1` (optimistic).
`matchIndex`	endpoint	log index	The highest log index confirmed replicated on this follower. Starts at 0.
`backfillPausedUntilTicks`	endpoint	ticks	Saturation cooldown deadline. Not cleared on leader transitions.
`regressedFrontiers`	endpoint	log index	Crash-restart regression notes (take-once, consumed by heartbeat).

An additional map, `mismatchAnchors`, records per-peer log-mismatch anchored-repair notes (also take-once).

18.4.2 Replication Cursor Flow

```
Leader wins election with maxLog = 100.

For each follower:
  nextIndex[follower] = 101   (optimistic)
  matchIndex[follower] = 0    (nothing confirmed)
```

```
Leader sends AppendEntries(prevLogIndex=100) to follower.

Case 1: Follower has entry 100.
  Follower accepts. Leader updates:
  matchIndex[follower] = 101

Case 2: Follower does NOT have entry 100.
  Follower rejects. Leader decrements:
  nextIndex[follower] = 100
  Leader retries with prevLogIndex=99.

This continues until the follower accepts.
Then the leader sends all entries from nextIndex forward.
```

18.4.3 Bulk Resets

All six maps (except `backfillPausedUntilTicks`) reset together when the node loses leadership. The saturation cooldown survives leadership transitions because it reflects storage pressure on the follower, not leadership state.

- `ClearAll()`: clears everything except `backfillPausedUntilTicks`.
- `ClearProgressKeepingCommitFrontiers()`: clears `nextIndex`, `matchIndex`, and notes, but keeps `lastCommitIndexes`.
- `RemovePeer(endpoint)`: removes all progress for one peer.

18.5 The Proposal Registry

The `ProposalRegistry` is the leader's in-flight proposal ledger. It tracks active proposals from submission to quorum completion.

18.5.1 Two Data Structures

Active proposals. A dictionary from `HLCTimestamp` (the proposal ticket) to `RaftProposalQuorum`. Each entry represents one proposal waiting for a quorum of acknowledgments.

Pending WAL operations. A dictionary from WAL operation ID to `RaftPendingWalOperation`. Each entry represents one WAL write in flight. When the WAL reports completion, the registry matches the operation ID to the pending entry and routes the completion to the state machine.

18.5.2 Key Operations

Method	Description
`TryAdd`	Registers a new proposal.
`TryGet`	Retrieves a proposal by ticket.
`FailAllWaitersAndClear`	Completes all active proposal waiters with failure, then clears. Called on step-down.
`TryReleaseTicketOnQuorumDurable`	Single-fsync fast path: when `WalSingleFsyncCommit` is on, advances `LocalCommittedIndex` and marks committed as soon as the propose quorum is durable.
`RetryUnresolved`	Re-sends entries to voters that have not acknowledged. Processes at most 8 proposals per round.
`PruneSettled`	Releases settled proposals older than 30 seconds.

18.5.3 Pending WAL Operation Pool

The registry maintains a bounded pool of `RaftPendingWalOperation` objects (maximum 256). It rents from the pool when starting a WAL write and returns the object when the write completes. This avoids allocations on the hot path.

18.6 The Heartbeat Driver

The `HeartbeatDriver` sends periodic heartbeats from the leader to all followers. It is also the catch-up path when writes go quiet.

18.6.1 What a Heartbeat Does

Each heartbeat tick runs the following logic per follower:

1. Compute the follower gap: `LocalCommittedIndex - effectiveFloor`.
2. Decide whether to backfill. Three triggers cause backfill:
   - The gap exceeds `BackfillThreshold`.
   - An idle-tail gap exists: live replication is quiet, but a committed entry sits above `LiveCommitFloor`.
   - A crash-restart regression or log-mismatch note exists (take-once, paced).
3. If backfilling: send entries through `BackfillSender.TrySendBackfillBatchAsync`.
4. If not backfilling: send an empty heartbeat.

The heartbeat also retries unresolved proposals through `ProposalRegistry.RetryUnresolved`.

18.6.2 Heartbeat and Quiescence

When a partition is quiesced, the `SendHeartbeat` method returns immediately (unless forced). The leader stops sending heartbeats to save network and CPU resources. The SWIM failure detector takes over liveness detection.

18.6.3 Handshake

The `ReceiveHandshake` method handles a peer's initial contact. It checks membership, detects NodeId collisions (fatal error), and records the peer's starting commit index.

18.7 The Log Applicator

The `LogApplicator` delivers committed log entries to the application consumer. It guarantees two properties:

1. **Exactly-once delivery.** Each committed entry is delivered exactly once.
2. **Gap-free delivery.** Entries are delivered in log-index order with no gaps.

18.7.1 How It Works

```
WAL state:

[101:Committed] [102:Committed] [103:Committed] [104:Proposed]

LastAppliedIndex = 100

DrainCommittedAppliesAsync(upToIndex: 103):
  Read entries 101, 102, 103 from WAL
  Apply 101 → LastAppliedIndex = 101
  Apply 102 → LastAppliedIndex = 102
  Apply 103 → LastAppliedIndex = 103
  Stop at 104 (Proposed, not committed)
  Return true
```

The applicator reads entries in 512-entry batches. It processes all entry types (including rolled-back entries) so the cursor advances past them. It stops when it encounters a gap above the snapshot floor, an unresolved `Proposed` entry, or a missing tail.

18.7.2 Deferred Leader Applies

When leader-side quorum completions arrive out of order, the applicator parks them in a sorted dictionary keyed by the lowest log index. The `FlushDeferredLeaderAppliesAsync` method delivers deferred batches that are contiguous with the applied cursor, in ID order. On a term change, all deferred entries are cleared.

18.7.3 Inherited Applies

When a new leader takes office, it may inherit `Proposed` entries from the previous term. The `DrainInheritedAppliesAsync` method applies these entries. It only delivers entries from strictly older terms. It normalizes `Proposed` entries to `Committed` copies for the consumer. The method returns one of three statuses:

Status	Meaning
`Covered`	The cursor covers the requested range.
`Hole`	An entry is missing above the snapshot floor.
`BlockedByInFlight`	A current-term `Proposed` entry is still in flight.

18.8 The Partition Lifecycle

18.8.1 Initialization

When `RaftPartition` is constructed:

1. It creates the `RaftWriteAhead` handler for WAL operations.
2. It creates host and facade adapters (to break circular dependencies).
3. It creates the `RaftPartitionStateMachine` with all collaborators.
4. It creates the `RaftPartitionExecutor` and starts it.
5. It wires the quiescence callback.

The partition starts as a `Follower` with `CurrentTerm = 0` and no votes.

18.8.2 WAL Restore

On first tick, the state machine replays entries from the WAL. The `Restored` flag is set to `true` when replay is complete. Until restore finishes, the partition rejects most operations.

18.8.3 Election

After restore, the partition waits for heartbeats from a leader. If no heartbeat arrives within the election timeout, the partition starts a pre-vote. If the pre-vote succeeds, it starts a real election. If it wins, it becomes leader.

18.8.4 Steady State

In steady state, the partition is either a leader or a follower.

Leader steady state:

- The periodic tick calls `CheckPartitionLeadershipAsync`.
- The tick checks: promotion barrier timeout, check-quorum step-down, quiescence re-arm, heartbeat cadence.
- Proposals arrive through `ReplicateLogs`, flow through the executor, and reach the `LogReplicator`.
- Acknowledgments arrive through `CompleteAppendLogs` and reach the `ReplicationAckProcessor`.
- WAL completions arrive through `CompleteWalOperation` and reach the `WalCompletionRouter`.
- Committed entries flow through the `LogApplicator` to the application consumer.

Follower steady state:

- The periodic tick checks: is the heartbeat recent? If yes, do nothing. If timed out, start pre-vote.
- Appended entries arrive through `AppendLogs` and reach the `FollowerAppendHandler`.
- The follower validates the leader's term, performs the Log Matching check, and writes to the WAL.

18.8.5 Step-Down

When a leader discovers a higher term (from a vote request, an append response, or any RPC), it steps down:

1. `NodeState` changes to `Follower`.
2. `LeadershipLease` is invalidated.
3. All active proposal waiters are failed.
4. The replication tracker is cleared.
5. Pre-vote state is reset.
6. Quiescence is cleared.

The node then waits for heartbeats from the new leader.

18.9 Quiescence

Quiescence stops heartbeating for idle partitions. When a partition has no active proposals and no recent writes, the leader stops sending heartbeats. This reduces network traffic and CPU usage for partitions that are not actively used.

18.9.1 Configuration

```
EnableQuiescence = true                                    // default: on
QuiesceAfter = TimeSpan.FromMilliseconds(1500)             // default: ~3x
HeartbeatInterval
```

Quiescence requires SWIM to be active (`PingInterval > 0` and `PingInterval < StartElectionTimeout`). The SWIM failure detector takes over liveness detection when heartbeats stop.

18.9.2 Entering Quiescence

The leader enters quiescence when all of these conditions are true:

1. `EnableQuiescence` is `true`.
2. The partition is not already quiesced.
3. No active proposals exist (`ActiveCount == 0`).
4. At least one proposal happened before (`LastProposalAtTicks != 0`).
5. No follower is lagging (`HasLaggingPeer()` returns `false`).
6. The time since the last proposal exceeds `QuiesceAfter`.

When all conditions are met, the leader:

1. Sets `Quiesced = true`.
2. Sends a quiesce marker (an empty `AppendLogs` with a quiesce flag) to all followers.

3. Stops sending heartbeats.

18.9.3 Follower Behavior During Quiescence

A quiesced follower does not start elections when the heartbeat timeout expires. Instead, it checks the SWIM liveness state of the expected leader:

- **Alive:** Stay calm. The leader is healthy but not heartbeating.
- **Suspect or Dead:** Un-quiesce and start a pre-vote. The leader may have failed.

A quiesced follower also denies pre-votes from challengers if the expected leader is SWIM-Alive. This prevents unnecessary elections.

18.9.4 Exiting Quiescence

The leader exits quiescence when:

- A follower falls behind (`HasLaggingPeer()` returns `true` on the periodic tick).
- A new proposal arrives.
- The node loses leadership.

Every demotion or state transition clears the `Quiesced` flag.

18.10 Let's Break It: Term Confusion

Two candidates start elections at the same time. Both increment their terms. One wins.

```
Initial state:
  node-a: term=5, Follower
  node-b: term=5, Follower
  node-c: term=5, Follower (was leader, stepped down)

1. node-a and node-b both time out simultaneously.

2. node-a: pre-vote succeeds → starts real election.
   node-a: term=6, Candidate, votes for self.

3. node-b: pre-vote succeeds → starts real election.
   node-b: term=6, Candidate, votes for self.

4. node-c receives RequestVote(term=6) from node-a first.
   node-c: grants vote to node-a. Persists: votedFor=node-a, term=6.

5. node-c receives RequestVote(term=6) from node-b.
   node-c: already voted in term 6. Denies node-b.

6. node-a: receives vote from node-c. Quorum = {node-a, node-c}.
   node-a becomes Leader for term 6.

7. node-a sends heartbeat(term=6) to node-b.
   node-b sees term 6 >= its own term 6.
```

```
    node-b steps down to Follower. Adopts node-a as leader.

Result: node-a is leader. No data was lost. No split-brain.
```

The key safety property: a node votes for at most one candidate per term. The vote is persisted to the WAL before the response is sent. Even if the node crashes and restarts, it remembers its vote and will not grant a second vote in the same term.

18.11 Summary

- Each partition is a miniature Raft instance with its own term, voted-for, commit index, applied index, and leader.
- `RaftPartitionCoreState` holds the consensus nucleus. All collaborators read from it. Only the executor thread writes to it.
- `NodeState` (Follower, Candidate, Leader) is volatile for off-thread reads. All other core state is executor-thread-only.
- `LeadershipLease` provides a fast-path leadership check without an executor round-trip.
- `ElectionCoordinator` implements pre-votes (Raft section 9.6) before real elections. Pre-votes prevent term inflation from partitioned nodes.
- `ReplicationTracker` maintains six per-follower maps: commit frontiers, start positions, `nextIndex`, `matchIndex`, saturation backoff, and regression notes.
- `ProposalRegistry` tracks in-flight proposals from submission to quorum completion. It pools `RaftPendingWalOperation` objects to avoid allocations.
- `HeartbeatDriver` sends periodic heartbeats and triggers backfill for lagging followers.
- `LogApplicator` delivers committed entries in gap-free, exactly-once order. It parks out-of-order completions and flushes them contiguously.
- Quiescence stops heartbeating for idle partitions. SWIM takes over liveness detection. Followers gate elections on SWIM state of the expected leader.
- The promotion barrier (a no-op entry) prevents a new leader from serving reads until a quorum confirms its commit index.
- The next chapter traces a complete `ReplicateLogs` call through every layer of the system.

19 Chapter 27: Hybrid Logical Clocks

Raft does not require a hybrid logical clock (HLC). The Raft algorithm uses terms and log indexes alone to order entries and to detect stale messages. Kommander, however, needs a second kind of timestamp: a unique proposal ticket ID. Every time a leader proposes a log batch, it must assign an identifier that is monotonically increasing, causality-preserving, and cheap to produce under high concurrency. The HLC fills that role.

This chapter explains why the HLC exists, how it packs two values into a single `long` for lock-free operation, and how the `SendOrLocalEvent` and `ReceiveEvent` algorithms work. The distinction between Raft terms and HLC timestamps is important. Terms are

the correctness mechanism. HLC timestamps are an implementation choice for ticket identity.

19.1 Why Kommander Needs an HLC

A proposal ticket ID must satisfy three constraints:

1. **Monotonicity.** Each new ticket must be strictly greater than the previous one on the same node. If two proposals received the same ticket, a client that polls by ticket ID could not tell them apart.
2. **Causality preservation.** If node A sends a message to node B, the timestamp that B mints after it receives the message must be greater than the timestamp that A assigned when it sent the message. Without this property, events that are causally related could appear to occur in the wrong order.
3. **Zero allocation under contention.** The clock is a single instance shared by every partition on a node. It fires once per propose, once per replication acknowledgment, once per heartbeat, and once per follower append. Under load, thousands of events per second pass through the same clock. Each event must produce a timestamp without allocating heap memory.

A simple atomic counter satisfies constraints 1 and 3 but not constraint 2. A wall-clock timestamp satisfies constraint 2 but not constraint 1 (clocks can jump backward after NTP adjustments). An HLC satisfies all three.

19.2 Physical Time Plus Logical Counter

An HLC timestamp has two components:

- **L** (logical-physical time): the maximum of the node's physical wall clock and the highest L the node has observed from itself or from any message. L never decreases.
- **C** (counter): a tie-breaker that increments when multiple events occur within the same millisecond. C resets to zero whenever L advances.

The pair `(L, C)` is a single scalar clock. Two timestamps compare first by L, then by C. If both L and C are equal, the timestamps are from the same event (or, across nodes, from different nodes at the same logical instant, disambiguated by the node ID).

The original HLC paper (Kulkarni et al., 2014) defines these two rules:

- **Send or local event:** set L to `max(currentL, physicalTime)`. If L did not change, increment C. Otherwise reset C to zero.
- **Receive event:** set L to `max(currentL, messageL, physicalTime)`. Set C based on which component(s) determined the new L.

Kommander implements both rules in `HybridLogicalClock.cs`.

19.3 Bit Packing: One Long, Zero Allocations

The previous Kommander implementation stored `(L, C)` in an immutable `ClockState` object and swapped it with compare-and-swap (CAS). Every event, including every CAS retry, allocated a generation-0 object on the heap. Because the clock fires on

every replication round-trip, it was the largest steady-state allocation source in the replication path.

The current implementation eliminates all allocation. It packs both L and C into a single 64-bit `long`:

```
63                              22  21                  0
├──────────── L (42 bits) ─────────┤  ├──── C (22 bits) ─┤
```

- **42 bits for L:** `2^42` milliseconds from the Unix epoch. This range reaches approximately year 2109. The value is sufficient for any practical deployment lifetime.
- **22 bits for C:** up to 4,194,303 events within a single physical millisecond before the counter overflows. On overflow, the `Normalize` function increments L by one and resets C to zero. The result `(L+1, 0)` is strictly greater than `(L, anything)` in HLC order.

Three helper methods perform the packing:

```
private static long Pack(long l, long c)
{
    if (l is <= 0 or >= MaxL)
        throw new RaftException("Corrupted HLC clock");
    return (l << CounterBits) | c;
}

private static long UnpackL(long state) => state >>> CounterBits;

private static uint UnpackC(long state) => (uint)(state & CounterMask);
```

`Pack` validates that L is positive and below `MaxL` (`1L << 42`). `UnpackL` uses the unsigned right shift operator (`>>>`) to avoid sign extension. `UnpackC` masks the low 22 bits and casts to `uint`.

The clock stores the packed value in a single mutable field:

```
private long _state;
```

All reads use `Volatile.Read` for acquire-fence semantics. All writes use `Interlocked.CompareExchange` for atomic swap. No lock, no allocation, no object header.

19.4 SendOrLocalEvent: Minting a Timestamp

`SendOrLocalEvent` produces a new timestamp for a local event or for an outbound message. It takes a node ID and returns an `HLCTimestamp`.

```
public HLCTimestamp SendOrLocalEvent(int nodeId)
{
    while (true)
    {
        long current = Volatile.Read(ref _state);
        long currentL = UnpackL(current);
```

```
        long physicalTime = GetPhysicalTime();
        long newL = Math.Max(currentL, physicalTime);
        long newC = (newL == currentL) ? (long)UnpackC(current) + 1 : 0;
        Normalize(ref newL, ref newC);

        if (Interlocked.CompareExchange(ref _state, Pack(newL, newC), current)
== current)
            return new(nodeId, newL, (uint)newC);
    }
}
```

The algorithm proceeds step by step:

1. Read the current packed state with `Volatile.Read`.
2. Unpack `currentL` from the high 42 bits.
3. Read the physical wall clock (`DateTimeOffset.UtcNow.ToUnixTimeMilliseconds()`).
4. Set `newL` to the maximum of `currentL` and `physicalTime`. This guarantees L never decreases.
5. If L did not change, increment C. If L advanced, reset C to zero.
6. Call `Normalize`. If C overflowed the 22-bit range, increment L and reset C.
7. Attempt CAS. If another thread modified `_state` between the read and the swap, the CAS fails and the loop retries from step 1.
8. On success, return a new `HLCTimestamp` with the node ID, L, and C.

Every returned timestamp is installed in `_state`. An earlier implementation (`TrySendOrLocalEvent`) attempted one CAS and on failure returned a computed-but-not-installed timestamp. This was unsound. Two threads that both failed the CAS could return the same `(L, C+1)` value. Consumers that order mutations by timestamp (for example, a lock-coherence guard that compares with `>=`) would drop the later mutation. The fix was to make every path retry until the CAS succeeds.

19.5 ReceiveEvent: Merging a Remote Timestamp

`ReceiveEvent` merges a timestamp from an inbound message with the local clock. It takes a node ID and the remote message's `(L, C)` pair.

```
private HLCTimestamp ReceiveEventInternal(int nodeId, long messageL, uint
messageC)
{
    while (true)
    {
        long current = Volatile.Read(ref _state);
        long currentL = UnpackL(current);
        uint currentC = UnpackC(current);
        long physicalTime = GetPhysicalTime();
        long newL = Math.Max(currentL, Math.Max(messageL, physicalTime));
        long newC;
```

```
        if (newL == currentL && newL == messageL)
            newC = (long)Math.Max(currentC, messageC) + 1;
        else if (newL == currentL)
            newC = (long)currentC + 1;
        else if (newL == messageL)
            newC = (long)messageC + 1;
        else
            newC = 0;

        Normalize(ref newL, ref newC);

        if (Interlocked.CompareExchange(ref _state, Pack(newL, newC), current)
== current)
            return new(nodeId, newL, (uint)newC);
    }
}
```

The three-way max of `currentL`, `messageL`, and `physicalTime` determines `newL`. The counter logic has four cases:

Condition	newC	Reasoning
`newL == currentL && newL == messageL`	`max(currentC, messageC) + 1`	Both local and remote are at the same L. The counter must exceed both.
`newL == currentL` (only)	`currentC + 1`	Local L is the highest. Continue its counter sequence.
`newL == messageL` (only)	`messageC + 1`	Remote L is the highest. Continue its counter sequence.
Physical time dominated	`0`	L advanced past both local and remote. Reset the counter.

After the counter logic, `Normalize` handles overflow. The CAS loop guarantees atomic installation, the same as `SendOrLocalEvent`.

Two public overloads accept either an `HLClockMessage` (which carries only L and C) or an `HLCTimestamp` (which also carries a node ID). Both validate that `L != 0` and delegate to `ReceiveEventInternal`.

19.6 Counter Overflow and Self-Correction

When C exceeds the 22-bit maximum (4,194,303), `Normalize` increments L by one and resets C to zero:

```
private static void Normalize(ref long l, ref long c)
{
    if (c > CounterMask)
    {
        l += 1;
        c = 0;
    }
}
```

The result `(L+1, 0)` is strictly greater than any `(L, C)` in HLC comparison order. This preserves monotonicity.

After an overflow, L runs ahead of the physical wall clock. This is normal HLC behavior. When the wall clock eventually catches up to L, the counter resets to zero naturally (because `physicalTime > currentL` causes `newC = 0`). No special recovery is necessary.

In practice, overflow is rare. A single millisecond would need over four million events on the same node. Even at peak throughput with thousands of partitions, the counter stays well within the 22-bit range.

19.7 The HLCTimestamp Structure

`HLCTimestamp` is the value that crosses API boundaries. It is a `readonly record struct`:

```
public readonly record struct HLCTimestamp : IComparable<HLCTimestamp>
{
    [JsonPropertyName("n")] public int N { get; }
    [JsonPropertyName("l")] public long L { get; }
    [JsonPropertyName("c")] public uint C { get; }
}
```

Field	Type	Purpose
`N`	`int`	Node ID. Prevents timestamp collisions across nodes.
`L`	`long`	Physical-logical time in Unix-epoch milliseconds.
`C`	`uint`	Counter within the same millisecond.

19.7.1 Comparison Order

`CompareTo` imposes a total order: **(L, C, N)**. It compares L first, then C, then the node ID as a final tie-breaker. Two timestamps from different nodes with the same `(L, C)` are distinct and order consistently. The `Zero` constant (`new(0, 0, 0)`) represents an uninitialized timestamp.

19.7.2 Arithmetic Operators

`HLCTimestamp` supports addition and subtraction with `int` and `TimeSpan` operands. These operate on the L component only. Subtracting two timestamps returns a `TimeSpan` of the L difference. These operators are useful for computing time intervals between events.

19.7.3 Packed Long vs. Struct

The packed `long` representation exists only inside `HybridLogicalClock._state`. It is the internal storage format for lock-free CAS. The `HLCTimestamp` struct is the external representation that carries all three components (N, L, C) separately. No public API converts between the two forms.

19.8 The Wire Format: HLClockMessage

`HLClockMessage` is a lightweight record struct with only two fields:

```
public record struct HLClockMessage
{
    public long L { get; }
    public uint C { get; }
}
```

It carries the `(L, C)` pair without the node ID. The receiving node supplies its own node ID when it calls `ReceiveEvent`. This keeps the wire format compact: the sender's node ID is already known from the transport layer.

19.9 Where the Clock Fires

A single `HybridLogicalClock` instance lives on `RaftManager`. Every partition on the node shares the same clock. The `IRaftPartitionHost` interface exposes the clock to each consensus collaborator.

The following table lists the components that call the clock and their purpose:

Component	Method	HLC Call	Purpose
`LogReplicator`	`ReplicateLogs`	`SendOrLocalEvent`	Mint proposal ticket ID
`LogReplicator`	`ReplicateCheckpoint`	`SendOrLocalEvent`	Mint checkpoint ticket ID
`ReplicationAckProcessor`	(ack handler)	`ReceiveEvent`	Advance clock from follower ack
`ReplicationAckProcessor`	(backfill)	`TrySendOrLocalEvent`	Timestamp backfill batches
`FollowerAppendHandler`	(append handler)	`ReceiveEvent`	Advance clock from leader append
`ElectionCoordinator`	(vote response)	`ReceiveEvent`	Advance clock from vote response
`ElectionCoordinator`	(election event)	`TrySendOrLocalEvent`	Timestamp election events
`HeartbeatDriver`	(heartbeat)	`TrySendOrLocalEvent`	Timestamp heartbeats
`WalCompletionRouter`	(WAL completion)	`TrySendOrLocalEvent`	Timestamp completion events

The most important site is `LogReplicator.ReplicateLogs`. This is where the proposal ticket is born:

```
HLCTimestamp                                                    currentTime
= host.HybridLogicalClock.SendOrLocalEvent(host.LocalNodeId);
coreState.LastProposalAt = currentTime;

for (int i = 0; i < logs.Length; i++)
    logs[i].Time = currentTime;

wal.EnqueuePropose(coreState.CurrentTerm, logs, currentTime, autoCommit);
pendingPropose.TicketId = currentTime;

return (RaftOperationStatus.Pending, currentTime);
```

The returned `currentTime` becomes the ticket that a client uses to poll for commit status. It is also stamped onto every log entry in the batch and passed to the WAL as the operation identifier.

19.10 Causality in Action

Consider two nodes, A and B, in a three-node cluster. Node A is the leader.

1. Node A proposes a batch. It calls `SendOrLocalEvent`. The clock reads `physicalTime = 1000`, `currentL = 999`. It sets `newL = 1000`, `newC = 0`. The ticket is `(A, 1000, 0)`.
2. Node A sends the log entries to node B. The message carries `L = 1000`, `C = 0`.
3. Node B receives the message. Its local clock is at `(998, 3)`. It calls `ReceiveEvent` with `messageL = 1000`, `messageC = 0`. Its `physicalTime` is `999`. The three-way max gives `newL = 1000`. Only `messageL` matches `newL`, so `newC = messageC + 1 = 1`. Node B's clock advances to `(1000, 1)`.
4. Node B sends an acknowledgment. The ack carries `L = 1000`, `C = 1`.
5. Node A receives the ack and calls `ReceiveEvent`. Its clock was at `(1000, 0)`, `physicalTime = 1000`. The three-way max gives `newL = 1000`. Both `currentL` and `messageL` match, so `newC = max(0, 1) + 1 = 2`. Node A's clock advances to `(1000, 2)`.

Every step produces a timestamp strictly greater than the previous one. The causal chain is preserved: `(A, 1000, 0) < (B, 1000, 1) < (A, 1000, 2)`.

19.11 The Distinction: Terms vs. HLC Timestamps

Raft terms and HLC timestamps serve different purposes. Do not conflate them.

Property	Raft Term	HLC Timestamp
Purpose	Election epoch and log consistency	Unique proposal ticket identity
Scope	Cluster-wide consensus mechanism	Per-node clock with causality
Increments	Once per election	Once per event (propose, ack, heartbeat)
Required by Raft	Yes	No
Stored in	`RaftPartitionCoreState.CurrentTerm`	`HybridLogicalClock._state`
Comparison	Simple integer comparison	Three-component `(L, C, N)` order

A term change means a new leader election occurred. An HLC advance means an event occurred. A proposal carries both: the term for Raft correctness, and the HLC timestamp for ticket identity. If the term is wrong, Raft rejects the proposal. If the HLC timestamp were duplicated, the ticket system would malfunction. Both are necessary, but for different reasons.

19.12 The LC Class: A Simpler Clock

The `Kommander.Time` namespace also contains a simple `LC` class:

```
public sealed class LC
{
    private ulong ticks;

    public ulong Increment(int count) { ticks += (ulong)count; return ticks; }
```

```
    public ulong Increment() { ticks++; return ticks; }
    public ulong GetTicks() => ticks;
}
```

This is a plain monotonic counter with no physical-time component. It is not part of the HLC system. The `LC` class exists for use cases that need only a logical sequence number, not a causality-preserving timestamp.

19.13 Takeaways

1. Kommander uses HLC timestamps as unique proposal ticket IDs. The HLC is an implementation choice, not a Raft requirement.
2. An HLC timestamp combines physical wall-clock time (L) with a logical counter (C). L never decreases. C resets to zero when L advances.
3. The implementation packs `(L, C)` into a single 64-bit `long`. The high 42 bits hold L (milliseconds, sufficient until year 2109). The low 22 bits hold C (up to 4,194,303 events per millisecond).
4. All updates use a lock-free CAS loop on the packed `long`. Zero objects are allocated per clock event.
5. `SendOrLocalEvent` mints a new timestamp for outbound messages and local events. `ReceiveEvent` merges a remote timestamp with the local clock.
6. Counter overflow rolls `(L, C_max)` to `(L+1, 0)`. This preserves monotonicity. The clock self-corrects when the wall clock catches up.
7. `HLCTimestamp` is a `readonly record struct` with fields N (node ID), L (time), and C (counter). It implements a total order: `(L, C, N)`.
8. A single clock instance on `RaftManager` serves all partitions. Every consensus collaborator accesses it through `IRaftPartitionHost`.
9. Do not conflate HLC timestamps with Raft terms. Terms are the correctness mechanism. HLC timestamps are the identity mechanism.